# THE OMNI-COMPASS OPERATOR'S MANUAL

*How to Embed, Wire, Tune and Run Omni-Compass on Your Stack*

**AJ Dubra**
Owner and Developer

**THE OMNI-COMPASS LLC**
October 2026

**The Omni-Compass Operator's Manual**
How to Embed, Wire, Tune and Run Omni-Compass on Your Stack
October 2026

Copyright © 2026 The Omni-Compass LLC. All rights reserved. No part of this book may be reproduced, stored or transmitted in any form or by any means, except as allowed by the Omni-Compass Evaluation License, without the written permission of The Omni-Compass LLC.

The Omni-Compass LLC reserves the right to change this book, the software it describes and their terms at any time, without notice. Every copy, export and printout of any part of this book carries this page, the LICENSE, the NOTICE and the Disclosures unchanged.

All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC (no filing number is stated, and no grant, registration or approval is claimed). www.omni-compass.com

“Omni-Compass”, the Omni-Compass rose and the related names and marks are trademarks of The Omni-Compass LLC. Other names that appear in this book are the property of their owners and are used only to identify the systems Omni-Compass works with.

**License.** This book and the software it describes are proprietary and are not open source (SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0). They may be used only to evaluate Omni-Compass and to reproduce its published results, in simulation and on systems the reader owns or controls. Any commercial use, commercialization, monetization, production use, redistribution, hosted or managed service, or incorporation into any product or service requires a written Omni-Compass Enterprise License, signed by The Omni-Compass LLC and paid for. The full terms are printed in Appendix F.

**Disclaimer.** The software is provided “as is”, without warranty of any kind. This book is the wiring and operating manual and carries no benchmark results; every published result carries its evidence class. A modelled result is a statement about a model; a software benchmark is a statement about the software it ran on; only a hardware meter speaks for hardware. Nothing in this book is a promise of a particular saving on a particular system: the only number that applies to a system is the one its own paired runs produce, on its own receipt.

Set in Liberation Serif and DejaVu Sans Mono, the text at 11 pt on a 16.5 pt line. Built from the repository by `docs/book/build_book.py`; the book and the code change in the same commit.

The Omni-Compass LLC · www.omni-compass.com

# Contents

**Foreword** v

**Preface** vii

**Prelude: A Night on the Cluster** ix

**Introduction: The Problem, the Idea and What This Manual Covers** xi

**The key: GAIN, EVEN and LOSS** xiv

**What You Are Looking At** xvi

**Quick Reference: The Card for the Glove Box** xix

**Disclosures, Declarations and Disclaimers** xxii

**Part One. The Governor, from the Outside** 1

1. What Omni-Compass Is . . . . . 1
2. Your Stack as One Body: Muscles and Organisms . . . . . 5
3. Tuning: The Factory Tune, the Dial and the Settings You Write . . . . . 8

**Part Two. The Harness: Plugs, Wires and the Rules They Obey** 12

4. The Plug, the Adapters and the Wire Check . . . . . 12
5. The Rules the Wires Obey . . . . . 17

**Part Three. Wiring It onto Your Stack, Step by Step** 20

6. Before You Start, and the Eight Levels . . . . . 20
7. Stack by Stack . . . . . 26

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**Part Four. Operating It** **36**

8. The OFF Switch, the Rules, and the Log . . . . . 36

9. Maintenance, Upgrades and Security . . . . . 40

**Part Five. Checking It on Your Own System** **42**

10. Paired Runs and Receipts on Your Own System . . . . . 42

11. Evidence Classes and How to Read a Receipt . . . . . 45

12. The Engine Version and the Three-Run Rule . . . . . 47

**Part Six. License and Your Copy** **49**

13. License and Commercial Terms . . . . . 49

14. Verifying Your Copy . . . . . 50

**Back Matter** **51**

Glossary . . . . . 51

Appendix A. Command Reference . . . . . 55

Appendix B. The Files an Installation Touches . . . . . 57

Appendix C. The Gauges . . . . . 59

Appendix D. Troubleshooting . . . . . 62

Appendix E. The Muscles: What Each Is For, and How It Is Wired . . . . . 65

Appendix F. The License . . . . . 240

Appendix G. Licensing: Questions and Answers . . . . . 256

Appendix H. Third-Party Notices . . . . . 258

Contact . . . . . 260

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

# Foreword

I built Omni-Compass to sit on top of what already exists. Kubernetes, the GPU driver, the building controller, the battery inverter, the robot's servo loop: those are muscles, and they are good muscles. What none of them has is one governor that reads all of them at once, holds each of them inside its safe range, and puts every setting back exactly where it found it when it stops. That governor is Omni-Compass, and this book is how you put it in.

**Every machine already has a controller.** Kubernetes has its autoscaler. A graphics card has its firmware. A building has its thermostat loop, a battery has its charge controller, a power grid has its tap changers, and a robot has its servo. Engineers spent decades making these controllers safe, and they are. But each one was built to watch one thing and to protect that one thing. None of them sees the whole body. None of them knows that the cooling loop is holding back the processor, or that the autoscaler is adding machines the database cannot feed. So each one plays it safe in the middle of its range, because playing it safe is its whole job. The headroom between the middle and the edge is paid for every hour and almost never used.

**That gap is what you are wiring it to close.** Omni-Compass is a **supervisory governor**: one governor over the muscles you already run. It never replaces them and it never competes with them. It reads them, it decides, and it writes one setting at a time through a two-way plug: one wire in, one wire out. Before it writes anything, it takes a snapshot. When it stops, every setting goes back to that snapshot.

**The rule I will not bend: do no harm.** Omni-Compass must never cause a LOSS against the system's own native controller. Where it cannot bring a muscle a gain, that muscle stays native. Before a slow setting moves, the governor tries the step on that muscle and keeps it only if the muscle's own work costs no more for it. A step that costs more is undone.

**The max is the factory setting.** Omni-Compass ships tuned the way I tuned it: every muscle runs up to its own limit, read live from the system, never past it and never below what native gives. The energy it takes to do more work is **power given**, shown beside the work, the speed and the wear it bought. It is never called a loss. Anyone can turn the dial down from the max, retune a muscle, or add one; nobody has to.

**One pedal.** I wanted it to drive like an electric car. Push the pedal for more, lift off for less. Lift off all the way and the settings drift back toward native. Reset hands everything back. If the governor ever crashes, its lease runs out and the plug walks every setting home, step by step. The kill switch is separate and instant, and it exists for security, not for daily driving.

**What this book holds.** This is the manual for the people who wire it: what the governor is from the outside, how to tune it, how the plug and the wires work, how to wire it onto each kind of stack one level at a time,

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

how to run it, how to switch it off, and how to check on your own system, with your own paired runs, what it does there. It is the book you keep in the glove box. When the software changes, this manual changes in the same commit.

**How I want it judged.** On your own receipts. Every paired run has exactly two arms: native, and Omni-Compass on top of native. Run it three separate times under rules you wrote down first, show every row, losses included, and read each row as one word, **GAIN**, **EVEN** or **LOSS**, by what the gauge is for. Where a receipt reads LOSS, look at the wiring first; this book tells you where.

**Where to begin.** Read the front matter first. It tells you what you are looking at before it asks you to touch anything. Then wire it one level at a time, watch it before you let it write, and judge it by your own receipts.

**AJ Dubra** Founder, The Omni-Compass LLC

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

# Preface

I wrote this book for the engineers who put Omni-Compass into a running system: how to embed it, how to wire it to every muscle of a stacked, clustered system or to any one muscle, how to tune it, how to run it, how to switch it off, and how to check on their own system, with their own paired runs, what it does there. It is the manual for the glove box, not a description of how the engine is built.

The book is in six parts, after the front matter.

| You are | Read |
|---|---|
| CTO, architect, head of platform | the front matter, Part One, the eight levels in Part Three, then Part Five |
| The engineer wiring it | everything, in order. Do not skip the wire check or the watch level |
| Platform engineer for one stack | Part Two, the chapter of Part Three for your stack, and Part Four |
| Data-center, facility or energy operator | the Quick Reference card, Part One, Part Three (levels five to seven, the processor and the data center under the servers), Part Four |
| Site reliability engineer, on call | the Quick Reference card, Part Four (the switches and the log), the troubleshooting appendix |
| Security and compliance | Part Four, the Disclosures, the license in the back matter |
| Procurement and counsel | the Disclosures, Part Six, the license and its questions and answers in the back matter |

**Part One, the governor from the outside,** says what Omni-Compass is as an installer meets it, how the muscles of a stack become one body, and how it is tuned: the factory tune, the dial, each muscle's own card and the settings an operator writes. **Part Two, the harness,** is the universal plug, the adapters, the wire check and the rules every wire obeys. **Part Three** wires it onto a stack one level at a time and then stack by stack. **Part Four** is operating it: the OFF switch, the rules, the log, maintenance and security. **Part Five** is checking it on your own system: paired runs and receipts, evidence classes, the engine version and the three-run rule. **Part Six** is the license and how to verify a copy. The back matter holds the glossary, every command, the files an installation touches, the gauges, troubleshooting, and every muscle with its wiring and its own card.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**Conventions.** `Code` is a command, file or switch exactly as typed. "Native" means a system as it runs today, without Omni-Compass. "Muscle" means any machine, service or controller Omni-Compass can read and set. "Knob" or "lever" is one setting on a muscle. "The band" is a service reading's safe range. A step that writes to a system is marked **WRITES**. Every number in a receipt carries its evidence class: S for a model (modelled), L for live software, P for a physical meter.

**The standards every page of this manual keeps.** An installation wired by this manual should meet, and its receipts should show, the following discipline throughout:

1. **Two arms, always.** Every paired run compares native against omni, the same system with Omni-Compass on top, on the same machines, with the same work at the same moments. Omni-Compass is never run standalone and never compared as a competitor to the controller it sits on.
2. **Written down, then run.** The arms, the knob, its cover, the gauges, their aims and the readings are written down before the first counted run. Rules are fixed on one tuning case that is shown and never counted, then applied unchanged to the cases that are counted. A change made after a result has been seen is written down with its time and reason, and the receipt that carries the result says so.
3. **Three separate runs, one rule.** A result is run three times (A, B and C) on the same engine version, and every judged row reads exactly one of: GAIN in all three runs, LOSS in all three runs, no difference beyond the noise, or the runs disagree (chapter 12). Nobody outside The Omni-Compass LLC has reviewed or approved the published results, and every three-run table says so.
4. **Every row, losses included.** A row that went against Omni-Compass is in the receipt beside the rows that went for it. When Omni-Compass loses, the first suspect is the wiring, then native's setup, then the scoring; when that is fixed, the run is repeated and the loss stays in the record.
5. **Nothing read across engine versions.** Each engine version is fingerprinted (`python3 tools/omni_version.py`). A receipt names the version it ran on and is never read against a receipt of another.
6. **Do no harm.** Omni-Compass moves a slow knob only where a paired trial on the muscle itself shows the muscle never behind for it (section 1.2). Where it cannot bring a knob a gain, that knob stays native, and the receipt says "nothing for Omni to move".

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

# Prelude: A Night on the Cluster

This is a story, not a result. It follows one night on one cluster to show, in plain order, what each part of this book does. The shop, the times and the settings are made up for the story. A measured result belongs to a receipt, and chapter 10 says how to make one on your own system.

**9:00 p.m. The shop is quiet.** An online store runs on Kubernetes. Its native controllers are the ones almost everybody uses. The Horizontal Pod Autoscaler watches the average processor load and adds copies of the web service (pods) when that load passes its target. The cluster autoscaler adds machines when the pods no longer fit. A careful engineer picked their settings months ago, for the worst day she could imagine, and she picked them well. Tonight the cluster runs on its floor of two machines, and every customer gets an answer in a blink.

**A week earlier, Omni-Compass went on in watch mode.** For seven days it read every gauge and wrote nothing. It learned how fast each pod starts, how the response time moves when the load moves, and where each setting's safe band lies. Then the team ran the wire check: every reading came from the place it claimed, every lever moved the thing it claimed, and every write was read back. Only then did they turn it to govern. Every setting it would touch was saved in a snapshot first.

**9:40 p.m. The sale starts.** In two minutes the requests double. The native autoscaler does exactly what it was built to do, but it is built to wait: it needs the average load to climb, then it needs a decision, then each new pod needs time to start. While it waits, customers wait. With Omni-Compass on top, the compass sees the response time slide toward the edge of its band and into the cushion. It pushes. It raises the autoscaler's floor of pods, a setting the autoscaler already respects, so new pods start before the average catches up. It does not fight the autoscaler and it does not go around it. It moves one setting, through the plug, and reads it back.

**10:15 p.m. The database starts to strain.** More web pods mean more connections to the database, and the database has a limit of its own. A controller that watches only the web service would keep adding pods and make the jam grow. Omni-Compass sees the whole body. One body, one brain: before it takes the next step for the web service, it checks what that step would do to every other part. The step that would push the database past its own cushion is refused. Nothing is pushed into a loss to buy a gain somewhere else. The web service holds where the database can still keep up, and the queue drains instead of growing.

**11:30 p.m. The rush fades.** The pedal lifts. Omni-Compass does not drop everything at once, because a sudden drop is how systems stumble. It eases each setting back toward native along a smooth curve. The extra machines go away as the work goes away. The energy the cluster used during the rush shows up in the report as power given, beside the work it bought and the minutes customers did not spend waiting.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**1:00 a.m. Idle.** The pedal is lifted all the way. The cluster sits on its floor of two machines again, exactly where the engineer left it. Omni-Compass keeps reading every second. Staying native is its live decision, made again each second, not a verdict from some earlier run.

**3:12 a.m. Something breaks.** An upgrade restarts the machine that runs the governor, and Omni-Compass stops in the middle of its work. Nobody is awake. Nothing bad happens. The governor's lease runs out, and the plug walks every setting back to its snapshot, step by step. The native controllers keep running as they always have, because they never stopped. When the governor comes back, it starts from what it reads, not from what it remembers.

**7:00 a.m. The morning report.** On a live system, the proof does not come from a feeling that the night went well. It comes from paired runs: the same work, run once on native and once with Omni-Compass on top, and compared gauge by gauge. Every gauge gets one word. Time waited that fell reads **GAIN**. Work done that rose reads **GAIN**. A difference inside the noise reads **EVEN**, and the report says so plainly. A gauge that moved the wrong way reads **LOSS**, and it stays in the report until its cause is found and fixed. The energy spent on extra work reads **power given**, never a loss.

**What the night shows.** Every part of that night is a chapter of this book:

- the band, the cushions, the line and the factory tune are in chapter 3;
- the plug, the snapshot and the wire check are in chapter 4;
- one body, one brain, and blind means hold, are in chapter 5;
- the watch mode and the levels are in chapter 6, and the cluster's own wiring in section 7.1;
- the lease and the kill switch are in chapter 8;
- the paired runs and the three words are in chapters 10 to 12.

Read the rest of the front matter before you touch a wire. It tells you what you are looking at and how to read what it tells you.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

# Introduction: The Problem, the Idea and What This Manual Covers

**The problem.** Production systems are governed by many local feedback loops: horizontal autoscalers, firmware power and clock governors, building-management loops, battery-management systems, connection pools, servo controllers. Each loop is tuned in isolation, against its own setpoint, with margins sized for a worst-case disturbance. Three costs follow, and every operator pays them:

- **Reaction latency.** Reactive loops act on lagging, averaged signals. A Kubernetes Horizontal Pod Autoscaler recomputes its desired replicas from averaged utilization on a sync period (15 seconds by default) and damps its decisions with stabilization windows; new pods then need time to schedule and start. Queues form before capacity arrives.
- **Static over-provisioning.** Worst-case margins persist through the far more common nominal regime. The operating point sits well inside the safe region, and the headroom between it and the constraint boundary is paid for and rarely used.
- **Uncoordinated coupling.** Loops share resources: power, cooling, connections, memory, network. One loop's correction is another loop's disturbance. More web replicas saturate the database's connections; more cooling pushes a power feed toward its limit.

Operators close these gaps by hand, with fixed settings and runbooks. That is safe, and it leaves the region between the nominal operating point and the safe boundary unused nearly all of the time.

**The idea.** Close one supervisory loop around the native loops. Omni-Compass treats each setting a native controller already exposes (a replica floor, a power cap, a clock ceiling, a temperature setpoint, a pool size) as an actuator and each native telemetry stream as a measurement, and governs the whole body from above. Seen from the outside, every decision period runs the same five steps, and those five steps are all an installation has to provide for:

1. **Sense.** It reads each muscle's own gauges through the plug: utilization, response-time percentiles, queue depth, temperature, power, state of charge.
2. **Place.** It places each reading in its band, from calm to the line, with a 5% cushion at each edge, so that every muscle's state is read on the same scale whatever its units.
3. **Decide.** It chooses at most one bounded step per actuator, smooth and never hammering, clipped to the actuator's cover, the hard range nothing can leave.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

4. **Actuate.** It writes the step through the plug, idempotently, and reads the value back to confirm the write landed.
5. **Verify.** It compares the muscle with and without the step in a paired trial on the muscle itself, judged on the muscle's own cost per unit of work, and checks every coupled muscle against its own cushion. A step that fails either test is undone.

The installation supplies the wires (sense and actuate) and the settings (the band, the line, the cover and the tune); the governor does the rest. Nothing in an installation re-implements the decision, and nothing in this manual asks the installer to.

**What Omni-Compass is not.**

- It is **not a replacement** for your controllers. Native keeps running underneath, always. Omni-Compass only moves settings that native already exposes and respects.
- It is **not a black box**. Every decision is written to a log, with the reading that caused it and the setting it changed (chapter 8).
- It is **not a one-time tuning script**. It decides again every second, and staying native is one of its decisions.
- It is **not a cost-cutter first**. It gives power where the power buys work or speed, and it shows that power beside what it bought.

**The promise, in three parts.**

1. **Do no harm.** A step stays only where the muscle loses nothing for it, measured on the muscle itself. Where Omni-Compass cannot bring a gain, the muscle stays native.
2. **Everything comes back.** Every setting is saved before the first write and restored when Omni-Compass stops. If the governor crashes, its lease runs out and the settings walk home on their own. A separate kill switch stops everything at once.
3. **Every claim is checked.** Each paired run compares two arms, native and Omni-Compass on top of native, on the same machines with the same work, three separate times. Each reading is one word, GAIN, EVEN or LOSS, decided by what the gauge is for, and each carries its evidence class: **P** for a physical meter, **L** for live software, **S** for our own models. A model is never shown as proof of hardware.

**What it governs.** The catalog names 1,107 muscles in 70 families, the settings that operators everywhere already turn by hand: machine pools and replicas, power caps and clock limits, cache sizes and connection pools, cooling setpoints and battery reserves, robot speeds and drone-swarm settings, and many more. Appendix E lists every one: what it is, what Omni-Compass reads from it, what it moves, and the factory setting it runs on. Chapter 2 shows how the muscles of one stack are grouped into one body.

**What you need before you wire it.**

- **A license for any use beyond evaluation.** Production use requires a signed, paid Omni-Compass Enterprise License.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Read access first.** Omni-Compass starts in watch mode: it reads and computes, and writes nothing.
- **The wire check.** Every reading must come from where it claims, every lever must move what it claims, and every write must read back. Nothing writes until the check passes.
- **Your own service line.** The response time or other limit your service must meet comes from your own service objectives, not from ours.
- **Paired runs on your own system.** The only number that applies to your system is the one your own paired runs produce, on your own receipt.

**What this manual covers, and what it does not.** It covers how to embed, wire, tune, run and check Omni-Compass on a stacked, clustered system or on any one muscle, the way a vehicle's manual covers its controls, its settings, its service and its warning lights. It does not cover how the engine is built, and it carries no benchmark results: those are kept with the code, each in its own table with its evidence class.

**How to read what follows.** Next comes the Quick Reference card, the page to keep beside the system. Part I describes the governor from the outside and how to tune it; Part II, the plug and the rules its wires obey; Part III, the wiring, level by level and stack by stack; Part IV, operating it; Part V, checking it on your own system. Read in that order, nothing in the wiring chapters should surprise you.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

# The key: GAIN, EVEN and LOSS

Every judged reading in Omni-Compass is one of three words: **GAIN**, **EVEN** or **LOSS**. Better, same and worse are not used anywhere. The word is decided by what the gauge is for, never by the plus or minus sign of its number: who sets the aim differs from one system to the next, and each gauge's aim is written in its preregistration before the runs.

| Word | What each letter stands for | Its sense in underwriting | What it means in a table |
|---|---|---|---|
| **GAIN** | **G** Growth (asset appreciation); **A** Advantage (risk reduction); **I** Improvement (margin expansion); **N** Net-positive (profitable outcome) | surplus and positive variance | the gauge moved toward its declared aim against native: more work done, a shorter wait, fewer misses or failures, or a resource cut |
| **EVEN** | **E** Equalized (zero variance); **V** Verified baseline (expected norm); **E** Equilibrium (balanced risk); **N** Neutral (no net change) | the break-even baseline: equilibrium | native and Omni could not be told apart: the same, or no difference beyond the noise; a knob native already runs at the top of its range is EVEN, at its limit on native |
| **LOSS** | **L** Leakage (capital drain); **O** Off-track (adverse selection); **S** Setback (claim liability); **S** Subpar (negative variance) | actuarial deficit and exposure | the gauge moved away from its declared aim: less work done, a longer wait, more misses or failures |

How the sign of a number becomes a word:

- **Time waited, missed or failed** (a response time, time over the line, failed requests): a fall is GAIN, a rise is LOSS.
- **Work done** (work inside the line, throughput, a hit rate): a rise is GAIN, a fall is LOSS.
- **A resource spent or held** (energy, machines, memory, processor time, a bill, wear on a part): a fall is a **cut**, and a cut is GAIN. A rise is **power given**: what Omni put in to buy the work and the speed shown beside it. Power given is always shown and never hidden, and it is never called a loss (the founder's order of 12 October 2026: the max rides every muscle's edge; it never shaves).
- So a minus can read GAIN (a response time cut 65%, a bill cut 30%) and a plus can read LOSS (failed requests up).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

Some preregistrations name their outcome with a label written before the runs (for example SUPERIOR WITHIN GUARDRAILS in the card protocol). The label stays as it was frozen, and every table shows it with its word first: TOWARD ITS AIMS reads GAIN; BOTH WAYS reads GAIN AND LOSS; AWAY FROM ITS AIMS reads LOSS; SAME AS NATIVE reads EVEN (the same as native); AT ITS LIMIT ON NATIVE reads EVEN (at its limit on native); SUPERIOR WITHIN GUARDRAILS reads GAIN (within the guardrails); ENERGY IMPROVEMENT WITH SERVICE TRADEOFF reads GAIN AND LOSS (energy cut, service traded for it); SERVICE IMPROVEMENT WITH ENERGY TRADEOFF reads GAIN, power given (service GAIN, energy given for it); NONINFERIOR / INCONCLUSIVE reads EVEN (inside the noise, within the guardrails); NOT ESTABLISHED reads EVEN (not established: inside the noise, a guardrail crossed on the mean). A run that breaks its own validity checks reads INVALID and carries no word.

A reading counts only when it holds the same way in all three of our runs, A, B and C, with every 95% interval clear of zero: "GAIN in all three of our runs". The other readings are **EVEN, no difference beyond the noise** (in a run the interval includes zero, so native and Omni could not be told apart: that is the result), **the runs disagree** (clear runs pointing different ways: the test is unstable there and is looked into) and **shown, not judged** (a gauge declared in advance as shown only, such as the knob's own setting).

Why these three words: they are the words of underwriting and actuarial work, where every outcome is graded as surplus, break-even or deficit, and each letter spells out the same rule. Omni-Compass is there to add (GAIN), never to take (LOSS), and to hand a muscle back to its own native setting where it cannot add (EVEN). Its first rule is do no harm, so **LOSS should never appear**: where it does, the row stays in the table, its cause is found and fixed (our own wiring first, then native's setup, then the scoring) and the test is run again.

Disclosure and disclaimer: GAIN, EVEN and LOSS, and the letters behind them, are The Omni-Compass LLC's own convention for reading its own results. The underwriting senses explain the words; they are not an actuarial, financial or insurance opinion, and no outside body defined, reviewed or approved them. A reading states what our own runs measured against native under the declared aims, not a promise about any other system; every number stands beside its word, so a reader can judge it by any other convention. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results. Preregistered means the rules were written down in our own repository before the runs; nothing was filed with any outside body.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

# What You Are Looking At

**What Omni-Compass is.** Omni-Compass is a supervisory governor. It is never a controller of its own and never a replacement for one: it sits on top of the controllers a system already runs (the Kubernetes autoscaler, a database's connection pool, a GPU's firmware, a building's battery controller, a power grid's tap changer, a robot's servo) and moves only the settings those controllers already expose to an operator, through one wire in (what it reads) and one wire out (what it writes). Everything the native controller does, it still does. Pull the plug and every setting walks back to where the operator left it (the lease); the kill switch returns them at once.

**What "native" means.** Native is the same stack, the same muscles, the same machines and the same work, run the way everybody runs it today: the shipped controllers at their shipped or operator settings. Every paired run has exactly two arms, native and omni, where omni is that same native stack with Omni-Compass on top. Nothing else differs between the arms, so whatever differs in the result is what Omni-Compass did.

**What a muscle and an organism are.** A muscle is one setting a native controller exposes (a pool's size, a power cap, a clock ceiling, a replica count, a battery's charge rate) together with the gauges that show what it does. An organism is a stack of muscles governed as one body: the muscles of one company's clusters, databases, caches, cards, cooling and power, read by one governor and judged together, so that no muscle is moved at another's expense. Chapter 2 sets out how a stack is grouped into one body; Appendix E lists every muscle Omni-Compass can be wired to, with what it reads and what it moves.

**What you see when it is hooked up.** Every paired run ends in a receipt with the same shape: the two arms side by side, every gauge of the system (the work done, the response times, the misses and failures, the machines and energy spent), the paired change with its 95% interval, and one word per gauge, GAIN, EVEN or LOSS (the key, above). A three-run table repeats the paired run three times, A, B and C, on the same engine version, and a word counts only when all three runs agree (chapter 12).

**How to read a receipt, in order.**

1. **The engine and the evidence class.** The header names the engine version (`python3 tools/omni_version.py`) and the class (P, L or S; chapter 11). A receipt is never read across engine versions, and S is never read as proof about real equipment.
2. **The work and the service rows first**: work done, response times, time over the line, failed requests. These say what the system delivered. A GAIN here is the point of the governor; a LOSS here is the first thing to look for.
3. **Then the resource rows**: machines, energy, memory, connections. A cut is a GAIN. A rise is **power given**: what Omni-Compass put in to buy the work and speed above it. Read the two together: power

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

given that bought nothing is a cost to find and fix, starting with the wiring.

4. **Then the noise.** EVEN, no difference beyond the noise, is a result, not a failure. The runs disagree means the test is unstable there and is looked into.
5. **Then the clock and the machine.** Where the test machine itself is the limit (its processor saturated, native alone failing a large share of requests), the receipt says so; a test machine's limit is never read as the governor's result, and the test is rebuilt so the machine no longer decides it.
6. **Every row is shown**, LOSS included. Nothing is cut from a receipt because it went against Omni-Compass.

**How the governor tells GAIN from LOSS.** The governor does not read a plus or a minus sign as good or bad. Before it moves anything, every gauge it reads is declared with its aim, and the aim decides the word: for time waited, missed or failed a fall is GAIN; for work done a rise is GAIN; for a resource spent or held a fall is a cut, and a cut is GAIN, while a rise is power given, kept only while it buys the work or the speed shown beside it. A slow knob moves only as far as a paired trial on that muscle has shown costs its own work nothing (section 1.2); a step that costs more is undone, and where no step passes the muscle stays native, which is EVEN. Inside a body, a step that would push another part past its cushion is refused, so no GAIN in one muscle is bought with a LOSS in another (chapter 5).

**Wiring it: who, what, when, where, why and how.** Read this before the wiring chapters; a governor wired without it may run and still not be the mechanism.

- **Why.** To get more work, faster, with fewer misses, from the machines and the energy already paid for, by letting one governor move the settings native controllers already expose, in the direction each gauge's aim asks, and only while no other gauge pays for it. If nothing can be gained, the purpose is met by leaving native alone.
- **What.** Two things per muscle and nothing else: one wire in (the gauges that show the muscle's state and its work, read live from the system, never typed into code) and one wire out (the one setting native already exposes to an operator, with its hard range read from the system). The governor does the arithmetic; the wiring never re-implements it.
- **Where.** On top of the native controller, never in place of it: on the autoscaler's target and ceiling, not on the pods; on the pool's size, not on the queries; on the card's power limit, not on its firmware. Every muscle of the stack is wired into the same governor, so the body is judged as one.
- **Who.** The operator who owns the system grants each wire, decides which knobs Omni-Compass may write and which it only watches, and keeps the kill switch. The governor writes only what it was granted; the native controller keeps doing its own job.
- **When.** Watch first: the governor reads every wire and writes nothing until the wire check passes and paired runs on the operator's own system show the receipt. Then one level at a time (the eight levels, chapter 6). It reads and decides once a second, all the time; staying native is a live decision, made again every second, never a setting left on.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **How.** The plug snapshots each setting once, at the start; the governor reads every gauge, places each muscle in its band, tries one step on one muscle at a time inside a body, keeps it only if the paired trial shows a GAIN and no LOSS, and hands every setting back to the snapshot on reset, on the lease after a crash, or at once on the kill switch. The proof is the receipt: native and Omni-Compass on the same work, every gauge, three runs, every row shown.

**What the end result is.** A system fully hooked up runs native exactly as before, with one governor reading every gauge of every muscle once a second and moving, muscle by muscle, only where a paired trial has shown a GAIN with no LOSS anywhere in the body; where it cannot help, the muscle stays native, which is EVEN, not a failure. The intended outcome, stated as the claim the receipts test: more work, faster, with fewer misses, on the same or fewer machines, with every unit of power given shown beside what it bought. Whether a given system gets there is answered only by its own receipts.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

# Quick Reference: The Card for the Glove Box

Keep this page where the people who run the system can reach it, the way a driver keeps the card in the glove box. Every line is explained in the section named beside it; nothing on this page replaces those sections.

**What it is.** A supervisory governor on top of the controllers you already run. It never replaces them: it moves only settings they already accept, inside a band you set, and hands every setting back when it stops (sections 1.1 and 4.1).

**The two names.** *Native* is your system as it runs on its own. *Omni* is the same system with Omni-Compass on top. There is no third arm and no other name (section 1.1).

**The modes** (section 1.3).

| Mode | What it does | When |
|---|---|---|
| Off (manual) | nothing; native runs alone | before installing, and whenever you choose |
| Watching | reads every gauge and logs what it would do; writes nothing | the first level of every installation (level 1) |
| Autopilot | one pedal: push for more, lift off for less, idle (lifted all the way) and reset, by its law | after the watch level has passed |
| Cruise | machines in service up to the pool's own top while a known queue waits | batch work |

**One pedal**, like an electric car's. *Push*: capacity added at once as traffic climbs, never held back. *Lift off*: capacity eased off a step at a time, handed back toward native, never below idle. *Idle*: the pedal lifted all the way, the floor of two machines in service, ready for the next burst. *Reset*: every setting handed back to where native had it, read back, and the record removed.

**The ways it stops, and what each leaves behind** (chapter 8).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| You want to, or it happens that | Do this | What is left |
|---|---|---|
| a run ends | nothing: the reset runs by itself at the end of every run | every setting at the operator's value, read back |
| stop everything at once, for any security reason | `python3 tools/omni_switch.py off` | every governor hands back and exits; none starts again until `python3 tools/omni_switch.py on` |
| stop one muscle | its kill file, or `OMNI_KILL=1` | that muscle handed back; the others run on |
| Omni-Compass dies without handing back (killed, crashed, hung) | nothing: the watchdog's lease hands back for it | every setting back at the operator's value, read back, as soon as the watchdog finds the process gone or its lease run out (chapter 8) |

**The eight levels of an installation** (section 6.3). Take them in order; go to the next only when the pass holds.

| Level | What it touches | Pass |
|---|---|---|
| 0 | nothing: `python3 verify.py` | `VERIFICATION: PASS` |
| 1 | nothing: watch mode, with a read-only identity | zero writes, and readings your operators agree with |
| 2 | the pods, through your autoscalers (**WRITES**) | p95, p99 and failed requests never behind native, over paired runs (chapter 10) |
| 3 | the machines, through the release gate and the verdict (**WRITES**) | fewer machines in service, with no LOSS on any service gauge |
| 4 | the replica decision itself (an option, not a step) | as level 3 |
| 5 | a GPU box, two wires: the clock ceiling and the power limit (**WRITES**) | work per energy up or equal; no request more than 2% slower at the median, p95 or p99 |
| 6 | the processor's clock and power on bare metal (**WRITES**) | energy per unit of work down, no LOSS on any service gauge |
| 7 | site power and the supply-air setpoint (**WRITES**) | the site meter's energy down, every rack inlet inside its band, no LOSS on any service gauge (section 7.10) |

**Before any level that writes:** run the wire check (section 4.4), start in watch, and keep the master switch in reach.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**Warning signs, and the first thing to do.**

| You see | First | Then read |
|---|---|---|
| a service slower or failing more with Omni than without | `python3 tools/omni_switch.py off`: the system is native again | section 4.5, wired right or wired wrong |
| a setting at a value Omni-Compass did not write | nothing: it stops writing that lever by itself (one writer) | section 5.2 |
| a meter that is blind or late | nothing: blind means hold; it moves nothing it cannot see | section 5.3 |
| your results point the other way from the published ones | presume the wiring first, not the law | `DISCLOSURES.md`, Chapter 4 |

**Who to call.** Licensing, pilots and the Omni-Compass Enterprise License: The Omni-Compass LLC, main office infrastructure, toll-free direct 1-888-339-9551, system route asset 1-888-DEX-YLL-1, www.omni-compass.com. Security reports: the process in `SECURITY.md`, never a public issue.

**What the letters of the number mean.** Read in order, DEX-YLL-1 is the way the engine runs:

| DEX | YLL | 1 |
|---|---|---|
| **Data Exchange**: the input. In software, data exchange is how one system's readings reach another. Here it is the two-way plug, one wire in and one wire out: the system's own readings come in, Omni-Compass's settings go out. | **Yield Limit Line**: the boundary. In engineering science, the yield limit is the most load a material can carry and still spring back unharmed; past it, the damage is permanent. Omni-Compass runs every muscle up to its own yield limit line, read live, and never past it. | **1**: the output. In code, 1 is true, the finished state; in science, 1 is unity, the whole. Here it is one closed loop: one body, one brain, every result read back and the next step decided from it. |

Read as one sentence: Omni-Compass takes the system in through a Data Exchange (DEX), runs every setting up to its Yield Limit Line (YLL) and never past it, and closes it all into one loop (1). Science and software people read each part on sight.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

# Disclosures, Declarations and Disclaimers

This page is the one place every declaration about Omni-Compass is made. The README, the manual, the book, every guide and every report point here. Where any other page seems to say more than this page, this page governs.

## Rights

1. Omni-Compass, its engine, its mathematics, its software and its documentation are the property of The Omni-Compass LLC. All patents, copyrights and trademarks covering them have been filed in the United States by The Omni-Compass LLC. No filing number is stated here, and no grant, registration or approval is claimed.
2. The software is licensed for evaluation and simulation only (`LICENSE`). Running it in production, on any system beyond evaluation, or in any product or service, requires a signed, paid Omni-Compass Enterprise License.
3. Names of other companies and products (NVIDIA, Kubernetes, Red Hat OpenShift, Amazon EKS, Google GKE, Microsoft AKS, Karpenter, Lambda and others) are the property of their owners. They identify the systems Omni-Compass was tested with or connects to. No affiliation, endorsement or certification by any of them is stated or implied.
4. All rights reserved. The Omni-Compass LLC reserves the right to change the software, the documentation, the results and the terms at any time, without notice. Every copy, fork, export, report, archive or printout of any part of Omni-Compass carries the README's notice, `LICENSE`, `NOTICE` and this page unchanged. The Omni-Compass LLC, www.omni-compass.com

## What a result is, and what it is not

1. Every result carries its evidence class: **T** a theorem, **V** verified in code, **S** our own models (modelled), **L** live software (real Kubernetes on GitHub's machines), **P** a physical meter (a real card's own power meter). A model is a statement about the model. A software benchmark is a statement about the software and machine it ran on. Only a meter speaks for hardware.
2. The models were written by the same people who wrote the law. They show how the mechanism behaves; they are not an independent test.
3. On Kubernetes in kind every machine stays powered, so energy there is a declared model, not a meter.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

4. Every comparison is paired: the same seed, load and clock with and without Omni-Compass, in an order rotated by repetition. Intervals are 95% intervals over the paired repetitions. Ratios across seeds are summarised as geometric means. Labels are assigned by code from rules written and committed before the run (`docs/GPU_PREREGISTRATION.md`, `docs/REALMS_PREREGISTRATION.md`, `docs/K8S_COMPASS_PREREGISTRATION.md`), never by hand.
5. Losses and failed runs are kept beside the wins (`docs/EVIDENCE_LEDGER.md`, `results/live/`, git history). The first real-card run, for example, saved energy (+3.6% work per energy, proven) and made the slowest answers 58.5% slower; it is published as it happened.
6. No result is a promise of a saving on any other system. No dollar figure in this repository is a measured result. Nothing here is a vendor certification, a safety certification, legal advice or investment advice.

## The rule Omni-Compass keeps on every muscle

Omni-Compass sits on top of a muscle that already works. By default it moves the muscle's settings toward more work, faster, up to the muscle's own limit and never past it (the max); on the dial anyone can tune it down toward the same work with less (the tune-down). Either way it keeps a step only where the muscle is never behind native for it: a step that costs the muscle more for each piece of work, or loses more than **2%** in any measure (work done, response time at the median, p95 or p99, time over the service line, or the machine's own cost of running Omni-Compass), is undone. A resource spent that rose is **power given**, shown beside the work, the speed and the wear it bought, never called a loss. Where no move passes that test, Omni-Compass leaves the muscle exactly as it runs alone.

**The factory tune.** The engine in this repository is presented as The Omni-Compass LLC tuned it and ships it: a self-running engine, set at the factory. Every muscle in the catalog is wired, and each runs on the setting found for it: its own card where one was found on development seeds and kept under the strict rule (364 muscles, `realms/muscle_cards_v5.csv`), the engine's own setting for every other muscle, all frozen and fingerprinted by version (`python3 tools/omni_version.py`). Muscles differ: no single setting is right for every muscle, family, sector, domain or realm, and none is presented as universal. Every published result is this factory tune. As with any engine, a licensee may then add on, retune, upgrade or tune down any muscle or any combination of muscles (the dial, the cards, the bands and the service lines are all exposed); a changed tune is the licensee's own configuration, and its results are its own, not the published ones. On a live system the verdict (`omnicompass/verdict.py`) measures it with paired trials on the muscle itself before every step. Every result reports each measure, and where one read LOSS at all, by however little, the result says so and by how much.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## The wiring declaration

**THE PUBLISHED BENCHMARKS ARE THE RECORD OF WHAT OMNI-COMPASS DID ON THE SYSTEMS, LOADS AND ENGINE THEY NAME, EACH WITH ITS EVIDENCE CLASS. WHERE A RESULT ON ANY OTHER SYSTEM DIFFERS FROM THEM, OR ANYTHING LOOKS WRONG, THE FIRST PRESUMPTION IS THE WIRING, THE SETTINGS, OR HOW OMNI-COMPASS WAS IMPLEMENTED OR READ, NOT THE LAW. CHECK THOSE FIRST (THE MANUAL'S "WIRED RIGHT OR WIRED WRONG") BEFORE DRAWING ANY CONCLUSION ABOUT OMNI-COMPASS.**

**Omni-Compass acts only through the wires it is given.** It reads the meters it is pointed at and moves the settings it is allowed to move. If a reading is the wrong one, slow, or blind; if a lever is the wrong one, already owned by another controller, or given a range (cover) that is wrong for the machine; if a band or a service line is set for a different workload; then Omni-Compass will do exactly what its law says with the wrong information, and the result will not be the published one.

Therefore:

1. **It cannot be slapped on.** Every installation goes through the levels of the manual in order (chapter 6), starting read-only, and every writing level begins with the wire check (manual, section 4.4) and a watch arm that must equal native.
2. **If your paired receipts differ from the published benchmarks in direction** (service LOSS, or no saving where one was shown), **the first presumption is wiring, not the law.** Confirm the installation with the checks in the manual's section 4.5 (*Wired right or wired wrong*) before drawing any conclusion about Omni-Compass.
3. **This has happened to us, and is published.** On the first real card (NVIDIA A10, 2026-10-02) the governor read being busy as trouble, set its power lid under the card's own working draw and allowed the clock under the card's own working clock; the card served bursts more slowly and the slowest answers were 58.5% slower. The cause was found in the card's own samples and corrected (`docs/GPU_PREREGISTRATION.md`, amendments 6 and 7). That is what a wiring fault looks like, and how it is found.
4. Omni-Compass never writes outside a lever's cover, never fights another writer (it stops and leaves that lever alone), reads back every write, and returns every lever to the value it read before its first write when it stops or when the OFF switch is used. These protect the machine; they do not make a wrong wiring right.

## Safety and responsibility

1. Run watch mode first; run the wire check before any write; keep the master OFF switch (`python3 tools/omni_switch.py off`, the whole harness at once) in reach at every level, and use it at the first sign of anything wrong, a suspected breach included.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

2. Omni-Compass is a supervisory governor. The machines' own controls (firmware, autoscalers, safety systems) stay in place and keep their own protections; Omni-Compass sets only values they already accept.
3. Do not connect Omni-Compass to safety-critical systems (vehicles, medical devices, grid protection, life safety) except as a modelled study, and never without the qualified review those systems require.
4. The software is provided "as is", without warranty of any kind, as stated in `LICENSE`. The operator is responsible for its use on their systems.

## Changes to the work and to this page

1. The software, the law's settings, the benchmarks, the results, the documents, the licensing terms and this page may change at any time, without notice. The version in the repository's `main` branch at a given commit is the version that commit describes; `RELEASE_MANIFEST.json` and `python3 verify.py` identify it.
2. Statements about work in progress, planned features, expected results and future runs are expectations, not promises. Where a result has not yet been measured, the documents say so (`docs/STATE_OF_PLAY.md`, `docs/DOSSIER.md`, Chapter 8).
3. A signed Omni-Compass Enterprise License governs its own terms for its own term.

## Privacy and data

Omni-Compass contains no telemetry. It sends nothing to The Omni-Compass LLC or anyone else, and makes no network connection of its own except to the systems an operator points it at. Its logs and receipts stay on the operator's machines.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## Where everything is

| Question | Page |
|---|---|
| What has been measured, and how strongly | `docs/STATE_OF_PLAY.md`, `docs/DOSSIER.md`, `docs/EVIDENCE_LEDGER.md` |
| The rules written before each run | `docs/GPU_PREREGISTRATION.md`, `docs/REALMS_PREREGISTRATION.md`, `docs/K8S_COMPASS_PREREGISTRATION.md` |
| How to wire it, level by level, and how to confirm it is wired right | `docs/OMNI_COMPASS_MANUAL.md` (chapters 4 and 6), the book `docs/OMNI_COMPASS_MANUAL.pdf` |
| The license, in plain answers, and third-party components | `LICENSE`, `LICENSING_FAQ.md`, `ENTERPRISE_LICENSE.md`, `NOTICE`, `THIRD_PARTY_NOTICES.md`, `PATENTS.md`, `TRADEMARKS.md`, `SECURITY.md`, `.github/CLA.md` |
| The muscles, what each is for and how it is wired | `docs/MUSCLE_CATALOG.md` |
| Proof the code is the code that ran | `python3 verify.py`, `RELEASE_MANIFEST.json`, `results/SEAL.json` |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

PART ONE

# The Governor, from the Outside

*What Omni-Compass is as an installer meets it, how the muscles of a stack become one body, and how it is tuned.*

CHAPTER 1

## What Omni-Compass Is

A governor on a steam engine does not build the engine and does not turn the shaft. It watches the speed and moves the throttle so the speed stays in a band. Omni-Compass is that, for every machine you run.

It is a process on a host. It reads meters, decides, and writes only the levers it has been given. It is not the chip, not the GPU driver, not Kubernetes, not the building controller. Those keep running exactly as they do today; Omni-Compass sets the values they already accept.

Three facts about that process fix what an installation can and cannot expect of it. First, it is **one process per stack**: a controller that wakes on a fixed period, reads the stack's own gauges through the stack's own interface, decides, writes through the same interface, and reads back what the device took. It has no agent inside the pods, no kernel module and no hook in the request path, so it can add latency to nothing it does not govern. Second, every lever it may touch is **named before it starts**, read once into a snapshot, and handed back to that snapshot when it stops, is killed, or loses a sense; a lever it was not given it cannot reach, because the plug for it does not exist. Third, it has a **watching mode** in which it reads and decides and writes nothing, and every rollout begins there (chapter 6): the first thing an operator learns about Omni-Compass on their own stack is what it would have done, in a log, with the stack untouched.

What follows from those three is the shape of every receipt in Part V. Because the native controller keeps running underneath, every paired run is the native stack against the same native stack with Omni-Compass on top, and never Omni-Compass against anything else. Because the levers are the stack's own settings, every gain is one the operator could have reached by hand, if they had been able to read every gauge every second and move every knob without ever being wrong; the claim is not a new mechanism but a steady hand on the mechanisms that exist. And because the process is small and separate, its own cost (processor time on the host, reads of the API) is recorded in every run and counted against it where it shows.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*
**www.omni-compass.com**

## Always on top of a native controller

The first design decision, and the one every other follows from, is that Omni-Compass is **supervisory**. It never replaces the controller a system already has, never bypasses it, and never runs in its place. The Kubernetes Horizontal Pod Autoscaler keeps scaling pods on its own CPU target; Omni-Compass moves that target inside a range the operator allows. The cluster autoscaler keeps adding and deleting machines; Omni-Compass marks machines idle and lets the autoscaler do the deleting. The card's firmware keeps boosting the clock and enforcing its power limit; Omni-Compass sets the ceiling the boost may reach and the limit the firmware enforces. PgBouncer keeps handing connections to clients; Omni-Compass sets its pool size through the pooler's own console. Kafka's consumer group keeps rebalancing partitions on its own; Omni-Compass changes how many consumers are in the group, which is what an operator does. Redis keeps evicting by its own rule; Omni-Compass sets the ceiling that rule works under.

This fixes how an installation is judged: every paired run has exactly two arms, **native** (the system with its own controller, as shipped and as the operator set it) and **omni** (the same system with Omni-Compass on top). There is no third arm in which Omni-Compass replaces the controller, and no comparison in which Omni-Compass is a rival to it. A gain is therefore always "what the governor added on top of what the native controller already did", and a loss is always "what the governor cost on top of it".

## Do no harm: the paired trial on the muscle

The second design decision is that Omni-Compass must never cause a LOSS. Before Omni-Compass moves a slow knob (a machine given back, a card's clock ceiling, a robot axis's speed, a drone's cruise override), it runs a **paired trial on the muscle itself**: the muscle's own cost per piece of work is measured at the deepest step already allowed and at one step past it, under the same traffic, with at least thirty pieces of work on each side; the step is allowed only if the trial's median cost is no more than 2% above the reference and no more than 2% above the cost first measured at native. A refused step is not tried again for a set number of decisions. Where no step passes, the trial reads **left native**, the knob is never moved, and the receipt for that muscle says "nothing for Omni to move". Gains are not capped; the 2% allowance caps only what may be spent in the trial to look for them.

For an installation this has two consequences. A knob the operator grants is not a knob Omni-Compass will move: it is a knob Omni-Compass may move once the muscle itself has shown the step costs it nothing. And a muscle that never passes a trial is not a fault: it is the right answer for that muscle on that stack, and it costs nothing.

## The pedals, the modes and the two switches

Omni-Compass drives a system the way a self-driving car drives itself: it feels, gauges and adjusts, with nobody in the seat. The vocabulary is used the same way everywhere in this manual:

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Off (manual).** Omni-Compass is not driving; native runs alone. This is the native arm of every paired run.
- **Watching.** Omni-Compass reads every gauge and logs the decision it would take, and writes nothing. Any difference between a watching arm and native is the cost of its being there (its processor time, its reads) plus noise, never a decision; a watching arm that writes anything fails its run. Watching is the first level of every rollout (chapter 6).
- **Autopilot.** One pedal: **push** (traffic climbs and capacity is added at once, never held back), **lift off** (traffic falls and capacity is eased off a step at a time, handed back toward native, never below idle), **idle** (the pedal lifted all the way: calm traffic, the engine at its floor of two machines in service, ready for the next burst), **reset** (every setting the governor ever wrote is handed back to where native had it, read back, and the record removed; every paired run's omni arm ends with a reset and checks it).
- **Cruise.** For work that comes as a pile: when pods have waited for a place for two decisions in a row, machines go into service up to the pool's own top (read live: every machine of a fixed pool at once, a growing pool a step a decision) and stay there, without second-guessing, until the line has been empty for two decisions. In cruise the governor also stops its five-second floor reads, because every machine is already in service and the reads would only cost processor time on the machines the queue is using.
- **Idle at once.** When the work is done and demand is at zero (the service at its autoscaler's own floor, wanting no more, at half its target or less), the pedal is lifted all the way in one move, straight to idle, with every safety check still holding.
- **The kill switch.** Separate from all of the above, and for **security only**: one switch in a human hand (`python3 tools/omni_switch.py off`) turns Omni-Compass's governing off across the whole machine at once, for anything rogue, or for anyone trying to drive a system through Omni-Compass. Every governor hands back and exits; none starts while the switch is off. The reset is the end of a run; the kill switch is the end of trust.

The floor of two machines is an operator's number; lifting off never takes a system below it, however far the pedal is lifted. There is no ceiling: every machine the operator has is usable, and the protection is the band inside each machine.

## Nothing is hardwired

Omni-Compass carries no table of per-stack special cases, and an installation adds none. The few numbers the governor holds are the same on every muscle and are declared, not chosen per benchmark: the centre of the band, the 5% cushion at each edge, and the 2% do-no-harm allowance. Everything else a muscle needs (how fast it answers, how often it may be moved, how far it may go) is read from the muscle's own declaration in the catalog or from the native controller's own settings at the start of the run, never typed in by hand. Where a stack needs a line, a cover or a step size (the MySQL pool moves in the server's own 128 MB chunks, the Redis ceiling in notches), the number comes from the stack being governed and is written down before the first counted run, so a reader of the receipt can see it was not chosen after the result.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

For an installer this means one thing: the settings an installation writes are the operator's own (chapter 3), and nothing else is to be set by hand. A behaviour seen in a receipt is the governor reading the wires and the settings it was given; where it looks wrong, the wires and the settings are the first place to look (section 4.5), because there is no hidden per-stack rule to blame.

## What it sets, muscle by muscle

Every muscle keeps its own control, and Omni-Compass sets what that control already accepts:

| Muscle | Its own control (kept) | What Omni-Compass sets |
|---|---|---|
| Kubernetes service | the HPA | the HPA's CPU target; replica floors |
| Kubernetes node pool | Cluster Autoscaler / Karpenter / MachineSet | how many machines stay in service (park and wake) |
| NVIDIA GPU | the card's firmware (boost, power and thermal limits) | the clock ceiling (up wire) and the power limit (down wire) |
| CPU | the kernel's frequency governor | the frequency ceiling |
| Data hall, building | the chiller and air handler loops | the supply-air setpoint; units in service |
| Battery site | the inverter | the reserve level; the peak ceiling |
| Robot joint, vehicle axis | the servo loop | speed and effort limits |
| Process loop, feeder | the PI controller | the setpoint inside its band |
| Database behind a pooler | PgBouncer's pool, the DBA's fixed setting | the pool size through the pooler's own console |
| Database's storage engine | MongoDB's WiredTiger cache, the operator's fixed setting | the cache size through the server's own console |
| Message broker | the consumer group at the operator's count | how many consumers are in the group |
| Cache | the operator's memory ceiling and eviction rule | the memory ceiling through the cache's own console |
| Substation | the tap changer's own controller | the tap position, one whole tap per move |
| Drone | the shipped autopilot | the cruise override inside the autopilot's limits |

When Omni-Compass stops, every one of those values goes back to what it was before Omni-Compass acted: not to the last value it wrote, but to the value it read once, before its first write.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 2

# Your Stack as One Body: Muscles and Organisms

## The muscles you can wire

A muscle is anything with a meter and a setting: a compute pool and its machine count, a GPU and its clock ceiling, a chiller and its setpoint, a battery and its reserve, a feeder and its tap, a robot joint and its speed limit, a pump and its flow. Each muscle has a native controller of its own, which is what the muscle does when Omni-Compass is not there, and Omni-Compass wires to it through one wire in (the meter) and one wire out (the setting). The catalog (`realms/catalog_v5.csv`) lists **1,107 distinct muscles in 70 families**; Appendix E gives every one with what it is, what Omni-Compass reads from it, what it moves and the factory setting it runs on.

The seventy families, by realm, are these. **Compute, AI and cloud** (twenty-seven): AI inference serving, AI training, blockchain and ledger nodes, CI/CD and build farms, cloud virtual machines and capacity, container resources, cross-cluster and edge placement, DPU and programmable IO, distributed cluster managers, game servers and real-time interaction, GPU fabric and RDMA, GPU hardware, HPC and distributed compute, host CPU and memory, Kubernetes dynamic device allocation, Kubernetes placement and scheduling, Kubernetes workload scaling (the largest family, 32 muscles), Linux kernel and operating-system control, media streaming, node fleets and Karpenter-class control, OpenShift and the Machine API, quantum computing control, serverless functions, virtualization and hypervisors, work admission and demand shaping, and the body's gut (data-plane intake and waste) and skin (perimeter and admission). **Physics, robotics and autonomous** (eight): automotive EV and mobile powertrains, aviation and autonomous flight, elevators and vertical transport, marine propulsion and vessel automation, rail traction and train control, robotics fleets and warehouse automation, robotics motion control, and spacecraft and flight software. **Energy, facility and industrial** (seventeen): agriculture and irrigation, building and critical-environment HVAC, cooling and chillers, district heating and cooling, energy storage and microgrids, facility and grid optimisation, grid transmission and distribution, healthcare critical environments, industrial PLCs and process automation, mining and mineral processing, oil and gas pipelines, PDUs, UPS and electrical distribution, pharmaceutical and food manufacturing, power generation and turbine control, renewable generation and inverter control, semiconductor fabs and precision manufacturing, and water and wastewater pumping. **Distribution and**

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**specialized** (eighteen): cache and memory services, commerce and payments, content delivery and edge caches, data analytics and ETL, databases and transactions, financial markets and trading systems, medical imaging and clinical systems, messaging and streaming, network routing and switching, observability and telemetry, ports and maritime logistics, reliability, security and recovery, runtimes and applications, search, indexing and vector databases, service meshes and API reliability, block, file and object storage, telecom radio access and edge radio, and workflow, logistics and fulfilment. The open-source power and resource managers many operators already run (Kepler, Climatik, EAR, CruiseKube, Goldilocks, the Kubernetes power manager, PEAKS, adaptive GPU power capping and NVIDIA's platform power budget) are muscles too, each in the family its knob belongs to: Omni-Compass sits on top of them as it sits on any native controller.

## One body: the spine, the realms and the organism you build

An installation governs one body. The muscles of a stack, its clusters, databases, caches, cards, cooling and power, are read by one governor and judged together, so that a step on one muscle is refused when it would push another past its cushion (chapter 5). Under every realm's own muscles stands the same **spine** of 267 muscles (Kubernetes, machines, GPUs and CPUs, network, storage, observability, security, cooling and electrical distribution), because every realm's work runs on computers that must be cooled and powered. The catalog groups the muscles into four realms and stacks them into six organisms:

| Organism | Muscles | What it is |
|---|---|---|
| 1. Compute / AI / Cloud | 540 | clusters, GPUs, AI training and inference, cloud capacity, kernels, functions, media, games, ledgers, build farms |
| 2. Physics / Robotics / Autonomous | 404 | joints, fleets, vehicles, flight and spacecraft axes |
| 3. Energy / Facility / Industrial | 480 | data halls, buildings, batteries, UPS, process loops, feeders, hospitals, farms, pipelines, mines, district heat, power plants, renewables, pharmaceutical and food plants |
| 4. Distribution / Specialized | 484 | networks, storage, databases, commerce, trading, content delivery, workflows, radio networks, clinical systems, ports |
| 5. The four stacked, every duplicate kept | 1,908 | all four realms on one clock, the shared spine counted in each |
| 6. The whole tower, every muscle once | 1,107 | every distinct muscle on one clock |

**Building your own organism.** A real installation is rarely a whole realm. It is the muscles one company runs, wired in the order of chapter 6: the cluster first (its pods, then its machines), then the cards and the processors under it, then the databases, brokers and caches beside it, then the site under all of them. Three rules make the

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

result one body rather than a collection of separate governors:

1. **One governor per body.** Every muscle of the body is wired into the same governor, so every step is judged against every other muscle's cushion. Two governors on one body are two brains, and the coupling between them (more pods feeding a database at its connection limit, more cooling pulling a power feed toward its limit) is exactly what neither would see.
2. **Every shared resource is a muscle or a gauge.** Where muscles share a power feed, a cooling loop, a connection limit or a network link, that shared thing is wired in, as a muscle where it has a setting and as a gauge where it does not (a site limit read in watts, a database's connection count), so the governor can see the coupling it must respect.
3. **A muscle with nothing to gain stays wired.** A muscle whose knob never passes a trial stays native, but its gauges still reach the governor, because its state may be what makes another muscle's step unsafe. Watched is not the same as unwired.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 3

# Tuning: The Factory Tune, the Dial and the Settings You Write

## The factory tune

Omni-Compass ships tuned. The engine in the repository is presented as The Omni-Compass LLC tuned it and ships it: a self-running engine, set at the factory, the way a vehicle leaves the plant with its engine already mapped. Every muscle in the catalog is wired and runs on the setting found for it: its own card where one was found and kept (section 3.3), the engine's own setting for every other muscle. The whole tune is fixed and fingerprinted by version (`python3 tools/omni_version.py`), so an installation can always tell whether it is running the factory tune or a tune of its own. Every published result is the factory tune.

No setting is universal. Muscles differ, and no single setting is right for every muscle, family, sector, domain or realm; none is presented as such. What the factory tune fixes is the starting point an installation can trust: each muscle on the setting that was found for it, every muscle wired, nothing switched off.

## The max, and the dial that tunes it down

The factory setting is **the max**: every muscle runs up to its own limit, read live from the system, never past it and never below what native gives. It rides the edge of each muscle's range, not the middle. The energy it takes to do more work is **power given**, shown beside the work, the speed and the wear it bought, never called a loss; the max never takes from one muscle to give to another. A knob that native already runs at the top of its range is "at its limit on native", and the max leaves it there.

The **dial** turns the max down. It has two positions, the same on every live controller:

| Position | Switch | What it does |
|---|---|---|
| **max** (the default) | `--dial max`, or `OMNI_DIAL=max` | the most work, faster: the service held well inside its line so every rush finds its capacity already there; never a machine given back below what the system ran on its own; on a card, the clock ceiling never stepped below the top while the card serves |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Position | Switch | What it does |
|---|---|---|
| **save** (the tune-down) | `--dial save`, or `OMNI_DIAL=save` | native's work on the least resource: the service held nearer its line, machines and clock steps given back where the paired trial on the muscle shows the response time allows (section 1.2) |

The tune-down never does less work than native and never misses its line more often; it spends less to do the same. An installation chooses its position per governor, writes the choice down before its paired runs, and reads every receipt against the position it ran.

## Each muscle's own card

The governor is one, but its settings are each muscle's own. A muscle's **card** holds the numbers the max uses for that muscle alone: for a machine pool, how full it runs on its forecast before it calls for more and how many decisions of history it reads; for a battery, how many wobbles of margin it keeps under its line and, where it is wired by its power, how long it reads its line before it holds and how much of it it keeps clear. The cards were found by a search over every muscle on development cases only, today's engine setting always one of the choices, and a card was kept only where it beat the engine setting beyond the noise on two separate sets of cases and was never behind native on any case under any load (never over its line more often, never less work, never its line cleared later). 364 muscles carry one (`realms/muscle_cards_v5.csv`); every card is written out in plain words with its muscle in Appendix E; the rest run the engine setting. Inside a body, a battery wired by its power keeps the body's own margins on its line, because the line carries the whole body's swing, which the card chosen for the battery alone never saw.

An installation reads a muscle's card before it wires that muscle, so that the receipt can be read against the setting the muscle actually ran on.

## The settings an installation writes

Everything else an installation writes is the operator's own, taken from the operator's own system and objectives, and written down before the first counted run:

| Setting | What it is | Where it comes from |
|---|---|---|
| **The line** (`--slo-ms`, or the stack's own line) | the response time, or other limit, the service must meet | the operator's own service objective, never ours |
| **The band** | the reading's range from calm to the line; on a service, from a tenth of the line (the bare service time) to the line | the line, by rule |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Setting | What it is | Where it comes from |
|---|---|---|
| **The cushions** | the last 5% of the band at each edge, 10% in all; a reading in a cushion is treated as already too close to the edge | fixed, the same on every muscle |
| **The centre** (`--compass-center`) | where in the band the governor holds the reading: 0.3 at the max, 0.4 at the tune-down, unless the operator writes another | the dial, or the operator |
| **The cover** | the knob's hard range; every write is clipped to it | read from the system (the autoscaler's range, the card's supported clocks, the pooler's connection limit), or written by the operator inside it |
| **The ceiling of a growing lever** (`--replica-ceiling`, the machine lever's top) | how far a lever that can keep adding may go | read live by default: what the machines can hold, one under it; an operator may write a lower one, or 0 to keep native's own cap |
| **The floor** | the least a pool may run on | two machines by default; never below 1 for any muscle |
| **The allowance** (`--allow`) | the most a step on trial may cost the muscle's own work | 2%, the same on every muscle |
| **The site's limits** (`--site-limit-w`, `--cooling-min-c`, `--cooling-max-c`, `--cooling-restore-c`) | the facility's power limit and the setpoint band | the facility engineer, inside the equipment's ratings |

Three settings are deliberately not on that list. The governor's decision period, its damping and how fast each muscle answers are the muscle's own declaration, read from the catalog or measured from the system at the start; an installation does not tune them (on a managed Kubernetes cluster in a cloud the declared decision period is 15 seconds, four decisions a minute in step with the autoscaler's own 15-second sync, its fast checks every 5 seconds), and a receipt that needed them changed is a receipt to read as wiring first (section 4.5).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## Profiles: service and batch

The governor has two **profiles**, service (the default) and batch, which differ only in what the pedals do with a pile of work: in the service profile the governor paces the slack; in the batch profile, cruise puts every machine in service while work waits for a place, and idle takes them straight to the floor when the queue is done (section 1.3). An installation running a queue of jobs chooses batch for that service and service for everything else, and writes the choice down beside the dial's position.

## A changed tune is your own configuration

As with any engine, a licensee may add on, retune, upgrade or tune down any muscle or any combination of muscles: the dial, the cards, the bands and the service lines are all exposed. A changed tune is the licensee's own configuration, and its receipts are its own, never the published ones. The discipline for a changed tune is the discipline for the factory tune: write the change down before the paired runs, run native and omni three times on it, show every row, and read the receipts against the tune that ran. Where a changed tune reads differently from the factory tune, the change is the first thing to look at, and then the wiring.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

PART TWO

# The Harness: Plugs, Wires and the Rules They Obey

*The universal plug, the adapters, the wire check that must pass before anything writes, and the rules every wire obeys.*

CHAPTER 4

## The Plug, the Adapters and the Wire Check

Think of a high-end car stereo: one head unit, one standard plug on its back, and an adapter harness for each make of car that matches the car's factory plug. You never cut a factory wire, and when you pull the stereo, the car works as it did. Omni-Compass is wired the same way.

### The plug (one design for every muscle)

Each plug has as many channels as its muscle has knobs. Every channel declares its cover (range), its direction (which way is "more"), its units and how fast it may move. Every plug keeps the same five-line contract, and an adapter for a new muscle is written to it:

1. **attach** - read the knob once, before any write: the snapshot. It never moves afterwards.
2. **read** - the service reading, placed in its band.
3. **write** - one value, clipped to the cover, then read back from the device.
4. **one writer** - if the knob is found at a value Omni-Compass did not write, someone else owns it: Omni-Compass stops writing and leaves that value alone.
5. **restore** - on stop, the knob returns to the snapshot (where Omni-Compass found it, not the last value it wrote). For muscles whose native controller moves the knob itself, restore hands control back.

### Adapters (one per standard plug)

Omni-Compass speaks each industry's standard control interface through an adapter. Status is stated plainly:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Standard | Muscles | Adapter status |
|---|---|---|
| Kubernetes API | HPAs, deployments, pods, nodes | **built** (`python3 -m omni_controller.controller`) |
| NVIDIA NVML / `nvidia-smi` | GPU clock ceiling, power limit | **built** (`python3 -m omni_controller.gpu_compass`, two wires; `omni_controller.gpu_governor`, one wire) |
| Linux cpufreq, RAPL | CPU frequency ceiling, package watts | **built** (`--cpufreq-policy-root`, `--rapl-cmd`) |
| Site meter, building controller by command | site watts, supply-air setpoint | **built** (`--power-cmd`, `--cooling-cmd` command templates) |
| Cloud node groups (Karpenter, Cluster Autoscaler, MachineSet) | machines | **built** through `--node-scale-cmd` templates |
| BACnet, Modbus, SNMP | chillers, air handlers, PDUs, UPS | designed; today reached through the command templates above |
| OPC UA, EtherNet/IP, PROFINET, EtherCAT, ROS 2, CAN | PLCs, drives, robots, vehicles | designed; reached today through a command template written to the plug's contract |
| IEC 61850, DNP3, OpenADR, IEEE 2030.5, SunSpec, OCPP | substations, batteries, inverters, chargers | designed; reached today through a command template written to the plug's contract |

The engineer's job for one muscle is three things: which adapter, the address of the knob, and its safe range.

**The plug contract, line by line, and why each line is there.** *Attach* reads the knob once before any write and keeps that value as the snapshot. It is read once and never again, because a snapshot that moved with the knob would restore to wherever the governor last left it, and the whole point of the restore is to return to where the operator had it. *Read* takes the service reading (a response time, a temperature, a queue, an error) and places it in its band; the plug declares the band, the governor does not guess it. *Write* takes one value, clips it to the cover before it leaves the plug, sends it through the muscle's own interface, and then reads the lever back from the device and records both the sent and the taken value; a device that took something other than what was sent (a clock that rounds to 15 MHz steps, a pool size the pooler capped) is recorded as it is, not as it was asked. *One writer* compares the lever as found at every read with the value last written; a lever found elsewhere has another owner, after which the governor leaves that lever alone for the rest of the run and says so in its audit and its exit code. *Restore* writes the snapshot back on stop and reads it back; a restore that does not read back is an error (exit 3), never a silent success. The same five lines are implemented for Kubernetes

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

objects (annotations hold the snapshot on the object itself, so a watchdog can restore from the object without the governor's memory), for `nvidia-smi`, for PgBouncer's console, for Redis's `CONFIG SET`, for the consumer group and for the simulators' knobs. An adapter written for a new muscle keeps the same five lines, and is proved against a fake device of that muscle before it is pointed at a real one.

## Power-up order

Read-only first; then take the knobs, the least consequential first; on stop, hand them back in reverse order. In the Kubernetes controller the order is: the latency feed and the cluster's own metrics (read); the HPA target (a setting the HPA accepts and corrects within a minute); the pods' CPU limits (conveyance, in place); the machines (the slowest and most consequential, through the release gate and the verdict). On stop the machines are woken first, then the limits, then the target, so the cluster is never left with fewer machines than its pods need while a setting above them is still being handed back.

## The wire check (mandatory before any write)

For the GPU it is one command, `sudo python3 tools/gpu_wire_check.py --gpu 0`, and `scripts/gpu_rented_run.sh` runs it first and stops if it fails:

| Step | It proves |
|---|---|
| 1 read | every meter answers: watts, temperature, utilization, limit, clock, top clock |
| 2 snapshot | the start limit is recorded once |
| 3 up wire | with the card busy, a ceiling below its own busy clock pulls the clock down under it; reset, the clock comes back above it |
| 4 down wire | a lower power limit is reported back exactly; the start limit is reported back exactly |
| 5 restore | clocks reset and the start limit read back |
| 6 stop | the governor, stopped by its signal, hands both wires back |
| 7 other writer | a limit set by someone else while the governor runs is left alone, and the governor exits 5 |

Any failure prints the wire, the step and what the device said, and nothing else runs. That turns a wiring fault from guesswork into one named line to fix.

## Wired right or wired wrong

The wire check proves the wires move. It does not prove the governor reads the right thing. A correct installation leaves signatures in its own receipts; a wrong one leaves others. Check them on your first paired runs, before you believe any number, good or bad.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Check | Wired right | Wired wrong, and what to look at |
|---|---|---|
| Watch arm against native | equal on every gauge (watch writes nothing) | different: the probe, the load or the machine differs between arms, not Omni-Compass |
| Requests served, failed requests | equal to native | fewer served or more failed: a lever is cutting capacity; check its cover and its sign |
| p95 and p99 response time | at native or faster | slower: the reading or the floors are wrong (rows below) |
| GPU: the card's clock while busy | at or above the clock the card reaches on its own | below it: the speed floor is missing or the lid is under the card's own draw |
| GPU: the lid while busy | at or above the card's own busy draw | at the envelope floor while busy: the lid is sized from a curve, not from the card's own meter |
| GPU: credit per write (`GPU_REPS.md`) | compass decisions pace the slack, race decisions cover the bursts; fail-up is rare | fail-up in most decisions: being busy is being read as a breach, or the service line is set too low |
| Kubernetes: machines given back | through the release gate, one per decision, with no pod waiting | machines given back while pods wait, or none at all while the service is far inside its band: check the latency feed, `--slo-ms` and the release gate's reasons in the audit |
| Kubernetes: the reset | every HPA target, replica range, CPU limit and worker back to native, no record left | anything left: an earlier run or another controller wrote the same objects |
| Every lever after the run | at its snapshot | not at its snapshot: another writer, or a restore that failed (exit 3): restore by hand and investigate |
| The feed itself | fresh samples every few seconds | stale or empty: Omni reads it as blind and fails up; fix the feed, not the governor |

**The rule.** If any row reads "wired wrong", the result of that run says nothing about Omni-Compass: fix the wiring, run the wire check and the watch arm again, and only then compare. The first real card run is the worked example: its receipts showed the card's busy clock under its own (736-768 MHz against 861-889), the lid at the 105 W floor under its own 135 W draw, and fail-up in 46% of decisions. Each of those is a row in this table.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## The wire check on the other stacks

The GPU has a one-command wire check because its wires are the hardest to get right. The other stacks carry the same proof in a different place: every Kubernetes arm records `kubectl auth can-i` receipts for what the governor's identity can and cannot do (and fails if it can do anything it must not), checks that the cluster holds nothing but the harness, and ends with the reset check that every HPA target, replica range, CPU limit and worker is back at native with no record left; every database, broker and cache arm reads the knob back after every write and hands it back at the end, and the report says "handed back: yes" or the run is invalid; every simulator arm checks that every override was handed back. The watching arm, where it is run, is the wire check of the reading: a watching governor that disagrees with native on any gauge has found a difference in the probe, the load or the machine, not in Omni-Compass.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 5

# The Rules the Wires Obey

Every muscle is wired both ways: a sensory wire in (its meters) and a motor wire out (its knob), with the read-back closing the loop. Between them, the governor decides how much authority each muscle has at each moment, by four rules that an installer must know because they decide what a receipt can show:

- **Expand is always allowed** (except under a security hold, Chapter 9): adding capacity, power, cooling or protection never waits.
- **Contract needs calm and a clean service record:** giving anything back is allowed only while the muscle is calm and the service has been inside its line for the last three decisions; a continuous muscle gives back at most its calm share of its surplus per decision, a discrete one one unit at a time, through a release gate (section 5.1).
- **Fail up:** while the service is past the wall (95% of the band) the knob returns at once to full capacity, the operator's own setting or the cover's top, whichever the muscle's direction asks.
- **Blind means hold:** if a sense goes stale or unreadable, nothing is given back until it returns (section 5.3).

The rules are asymmetric on purpose. Adding is cheap to undo and expensive to delay, so it is never gated beyond the security hold. Taking back is cheap to delay and expensive to get wrong, so it is gated three ways: by calm, by the service record (inside the line for the last three decisions, not merely this one), and by the release gate, whose reasons are written into the audit every time it holds.

"Two-way" is a statement about wiring, and it is checked, not assumed. Each muscle's plug carries exactly one sensory wire and one motor wire, and the wire check that must pass before any write (section 4.4) proves that the wires move: a value written through the motor wire is taken by the device and read back as the value sent, and the snapshot is restored and read back at the end. No governor writes a knob whose wire check has not passed, because a knob that can be written but not read back cannot be handed back with proof. What the wire check does not prove, section 4.5 says plainly: that the governor is reading the right gauge for the knob; that is what the watching arm is for. The receipts of every run carry, for every write, the value sent and the value the device reported, side by side.

## The release gate

On Kubernetes the **release gate** lets one machine be given back only when every one of the following holds, and writes the first reason that fails into the audit, so a reader can see why a machine was kept:

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

1. the remaining machines would stay under 95% full;
2. no pod is waiting for a place;
3. the pods are not scaling up;
4. the service is not past its line;
5. the muscle's own pressure is under its release threshold;
6. every sense is live;
7. the last command has landed (no new order goes on top of one the cluster has not yet carried out).

One machine per decision is the rule in the slow, reversible direction; past the wall, one machine up at once. The same shape holds on every discrete muscle: a consumer, a chunk of a buffer pool, a notch of a cache ceiling, a tap. The gate's records read, in the governor's own words, "release permitted" (with the utilisation the remaining machines would carry), "no contraction authority" (a calm system whose state has not yet earned the right to give back), or a list of reasons at once, such as "latency breached now; a sense is blind". A machine kept in service always has a stated cause.

## The snapshot, the read-back and the one-writer rule

The governor knows where its own levers are. The rule begins with the **snapshot**: the governor reads every knob it may move once, at the start, before its first decision, and that reading is the operator's setting for the whole run. Every reset, every hand-back and every kill returns the knob to the snapshot, never to a value computed later, and the receipt records the snapshot beside the final read-back so a reader can check the two are equal. The rule continues with the **read-back** after every write: the plug sends a value, waits for the device to take it, and reads the device's own report of what it holds. Where the device takes time to apply a setting, the plug knows that and does not mistake the transition for a foreign hand: MySQL's buffer pool is resized by the server in chunks over several seconds and reported as it goes, so the plug reads the server's own resize status, treats a pool caught mid-resize as its own write in progress, and waits for the server to report completion before it judges the write. The same shape holds on the autoscaler's bounds, Redis's memory ceiling and MongoDB's cache size: the value is read from the device's own console, and only a value the governor neither wrote nor is waiting on is read as someone else's.

If a lever is found at a value Omni-Compass did not write, someone else owns it: the governor stops writing that lever and leaves it alone, because restoring over it would fight the new owner. An operator who changed a knob by hand meant to, and a second controller that changed it is a conflict to be reported, not won. The lever is left where it was found, the audit records the value and the time, the governor goes on governing the other levers, and its exit code says so. A reader of the receipts who finds such a record knows at once that two hands were on the stack, and the paired run that contains it is read with that in mind. For an installer this rule has one consequence above all: **a knob may have only one writer besides its native controller**. A runbook, a cron job, a building schedule or a second tool that writes the same setting must be stopped or handed the knob before Omni-Compass is granted it.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## Blind means hold, and fail up

A sense that goes stale or unreadable is not read as calm. It is read as **blind**, and while any sense is blind nothing is given back; the knob holds, or, if the service was past its line when the sense went blind, returns to full capacity at once. Omni-Compass never acts on a reading it does not have. An API that answers a read with a server error produces a held decision, logged with the reason the API gave, and the next decision is taken on schedule when the API answers again. The same holds for a response-time feed that pauses: the governor holds through the gap and resumes when the feed returns. A stale or missing reading produces a hold, never a move, and the hold is in the record.

For an installer, the consequence is that the freshness of every feed is part of the wiring. A latency file that is written late, a meter that is polled less often than the governor decides, or an exporter that caches its readings will read as blind and keep the governor holding; the audit says so (`gate: a sense is blind`), and the fix is in the feed, never in the governor.

## One body, one governor

The alternative to one governor is a controller per layer, each tuned alone: an autoscaler that adds pods while a power cap is trying to hold the machines down, a cooling loop that chases heat the clock governor is about to remove. These controllers fight because none of them knows the others' intent. One governor that sets every knob from one reading of the body cannot fight itself: the step on each lever is decided from the same reading of the same body at the same moment, and a step that would push another part of the body past its cushion is refused. Nothing anywhere is pushed into a LOSS to move one thing to a GAIN; one trial runs at a time inside a body, so every receipt can name the step that produced it.

This is why chapter 2 asks for every muscle of a body, and every resource the muscles share, to be wired into the same governor. Where a shared resource is left out, the governor cannot see the coupling, and no rule in this chapter can respect a limit the wires do not carry.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

PART THREE

# Wiring It onto Your Stack, Step by Step

*The eight levels, from watching to governing the site, and the wiring of each kind of stack.*

CHAPTER 6

## Before You Start, and the Eight Levels

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

## What your system needs

| You run | You need |
|---|---|
| Kubernetes (vanilla, EKS, GKE, AKS, OpenShift/OKD, Rancher, kind) | Kubernetes 1.34 or newer (in-place pod resize); metrics-server (`kubectl top nodes` works); an HPA with a CPU target on each governed service; a readiness probe and a short preStop pause on each service |
| A response-time feed (strongly recommended at every level) | a CSV per service, `elapsed_seconds,latency_ms,ok`, written continuously; `scripts/latency_probe.py` writes one against any HTTP endpoint |
| NVIDIA GPUs | a driver with `nvidia-smi`; root (or the capability to run `nvidia-smi -pl` and `-lgc`); on a VM, full GPU passthrough |
| CPU power control | Linux cpufreq with the schedutil governor; RAPL for package watts; usually bare metal |
| Site power and cooling | a command that prints site watts, and a command that sets the supply-air setpoint through your building management system |

**Why each requirement is there.** Kubernetes 1.34 or newer is required because the conveyance muscle resizes a pod's CPU limit in place, which older versions do only by restarting the pod, and a restart is a disruption Omni-Compass never causes. metrics-server is the HPA's own eyes and the governor's: without it neither knows what the pods use. An HPA with a CPU target on each governed service is the native controller Omni-Compass sits on; a service without one has nothing for level 2 to move. A readiness probe and a short preStop pause are what make a drain invisible to clients: the Service stops sending requests to a pod before it stops, and kube-proxy sends them to another ready endpoint. The response-time feed is the one wire a cluster does not already have: Kubernetes knows what its pods use, not what its users wait; the compass needs the second, and `scripts/latency_probe.py` writes it against any HTTP endpoint every five seconds. On the GPU, root is needed only for the two `nvidia-smi` writes, and a VM must pass the whole card through or the driver refuses to lock clocks (the wire check's step 3 is where that shows).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## Get the software and check it

```
git clone https://github.com/The-Omni-Compass-LLC/The-Omni-Compass.git
cd The-Omni-Compass
pip install -r requirements.txt
python3 verify.py                       # every check; ends with VERIF
    ↪ ICATION: PASS
```

Build the container image once:

```
docker build -f deploy/Dockerfile -t &lt;your-registry&gt;/omni-compass:&lt;tag
    ↪ &gt; .
docker push &lt;your-registry&gt;/omni-compass:&lt;tag&gt;
```

It runs as non-root, with a read-only root filesystem and no Linux capabilities.

What `verify.py` checks is worth knowing before it is run, because its last line, `VERIFICATION: PASS` or `FAIL`, is the only thing an installer has to read. It runs every test in the repository (the plugs, the readings rule, every stack's harness against a fake server), checks every fingerprinted file of the engine against its registry so that a changed file fails before anything is installed, compares the release's files with its manifest, and checks that every path the documents name exists. It takes a few minutes on a laptop and needs no network beyond the clone. GitHub runs the same script on every push under Python 3.12, and each run is listed under the repository's Actions tab with the commit it checked; install only a commit whose run passed.

To know which engine a checkout carries, ask it:

```
python3 tools/omni_version.py                 # the engine version (o
    ↪ mni-v5), or every file that differs
python3 tools/omni_version.py --commit &lt;sha&gt;   # the same question of
    ↪ any pushed commit
```

Every receipt names the commit it ran on, and the three-run tools refuse to combine runs from different engine versions (chapter 12). No receipt is ever read across engine versions, and the command above is how an installer checks that for themselves.

## The levels

Take them in order. Each has a pass condition; go to the next level only when it holds. The OFF switch works at every level.

The levels exist because a governor earns authority the way a new operator does: by watching first, then by touching the setting that is cheapest to undo, then the next. Level 0 touches nothing and proves the software is the software (`verify.py` checks every sealed file's fingerprint and runs every test). Level 1 watches a live

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

cluster for as long as the operator likes and logs every decision it would have taken, so the operators can read those decisions against what they would have done; nothing passes level 1 until they agree with its readings. Level 2 moves the HPA target, which the HPA itself corrects within a minute if the governor is wrong, and raises replica floors ahead of bursts; it is the first level that can show a gain and the first that can show a loss, and the paired receipts of chapter 10 are its pass condition. Level 3 moves machines, the slowest lever, through the release gate and the verdict; it is where the bill moves and where the most care is spent. Level 4 hands Omni-Compass the replica decision itself, with the HPA kept as the fast reflex upward; it is an option, not a step on the way, and it is not part of the factory tune. Levels 5 to 7 are the GPU, the CPU and the site, each with its own wire check and its own pass. An installation may stop at any level and run there indefinitely; most will stop at level 3.

**Level 0 - Evaluate without touching anything.**

```
python3 verify.py
python3 tools/run_scale.py --runs 10 --scale 1 --out results/scale/eva
    ↪ l     # the six organisms, modelled
python3 tools/run_gpu_card.py results/sim/gpu_two_wire/eval
    ↪        # the two-wire card, modelled
```

Pass: `VERIFICATION: PASS`. Nothing in your systems is touched.

**Level 1 - Watch (read-only).**

```
kubectl apply -f deploy/install/omni-compass.yaml     # set the image
    ↪ line; it runs --mode observe
kubectl -n omni-compass logs deploy/omni-compass -f
```

The install file grants a read-only identity; `scripts/pilot_shadow.sh` records `kubectl auth can-i` receipts that show it cannot write. Every decision it would take is logged with its reason. Pass: zero writes, and readings your operators agree with.

**Level 2 - The pods, on top of your autoscalers. WRITES.**

1. Grant `patch horizontalpodautoscalers` and `patch pods/resize` (`deploy/rbac-target.yaml`).
2. Run with `--mode target --latency-file <feed> --slo-ms <your p95 target>`.
3. Optional pod muscles, one at a time:

| Muscle | Switch | What it does |
|---|---|---|
| convey | `--latency-file` and `--cap-deployments ns/name` | gives each machine's idle CPU to the serving pods on it |
| rightsize | `--rightsize-deployments ns/name` | each pod's CPU request follows its measured use times (1 + headroom), in place |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Muscle | Switch | What it does |
|---|---|---|
| coldstart | `--coldstart-deployments ns/name --coldstart-signal ns/configmap` | scales a service to zero while no work waits, wakes it when work arrives |
| batch | `--batch` | admits held Jobs labelled `omnicompass.io/batch=true` when there is load and power headroom |
| batch pace | `--batch-pace` | pauses Jobs labelled `omnicompass.io/pausable=true` under power or heat stress, resumes them after |
| rollout guard | `--rollout-guard ns/name` | pauses a rollout while change is not permitted |
| contain | `--contain-namespaces ns --contain-cpu-m <m>` | holds an agent namespace to a CPU budget |
| security hold | `--security-configmap ns/name` | key `hold: "true"` blocks every expansion |

Your HPAs keep scaling as before; Omni-Compass sets their targets and raises floors ahead of bursts. Pass: p95, p99 and failed requests never behind native, over paired runs (chapter 10).

**Level 3 - The machines. WRITES.**

1. Grant `patch nodes` and `patch pods` (`deploy/kind/rbac-omni.yaml`).
2. Run with `--mode nodepool --active-nodes-only --closure /app/law/closure.json --node-scale-cmd "<command with {n}>"` and `--node-restore-cmd "<command>"` for the OFF switch. For the compass on the same levers, use `--law compass` (it reads the mean response time of the latency window in the band from a tenth of `--slo-ms` to `--slo-ms` and holds it at the centre of section 3.4, 0.3 at the max and 0.4 at the tune-down; p95 at the line, a blind feed or a waiting pod adds capacity at once; the HPA target never goes above the operator's; at the max no machine the cluster had on its own is given back, and at the tune-down machines go back one at a time through the release gate and the paired trial).

| Your platform | The park/wake command |
|---|---|
| any cluster, bare metal, kind | `bash scripts/kind_nodepool.sh {n}` |
| Karpenter / EKS Auto Mode | the NodePool CPU limit at `{n}` times node CPU, parked nodes kept |
| Cluster Autoscaler node group | the group's desired size, scale-down through parking |
| OpenShift / OKD | the worker MachineSet replicas, parked |

A machine is given back only when every sense is live, the last order landed, pods are not scaling up, nothing waits for a place, and the remaining machines stay inside the band. Pass: fewer machines in service, with no

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

LOSS on any service gauge.

**Level 4 - Omni-Compass decides; Kubernetes is the muscle. WRITES.** Add `--strict-replicas`: Omni-Compass decides each service's replica floor and when to shrink; the HPA stays as the fast reflex upward. Pass: as level 3.

**Level 5 - A GPU box, two wires. WRITES.**

```
sudo python3 tools/gpu_wire_check.py --gpu 0                    # must
    ↪  end: WIRED RIGHT
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 7

# Stack by Stack

Every wiring below is the same plug (section 4.1): one wire in (the stack's own meter), one wire out (one setting the stack already accepts), a snapshot taken once before the first write, every write read back, everything restored on OFF. Native is always the stack's own controller left as shipped; omni is that controller with Omni-Compass on top.

| Stack | Levels | Notes |
|---|---|---|
| Vanilla Kubernetes, kind, Rancher | 1-7 | as written |
| Amazon EKS | 1-5 | machines via Cluster Autoscaler node group or Karpenter; CPU power control is not exposed on EC2 VMs; GPUs on bare-metal or full-GPU instances |
| Google GKE, Azure AKS | 1-5 | the provider's node-pool size as the park/wake command; on AKS the managed cluster autoscaler stays native and deletes the machines Omni-Compass idles (7.1) |
| Red Hat OpenShift / OKD | 1-5 | the same permissions through a Role; machines via the worker MachineSet |
| NVIDIA GPU servers without Kubernetes | 5 | the two-wire GPU governor alone |
| Bare-metal CPU servers | 6 | cpufreq and RAPL through sysfs |
| Slurm / HPC | 5 on the GPU nodes | a job-level Slurm connector is not built |
| Building management | 7 | through your BMS's command line or API, in the command templates |
| PostgreSQL behind PgBouncer (any database behind a pooler with a console) | one knob | the pool size through the pooler's own admin console (7.2) |
| A power grid's substation tap changer (pandapower, SimBench) | one knob | the tap position, one whole tap per move (7.3) |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Stack | Levels | Notes |
|---|---|---|
| Buildings with batteries (CityLearn) | one knob a building | the battery's charge and discharge inside the simulator's own limits (7.3) |
| Robot arms (MuJoCo Menagerie) | one knob an axis | the joint speed where the axis has slack; never on an axis at its takt (7.3) |
| A message broker's consumer group (Apache Kafka) | one knob | the consumer count inside [1, partitions], read from the consumers' own records; the rebalance paid on every move (7.4) |
| A cache's memory ceiling (Redis) | one knob | `maxmemory` through the cache's own console, grown only while the cache is full (7.5) |
| Drone swarms (gym-pybullet-drones; PX4 and ArduPilot next) | one knob a drone | the cruise override inside the autopilot's limits; a separation wall; collisions void the cell (7.6) |
| A database's storage-engine cache (MongoDB under YCSB) | one knob | the WiredTiger cache size through the server's own console, grown only while the cache is full (7.7) |

## Managed Kubernetes on Azure: the bill as the gauge

On Azure Kubernetes Service the muscle is the same as on any cluster (chapter 6, levels 1 to 3) and one thing is new: the bill is real. The workflow `aks-metered` builds a fresh cluster for every arm the same way: a system pool of one machine, tainted so no workload lands on it (Azure's add-ons and the load generator live there; it is never measured and never governed), and one or several **work pools** under Azure's own cluster autoscaler, every pool labelled `omni-role=work`, every pool starting full. Native is that autoscaler alone. Omni on top is the same autoscaler with Omni-Compass idling the machines it gives back, which the autoscaler then deletes on its own schedule. The bill counts every work machine that exists, every 15 seconds, priced at Azure's list price for its family; each cluster is deleted when its arm ends, so nothing keeps billing.

**Sizing the fleet to the lever.** The smallest saving a fleet can show is one machine. On 4 workers one machine is 25% of the fleet, so only a saving of about 30% or more can clear the noise; on 15 workers about 10%; on 96 about 3%. A few percent of a 4-worker fleet is less than one machine, so a 4-worker receipt reading no difference beyond the noise on the machine rows is the honest reading of a fleet too small for the lever, not a result against it. A new subscription allows 200 vCPUs in a region but 10 per machine family, so one family gives five 2-vCPU workers; the way past it is **several families at once**: eight pools of four to five workers each make a fleet of 39 (`worker_pools` input, `size:max,size:max,...`; `pool_prices` names each family's price). The app's replica ceiling is raised with the fleet (`hpa_max`: nine 200-millicore pods fit one 2-vCPU worker, so 9 × workers lets the fleet fill), the same in both arms; Omni-Compass's replica room,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

read live from what the machines can hold, lets the omni arm raise that cap while it binds and hand it back once demand holds still (section 3.4). A cloud region can refuse a new cluster for hours at a time; check the subscription's allowances before building anything, so that a refusal costs cents and not a cluster. The account setup, click by click, is in `docs/AZURE_SETUP.md`.

**In the cloud, the bill is the gauge, and it is never raised.** In a public cloud every machine that exists is billed, whether it carries work or not, so the aim on a managed cluster is native's work for a smaller bill, never a bigger one. The cloud therefore runs at the tune-down (`dial save`, the `dial` input of `aks-metered` and its default) with native's own replica cap (`replica_ceiling=0`): the governor holds the service at native's line and gives back the machines the response time does not need, which a cloud stack shows in the quiet stretches between bursts and in a day's trough. The max (`dial max`) belongs where the machines are already paid for (your own racks, a chip whose heat line is its limit), where more work from the same machines is the gain; in a cloud it buys speed with machines, and every machine it adds is on the bill. The benefit of the max is taken before the cloud, in the service's own engine, or after it, in the chip. In the cloud the governor decides every 15 seconds (`OMNI_INTERVAL`, passed as `--interval`), four times a minute, and its fast checks (a machine for pods that wait, the idle processor to the serving pods) run every 5 seconds; a pod's start and a new machine's join are not shortened by deciding faster, so the governor acts on the reading while it climbs, before the line, and gives the running pods the processor the machine already has, which takes seconds and needs no new pod.

## A database behind its connection pooler

PostgreSQL is wired through PgBouncer, the pooler most of its deployments already run. Nothing models the database. The wire in is the pooler's own report (`SHOW STATS`, `SHOW POOLS`), read once a second over one connection held open for the whole run, so that no process is launched for a reading: the larger of two figures, the time a transaction spent in the server plus the time clients waited for one, over the transactions completed, and the share of the pooler's clients queued for a server. The second figure is what makes the wire honest: a pool too small for the offered rate shows at the pooler as clients waiting even while its own service time reads calm. The wire out is one setting through the pooler's own admin console, `SET default_pool_size`, inside the cover [2, 90]: never under two server connections, never within ten of PostgreSQL's connection limit. The reading is held at 40% of a 50 ms line. Where the time was spent decides the direction: while clients waited for a server one percent or more of the last second, servers are added back, one for every percent of waiting, up to the pooler's own setting; a server is taken back only while clients waited under one percent of the second and a transaction was served, and only one the pooler itself shows idle. At 95% of the line the knob is handed back to the pooler's own setting at once. If the knob is found at a value Omni-Compass did not write, it stops writing (one writer). At the end of every omni arm the pool size is restored to the snapshot and read back. The same plug fits any pooler or proxy with a console and a pool-size setting (PgBouncer, Pgpool-II, ProxySQL, the application's own pool); the processor time the plug's own readings take on the host is recorded beside the host's, so the receipt shows what the wiring itself costs.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## Independent simulators, each with its own native controller

Three published simulators show how the plug is wired onto three more kinds of native controller, and the simulator's own controller is always native:

- **A building's batteries** (CityLearn): the knob is each building's battery charge and discharge inside the simulator's own limits; native is the simulator's own rule-based control.
- **A substation's tap changer** (pandapower on SimBench grids): the knob is the tap position, one whole tap per move and a week of evidence before stepping down; native holds the tap at 1.00 per unit as the grid ships.
- **A robot arm's joints** (MuJoCo Menagerie): the knob is the joint speed of an axis that has slack; a paired physics trial before the run leaves an axis at its takt native, so an arm whose axes are all at their takt has nothing for Omni-Compass to move.

The same three wirings apply to the real controllers they model: a building management system's battery dispatch, a substation's tap-changer controller, a robot's servo limits. Receipts from a simulator are evidence class S: statements about the simulator's model, never about hardware.

## A message broker's consumer group

Apache Kafka is wired as an operator runs it: one broker as shipped, a topic of eight partitions, a consumer group at the operator's count of two. Nothing models the broker. The wire in is the group's own end-to-end latency: each message carries its production time, each consumer records when its handling ended, and the reading is the mean over the messages consumed in the last second; a second with nothing consumed while messages wait reads as the line (someone is waiting and no one is taking), and nothing consumed with nothing waiting reads as calm. The wire out is the count of consumers in the group, which is what an operator changes, inside the cover [1, 8]: never fewer than one, never more than the topic has partitions. The compass holds the reading at 40% of a 500 ms line with a response time of 3 s, because a consumer added or removed shows only after the group rebalances. Slow service with messages waiting adds consumers in proportion to the force; calm with nothing waiting gives back one idle consumer a second after a five-second dwell; at 95% of the line every consumer the topic can use starts at once. The rebalance that every move costs is paid by Omni-Compass and counted in the gauges. At the end of every omni arm the count is handed back to the operator's and read back. The receipt this wiring is built to produce has the queue kept short and no message lost, at the cost of consumers running, which reads power given by rule and stands beside the gain.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## A cache's memory ceiling

Redis is wired as Ubuntu ships it, with the two settings an operator sets for a cache, a memory ceiling of 64 MB and the allkeys-lru eviction rule. The application in front of it is ours and declared: 1,500 requests a second from 32 threads on a Zipf working set that widens and narrows one notch at a time; a miss costs a declared 5 ms trip to the store behind the cache and a write-back. The wire in is the application's own request latency, the mean over the last second (a hit about 0.3 ms, a miss about 5.5 ms). The wire out is one setting through Redis's own console, `CONFIG SET maxmemory`, inside the cover [16, 512] MB. The compass holds the reading at 40% of a 2 ms line. Its direction rule carries the do-no-harm gate in the form this stack needs: misses grow the ceiling by notches of 8 MB **only while the cache is full** (memory used at 90% of the ceiling or more), because a miss in a cache with room to spare is a cold miss that no ceiling can mend; calm with nothing evicted in the last second gives back one notch a second, after a five-second dwell since the last growth; at 95% of the line with the cache full a quarter of the cover is added at once. One writer: a ceiling found at a value Omni-Compass did not write stops it. At the end of every omni arm the ceiling is handed back to the operator's 64 MB and read back. The memory the compass holds for a wide working set is the resource this wiring trades and reads power given by rule; the gain is the hit rate and the work inside the line.

## Drone swarms: the autopilot's cruise

In gym-pybullet-drones (University of Toronto, MIT License) every Crazyflie 2.x flies its shipped position controller along the planner's path at the planner's cruise: that is native. The wire in is each drone's tracking error, the distance between the drone and the point its autopilot was told to be at, on a band from 0 to a declared safe error of 0.25 m. The wire out is one knob a drone, the cruise override: the fraction of the planner's cruise the carrot moves at, never under 1.0 (the planner's own) and never over 2.0 (a quarter of the autopilot's own speed limit), moving by at most 0.01 a control tick. With slack the compass spends it as speed; as the error grows it eases; at 95% of the safe error the cruise is the planner's at once. Two guards belong to this stack: a **separation wall** (a drone with another within 0.5 m flies the planner's cruise, whatever the error reads) and the **paired physics trial** before the counted missions, which asks the simulator's own figures whether a faster mission costs less energy at all and keeps the error under the safe error; where it does not, the override is left native for that cell and the cell reads "nothing for Omni to move". Energy is a declared model from the simulator's own motor constants, said to be a model wherever it is printed; a collision voids the cell. At every landing and at the end of the run the override is handed back and read back.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## A database's storage-engine cache

MongoDB is wired as an operator runs it: the server as its publisher ships it, its configuration as shipped but for the one setting an operator sets for the storage engine, the WiredTiger cache (512 MB in the reference wiring). The wire in is the server's own mean read latency over the last second, from its `serverStatus` operation latencies, differenced, beside the cache's own counters (bytes in the cache, pages requested from it and pages read into it). The wire out is one setting through the server's own console, `setParameter wiredTigerEngineRuntimeConfig cache_size`, inside the cover [256 MB, 2,048 MB]: the server's own floor at one end, a quarter of the machine at the other. The line is set on the stack's own figures so that a hit reads calm and a full cache that misses reads slow (1 ms in the reference wiring, held at 40%). The direction rule is the cache's: slow reads grow the cache by notches of 64 MB only while the cache is full (bytes in it at 90% of its size or more) and missing (pages read into it one percent or more of the pages requested from it in the last second), because slow reads in a cache with room to spare, or in a cache that already holds its working set, are not the cache's to mend; calm gives back one notch a second after a five-second dwell, and only while the cache's miss share is under one percent, because "no page evicted" is true of every cold cache while it fills; at 95% of the line with the cache full a quarter of the cover is added at once. One writer, read-back and the hand-back at the end are the plug's, as everywhere. Where the data files fit in the operating system's own page cache, a storage-engine cache miss is a read from memory and a decompression, not a disk read, so the gain available to this knob is smaller than on a machine whose data does not fit in memory; say so beside the receipt. The same plug fits any store whose engine exposes a cache size at run time (InnoDB's buffer pool, RocksDB's block cache); a store that can only be sized at restart has no knob Omni-Compass moves.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## A database's buffer pool, resized online in the server's own chunks

MySQL is wired as an operator runs it: the server as its distribution ships it, its configuration as shipped but for the settings an operator sets for InnoDB, the buffer pool (512 MB in the reference wiring), the chunk the server resizes in at its shipped 128 MB, one pool instance, and the performance schema on. The wire in is the server's own mean statement latency over the last second, from the performance schema's statement summary, differenced, beside the pool's own read requests and reads from disk. The wire out is one statement through the server's own console, `SET GLOBAL innodb_buffer_pool_size`, inside the cover [128 MB, 2,048 MB] and always in whole chunks, because that is how the server itself moves the pool. Two things about this knob differ from a cache the server resizes at once, and the plug carries both. The server resizes asynchronously and reports its progress in its own status variable, so the plug waits for the resize to complete before reading back, and a pool found mid-resize is the server still carrying out Omni-Compass's own write, not a foreign hand. And a shrink evicts pages while the resize runs, so the dwell after any move is ten seconds rather than five. The line is set on the stack's own figures, above a hit and under a miss (a 0.6 ms statement line in the reference wiring, held at 40%). Slow statements grow the pool by chunks only while the pool is full and missing (one percent or more of its read requests), at 95% of the line with the pool full four chunks are added at once, and calm gives one chunk back after the dwell only while the pool's misses are under one percent of its read requests and a read request was served, because InnoDB keeps stale pages resident, so neither "pages free" nor "no page read from disk" ever says that a pool holds its working set, and the miss share does. **The hand-back waits for the server.** Under load InnoDB finishes a shrink only when the pages the load has pinned are released, and it ignores a new pool size while a resize is in progress; the plug therefore waits for the resize the server is carrying out, writes the snapshot, writes once more if the server ignored the first write, and records the restore (what was in flight, the seconds waited, the writes, the final read-back) in every omni arm's receipt. A knob the device applies on its own time is handed back on the device's time, and the proof of the hand-back is the read-back, not the write.

## The machine itself: the kernel's frequency governor under the processor's meter

The lowest level a governor can sit on is the machine it runs on. Linux already has a controller for the processor's clock, the frequency governor (`schedutil`, or the processor driver's own), choosing the clock every few milliseconds under a ceiling the operator sets once; and the processor has its own energy meter (RAPL) ticking in microjoules. Omni-Compass sits on top of the governor exactly as it sits on the HPA: the governor keeps running, and Omni moves one knob, the frequency ceiling of every CPU, through the kernel's own files.

The reading is the service's own mean latency, held at 40% of a 20 ms line by the compass law; calm gives one notch of the ceiling back (5% of the top clock), a slow service raises it by notches, the wall puts the top on at once. Every notch down is tried on the machine itself first (section 6.4): the paired trial, eight cost samples at the deepest notch already allowed and eight one notch further, allowed only if the cost does not rise within

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

2%. The cost is the index's own reading for one machine (package power × latency / work inside the line), so a notch that saves less energy than it costs in speed is refused and the ceiling stays where the operator left it; a second declared objective, energy per request inside the line with the line as the guardrail, is run beside it. The snapshot of every CPU's governor and ceiling is taken once; the ceiling is handed back and read back at the end; a ceiling found at a value Omni-Compass did not write stops it; the master switch and the lease hand back if Omni-Compass dies.

The meter is the processor's own counter, class P with the caveat that it is the processor's calibrated counter and not an external meter; a smart plug on the wall socket (`WALL_METER`) adds the whole machine, read by the harness only. GitHub's own machines are virtual and expose neither the governor nor the meter. The paired run for this muscle runs on a machine on the metal as root, in one command (`sudo bash scripts/cpu_power_run.sh`): a workstation on Linux, a bare-metal server, or a self-hosted runner on either.

## The data center under the servers: cooling, power and batteries

Under every cluster in this book stands a building. Its native controllers are older and more conservative than any autoscaler: the building management system holds the supply-air temperature through the cooling units' own loops, the chilled-water plant runs its own sequence, the uninterruptible power supply holds its reserve, the power distribution units meter every rack, and every server's management controller and every card's firmware enforce their own power limits. Omni-Compass sits on top of them exactly as it sits on the HPA. They keep running; it moves only the settings they already accept, and only inside the band the operator writes down.

**What it may set, and what it never sets.**

| Muscle | Native controller | The knob Omni-Compass may hold | Switch | Status |
|---|---|---|---|---|
| Cooling | the building management system and the cooling units' own loops | the supply-air setpoint, inside the operator's band | `--cooling-cmd "<sets {c}>" --cooling-min-c --cooling-max-c --cooling-restore-c` | built (level 7) |
| Site power | the site's own limit and its breakers | none: site watts are read, and the site limit enters the engine as a stress | `--power-cmd "<prints site watts>" --site-limit-w <limit>` | built (level 7) |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Muscle | Native controller | The knob Omni-Compass may hold | Switch | Status |
|---|---|---|---|---|
| Servers' processors | the kernel's frequency governor and the processor's own limits | the clock ceiling and the power limit | section 7.9 | built (level 6) |
| Cards | the card's firmware | the clock ceiling and the power limit | level 5 | built |
| Uninterruptible power and batteries held as reserve | the UPS's own reserve | **none, ever**: a reserve held for an outage is never a lever (the physics criterion) | | native by design |
| Batteries held for shifting energy | the site's dispatch | the charge window | | designed as an organ, not yet wired |

**How the cooling muscle moves.** The setpoint is moved by the heat state between the operator's minimum and maximum: a warmer setpoint while every reading is cool, which saves the chiller's energy, and a colder one as heat rises. The band must sit inside what the equipment is rated for; for most data-center equipment that is the 18 to 27 °C inlet range of the ASHRAE thermal guidelines, and inside any narrower limit the equipment's own warranty sets. The restore value is the operator's own setpoint, handed back and read back on OFF, at the end of every run, and by the lease if the governor dies.

**The do-no-harm gate at the site meter.** A warmer setpoint saves chiller energy only up to the point where the servers' own fans spend the saving: fan power rises steeply with inlet temperature. That is why the cooling knob is judged at the site meter and not at the chiller: a step is allowed only where the paired trial shows the site's energy for the same work no higher, every rack inlet inside its band, and no LOSS on any service gauge. Where it does not, the setpoint stays the operator's, and the result says so.

**Wiring it, in order.**

1. **Watch.** Give Omni-Compass read access to the building management system's points (supply and return air, every rack inlet, the chilled-water temperatures) and to the site meter, and let it watch at least one full cycle of the site's demand. Nothing is written.
2. **Write the band.** The minimum, the maximum and the restore value of the supply-air setpoint, inside the equipment's ratings, signed off by the facility's engineer; the site limit in watts.
3. **The wire check** (section 4.4). The cooling command must read back the setpoint it wrote; a building schedule that rewrites the setpoint is another writer, and Omni-Compass stops writing that lever by itself (section 5.2).
4. **One hall first.** A test hall or a single row, paired against its own native weeks, before any other hall.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

5. **Keep the switches in reach.** The master switch and the kill file work at the facility level exactly as at every other level.

**What is proven, and what is not.** The facility muscles are modelled today (class S): the data-hall cooling family of the catalog, the power distribution family, and the site budget shared by groups of cards, which held out with no minute over its budget. No live facility has run yet. A class P result needs a building management system's point list, read access, a site meter and a test hall; until a facility pilot has produced its own paired receipts, this section is a wiring instruction, not a claim of savings.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

PART FOUR

# Operating It

*The OFF switch, the rules the governor obeys, the log it keeps, and the care of a running installation.*

CHAPTER 8

## The OFF Switch, the Rules, and the Log

**The rules Omni-Compass keeps on your system.**

1. **The master switch, for the whole harness, in a human hand:** `python3 tools/omni_switch.py off` turns every Omni-Compass governor on the machine off at once (the Kubernetes controller and the GPU governors). Each puts every setting it ever wrote back to the value it read before its first write, reads it back and exits; the command waits until all have, and says so. While the switch is OFF, no governor will start. `python3 tools/omni_switch.py on` allows them to be started again (nothing restarts by itself); `status` shows the switch and every governor running. Use it the moment anything looks wrong, including a suspected breach. Each governor also keeps its own switch for one muscle at a time: the kill file (or `OMNI_KILL=1`, or SIGTERM to the GPU governor). **If a governor dies without handing back** (killed outright, the machine crashed, it hung), nothing it wrote is left in place: every governor records, before its first write, the exact commands that put every setting back, and renews a lease every decision. Run `python3 tools/omni_switch.py watchdog` as its own service next to the governors: it hands back for any governor whose process is gone or whose lease has run out (a hung one is stopped first). `status` names any governor that died without handing back.
2. It records before it acts (annotations on Kubernetes objects; the `snapshot` line in the GPU audit).
3. It watches before it writes.
4. It never acts blind.
5. Service first: while response time is over its target, and for `--slo-clear` decisions after, nothing is given back.
6. Every change is read back; no new order goes on top of one that has not landed.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

7. The living band and the cover: no organ is driven outside its range; machines are parked, never switched off by Omni-Compass; it never evicts or moves a pod.
8. One writer: if anyone else changes a knob, Omni-Compass stops writing it and leaves it alone.
9. Everything is logged: every read, decision, write and reason goes to the audit log.

**On autopilot: why nobody babysits it.** Omni-Compass is an autopilot: a closed-loop supervisory controller that runs, steers and corrects itself inside the system it governs, as a self-driving car's controller does inside the car. Once the paired trial on a muscle has passed and the governor is set to govern, nobody turns a knob and nobody has to react. Every change is the governor's own, and the governor audits itself: every write is read back from the system, every machine it gives back is kept only after its own paired trial shows the response time holds, every reading, decision and reason is in its audit, and it hands every setting back at the end; nobody has to check its work either. It holds the response time at the centre of its band, under the line, and acts as the reading approaches the line, so the correction lands before the line is crossed, not after a person is paged. What a native stack pays in people for the same service (an engineer watching dashboards, paged while the service is over its line, retuning an autoscaler's target or a node pool by hand) is the **babysitting tax**, and every Kubernetes and cloud receipt counts it in both arms: the **alert minutes** (whole minutes whose 95th percentile, failed requests counted over, was over the line: the minutes an on-call page would be open; judged, the aim lower), the **settings commands the governor wrote on its own**, and the **share of its additions decided while the service was still inside its line** (both shown, not judged). No person touches either arm of a paired run. The aim in the cloud is **no alert minute at all**: a minute over the line is a person woken, so an alert minute in the omni arm is a LOSS of the aim, its cause found and fixed and the test run again. Where native itself is short of machines (over its line on its own), the receipt says so beside the row: the same machines can be made to carry more, but no governor creates compute that the machines do not have.

**The three ways a governor stops, and what each leaves behind.** A referee or an underwriter should know exactly what happens to a governed system in each of the three ways Omni-Compass can stop, because the answer is the same in all three: every setting at native, read back, with no record left.

- **The reset** (every setting handed back to native) is the planned end of a run. The governor sees its reset file, hands every setting back in reverse order of power-up, reads each back, writes its last audit line and exits 0. Every benchmark arm ends with one and then checks the cluster from outside the governor: the HPA target at the operator's, the replica range as set, every pod's CPU limit at its shipped value, every worker back in service, no `omnicompass.io/` annotation left on any object. An arm whose reset check fails is invalid and is left out of the table, never counted.
- **The kill switch** (security) is the unplanned end of trust. `tools/omni_switch.py off` writes the switch file and sends every registered governor SIGTERM; each performs the same hand-back as a reset and exits; the command waits until all have and reports any that have not. No governor starts while the switch is off. It is one switch for the whole machine, not one per muscle, because the case it exists for, something driving a system through Omni-Compass, is not a case for picking which muscle to stop.
- **A death without a hand-back** (killed outright, a crash, a hang) is the case the lease covers. Before its first write every governor registers itself with the exact command that puts every setting back

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

(`omni_controller.controller --restore-only`, which restores from the snapshot annotations on the objects themselves, so it needs no memory of the dead process), and renews a lease every decision. The watchdog (`tools/omni_switch.py watchdog`), run as its own service beside the governors, finds any governor whose process is gone or whose lease is older than its limit (five decision periods, at least two minutes), stops a hung one, runs its recorded restore command, and records what it did. Drill it before production: kill the governor outright in the middle of a governed paired run on your own system, and record how many seconds pass before every setting is back at the snapshot and what the service saw in that gap.

**Reading the log.**

| Line | Meaning |
|---|---|
| `gate: a sense is blind` | a reading failed; nothing is given back until it returns |
| `gate: pods scaling up` | pods first, machines after |
| `decision failed (n in a row)` | the cluster could not be reached; nothing was written; turn it OFF for native |
| `decided_by: compass` | the compass set the wires this decision |
| `decided_by: fail_up` / `blind_fail_up` | the service crossed the 0.95 wall, or a sense went blind: full capacity at once |
| `decided_by: thermal_hold` | the card reported a heat slowdown; nothing was tightened |
| `foreign_writer` | someone else changed a knob; Omni-Compass now observes only |
| `restored ... ok: true` | the OFF switch put every setting back and read it back |
| `"watchdog": "handed back"` (in the watchdog's own log) | a governor died or hung without handing back; the watchdog ran its recorded restore command; `ok` says whether every command exited 0 |
| `cruise: on` / `cruise: off` | work waited for a place for two decisions: machines in service up to the pool's own top; the line has been empty for two: cruise ends |
| `idle_floor` | the work is done and demand is at zero: the pedal lifted all the way, straight to the floor in one move |
| `verdict_state: left native` | no step of the slow knob passed the paired trial (section 1.2): the knob is never moved on this muscle |
| `shield_interventions` | the shield clipped a write to its bounds or its step limit; the count says how often |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Line | Meaning |
|---|---|
| `overhead` (the last line of the audit) | the governor's own CPU over the window: its process and every command it ran |

**How to read an audit from end to end.** The audit (`audit.jsonl`, one JSON record a line, every line timestamped) is the governor's complete account of itself, and a reader can follow a run through it without the manual: the first lines are the snapshot (every setting as found, written to the objects as annotations so a watchdog can restore from them); then, every decision, the readings (the response time and its age, the pods waiting, the machines seen, whether any sense is blind), the reading's place in its band (`compass`), the authority granted and the release gate's reason (`authority`, `node_gate`), the decision itself (the machines recommended against the machines seen, the HPA target set), every write with the command that made it and the value read back, and the decision's wall time (`decision_ms`); in cruise, the floor step's silence; at the end, the reset's writes and read-backs and the `overhead` record. The paired-run harness prints a decision trail from the audit at the end of every omni arm, one line a decision, so a reviewer reading a run's log sees what the governor saw and did at each minute.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 9

# Maintenance, Upgrades and Security

- Run `python3 verify.py` after every upgrade; it must end `VERIFICATION: PASS`.
- Upgrade in watch mode first; take the levels again from level 1.

**What an upgrade is, and what it is not.** The engine is fingerprinted by version (chapter 12). An upgrade that changes any rule, guard, preset or muscle of the engine is by definition the next engine version, and every receipt an operator made on the old version must be made again on it before the two are read side by side; `tools/omni_version.py` says which version a checkout carries, and a checkout that matches none says which files differ. An upgrade that changes only the harness, the tools or this manual leaves the engine's fingerprint as it was and every receipt standing. An operator can therefore tell, from one command, whether a new checkout is the engine their receipts were made on or something that must prove itself again.

- The container runs non-root, read-only, with no capabilities; the Kubernetes identity has only the permissions of the level you run.
- The GPU governor needs root only for `nvidia-smi -pl` and `-lgc`.
- Report a security issue as `SECURITY.md` describes.

**Least privilege, proven on every run.** The Kubernetes governor runs as its own service account (`deploy/kind/rbac-omni.yaml`) with exactly the verbs the level needs: patch on the governed HPA, nodes, pods and the governed deployment, get on its security ConfigMap, create on pod evictions. It cannot delete nodes, create or delete pods or deployments, touch the load generator or anything in `kube-system`, read any secret, create a namespace, or write its own security hold. Every paired arm records `kubectl auth can-i` for each of these and fails if a "cannot" reads yes or a "can" reads no (`rbac_omni.txt` in every paired run's files). The security hold itself is a ConfigMap another identity owns: when its key reads `hold: "true"` every expansion is blocked, and the governor cannot clear it.

**Credentials, and the notice.** Cloud credentials live only in an encrypted secret store and are used only by the jobs that need them; no credential, key or token belongs in any file, log, receipt or commit. Every source file carries its SPDX license identifier so that license scanners (Black Duck, FOSSA, Snyk) read the evaluation license correctly, and every generated report carries the legal notice at its head and foot.

**Supply chain.** Everything the paired-run harnesses download is pinned and checked: metrics-server by version and SHA-256 (`scripts/kind_bench.sh`), Apache Kafka against the Apache Software Foundation's own SHA-512 file for that release, Redis as the Ubuntu runner's own package, MongoDB from

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

its publisher's own repository signed by the publisher's key and YCSB against the SHA-256 written in its preregistration, gym-pybullet-drones at one named commit, Python packages by `requirements.txt`. A download whose checksum does not match stops the run before anything else happens.

**The upgrade drill, step by step.** An operator who receives a new checkout does five things, in order, and stops at the first that fails. First, `python3 tools/omni_version.py`: if it prints the version the operator's results were made on, the engine is unchanged and the rest of the drill is about the harness; if it prints a list of differing files, the checkout is a new engine and nothing the operator measured before applies to it until measured again. Second, `python3 verify.py`, which must end with its pass line; a checkout that fails it is not installed anywhere. Third, the wire check on the operator's own stack, which proves that every wire the new checkout would use still follows, reads back and returns. Fourth, watch mode for at least one full cycle of the operator's load, reading the log the governor would have written and comparing its would-be decisions against what the operator would have done. Fifth, the levels again from level one, each with its paired receipt. An upgrade that skips a step has not been upgraded; it has been installed.

**The security hold, and who holds it.** The hold is the one control that outranks the governor's own judgment, and it is deliberately not the governor's to hold. It lives in a ConfigMap owned by another identity (the operator's security team, in practice); while its key reads true, every expansion the governor would make is blocked and every contraction is still allowed, so a governor under a hold can only give capacity back, never take more. The governor reads the hold on every decision and records in its audit that it read it; it has no verb that could write it. The kill switch (chapter 8) is the blunter instrument beside it: the hold constrains a running governor, the switch stops every governor on the machine and hands everything back. An operator who suspects anything untoward uses the switch first, reads the audit second, and rotates the governor's identity third; the hold is for the slower case of a change window or an incident elsewhere during which no one wants capacity added.

**What has been reviewed, and what has not.** Tested on every paired run: the governor's permissions against the list above, read from the cluster itself; the refusal of every write past a cover, of every give-back while blind, and of every move under a hold; the lease and the watchdog's hand-back on a real cluster; the checksums of every download. Not yet done, and said so: an independent security review of the code by a party outside the company, and a penetration test of the deployed governor on a customer-like cluster. Until those are done, the operator's own review stands in for them, and this chapter tells the operator exactly what to look at.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

PART FIVE

# Checking It on Your Own System

*Paired runs and receipts on your own system, evidence classes, the engine version and the three-run rule.*

CHAPTER 10

# Paired Runs and Receipts on Your Own System

The method of every published benchmark is the method an installer uses on their own system, and it is deliberately simple: the same system, the same work, native and then omni back to back on the same machines, repeated with the order rotated, each gauge reported with the 95% interval of its paired difference. Pairing is what makes a difference of a few percent visible on a shared machine whose own noise is larger than that: two arms run minutes apart on one runner see the same neighbours and the same hour, so their difference is a difference between arms, while two arms on separate machines differ by about 15% in p95 on their own. The order rotates so that no arm always runs first, warm or cold. The work is held equal by an open-loop load, a fixed rate of requests at fixed moments, so that a lower resource count can never mean that less work was done. The steps below are the whole method.

1. **Paired runs.** Run your service the same way twice, once native and once with Omni-Compass on top, back to back on the same machines, the order rotated, at least 5 times (10 for a result you publish). `scripts/kind_paired.sh` and `tools/live_reps.py` do this on Kubernetes; `scripts/gpu_rented_run.sh` does it on a GPU box, in one command: the wire check, a smoke run, then the counted run.
2. **The switch drill** after every run: turn Omni-Compass OFF; check every setting is back at its recorded original; turn it ON.
3. **The receipt.** Each gauge: native, Omni-Compass, the change, the 95% interval of the difference, and whether the interval excludes zero. A change is proven only when it does.
4. **The label, by rule written before the run:** SUPERIOR WITHIN GUARDRAILS / ENERGY IMPROVEMENT WITH SERVICE TRADEOFF / NONINFERIOR / NOT ESTABLISHED / LESS

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

WORK PER ENERGY / INVALID. Guardrails: work not lower by more than 1%; the share of time outside the service line not higher by more than 1 percentage point. The band-first rule is stricter: no win is claimed while the time outside the service line is above native's.

5. **Write it down, then run.** The arms, the knob, the cover, the gauges and the readings are written down before the first counted run; the rules are fixed on one tuning case and then applied unchanged to the cases that are counted. A change made after a result has been seen is written down with its time and reason, and the receipt says so.
6. **Every row, losses included.** A row that went against Omni-Compass is reported with the others. If Omni-Compass loses, the first suspect is the wiring, then native's setup, then the scoring; that is fixed and the run repeated, and the loss stays in the record.
7. **Keep the raw files.** Every run's files (capture, response times, audit log, controller log, one SHA-256 manifest per repetition) are kept with the receipt, and every receipt names the runs and the commit it was made from, so that anyone holding the files can rebuild it.

**How many repetitions, and why.** The width of a paired interval shrinks with the square root of the number of pairs, $w \propto 1/\sqrt{n}$, so going from five pairs to ten narrows it by about a third ($1 - 1/\sqrt{2} \approx 0.29$), and from ten to forty by half again. On shared cloud runners, ten pairs of a 900 s window typically put the interval on a cluster's work inside the line at about ±10 points of native and on its 95th-percentile response time at ±15 to ±30 points: wide intervals, which clear zero only when the effect is tens of percent. A gauge computed from a declared formula over counts (an energy model over machines in service) has a much narrower interval, and a difference inside it is read as no difference, never as a small gain. A gain of a few percent on a cluster needs forty pairs or more to clear zero at these widths. On a stack with a finer knob (connections, consumers, megabytes) the intervals are narrower at three pairs than a cluster's at ten, because the knob moves in hundreds of small steps rather than in whole machines. Run five pairs first, look at the interval widths on the gauges you care about, and set the count for the counted run from those widths, never from the result.

**Why the load is open-loop.** A closed-loop load generator waits for each answer before sending the next request, so a slower system is offered less work and its throughput "improves" its own latency; two arms under closed-loop load are not doing the same work. An open-loop generator sends requests at fixed moments whatever the system does, so both arms are offered the same work to the request, a queue forms when the system falls behind, and the time over the line and the failed requests record the cost. Every paired-run harness (the cluster's load generator, pgbench's rate limit, Kafka's producer at a fixed rate, Redis and YCSB at a fixed target rate) offers its work open-loop, and a lower resource count in the omni arm can therefore never mean less work was asked. A burst is a schedule of open-loop rates written before the run and applied identically in both arms; only the load generator reads it, never the governor.

**What a receipt looks like.** One row per gauge: the gauge's name and direction; native's value and omni's; the paired change as a percentage of native with its 95% interval; the reading. A head naming the system, the knob, the cover, the window, the number of pairs, the order rotation, the engine's fingerprint and the commit; a foot saying whether every arm was handed back. `tools/live_reps.py` prints exactly this for a

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

Kubernetes run and `tools/confirm_abc.py` prints it for three runs side by side; a receipt from any other tool should be laid out the same way so that a reader of this manual can read it without learning a new shape.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 11

# Evidence Classes and How to Read a Receipt

**One number, one word.** In every receipt a change keeps its raw sign: a response time that fell reads "-65%", a cache that grew reads "+300%", and the Reading column beside it gives its word, GAIN, EVEN or LOSS, decided by what the gauge is for and never by its sign (the key at the front of this manual): a response time cut 65% is GAIN, a cache grown 300% is **power given**, a fall in work is LOSS. Wherever a change is written in a sentence, it is written in words, never as a bare sign: "p95 cut 65% (GAIN)", so it cannot be misread against a receipt that shows the same change as "-65%". Read the Reading column, never the sign alone.

| Class | Rung | What it is | What it can show |
|---|---|---|---|
| T / V | E1 | deterministic tests of the software | the software does what its documentation says |
| S | E2 | a run on a modelled plant | what the governor does to the model, and where it breaks |
| L | E3 | real software, no hardware meter | real decisions on real software; energy there is a declared model |
| P | E4 | a physical meter (a card's own power reading, the processor's energy counters, a site meter) | the hardware's own answer |

Read every number with its class beside it. A modelled number is never quoted as a hardware result. When a receipt's energy line is modelled, the receipt says so.

**What each class can and cannot carry.** A class **T / V** result says that the software computes what its documentation says it computes; it says nothing about whether the governor helps any machine. A class **S** result says what the governor does to a model of a plant under that model's own native controller; it carries exactly the model's fidelity and no more, and every S line that reports energy says that the energy is a declared model. A class **L** result says what the governor did to real software making real decisions on real requests; where the machines are containers on one host, they are counted in machine-hours in service and their energy, where it is reported, is a declared formula, which the receipt says; the processor seconds of the host are

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

measured. A cloud bill is an L result with a real bill: the machines billed are the provider's own count and the price its list price, and the energy is still not a meter. A class **P** result says what a physical meter read, and only a P result speaks for hardware energy.

**How to read a row.** Every row in every three-run table has the same shape: the gauge and its direction; native's value in run A and omni's value in run A, so the size of the numbers is visible; the paired change in each of the three runs as a percentage of native with its 95% interval; and the reading. Read the reading first, then the intervals, then the sizes. A reading of "GAIN in all three of our runs" with intervals of −0.5% to −0.1% is a small, real effect; a reading of "no difference beyond the noise (3 of 3 runs)" with intervals of −30% to +25% is a test that could not see an effect of that size either way, which is a statement about the test's power, and the receipt should say so (a 4-worker fleet cannot show a saving smaller than one machine, section 7.1). A "LOSS in all three runs" row stands in the receipt.

**Three questions to ask of any class L result.** First, what was native, exactly: which controller, at which published version, with which settings, and were those settings an operator's or ours? (Write them down before the run; a native set anywhere other than the operator's own setting must say so.) Second, what did the governor spend: machines, memory, connections, CPU, its own cost? (Every receipt has the resource rows; a result with a gain and no resource row is not a receipt.) Third, who else could have produced the gain: the order of the arms, the warm-up, the runner's neighbour, the clock? (Rotation of the arm order, the warm-up before every window, the paired design on one machine and the clock rule are the answers; whatever residue they do not remove is written beside the receipt.)

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 12

# The Engine Version and the Three-Run Rule

**Every engine version has a fingerprint.** The engine is fingerprinted file by file: one SHA-256 per file and one digest over all of them, held in the version's registry (`OMNI_V5.json` for the engine that stands). Any change to a rule, guard, preset or muscle makes **the next version**, and every receipt is made again on it; no receipt is ever read across versions. When the engine is declared final, the engine that stands then is published as Omni-Compass 1.0.

```
python3 tools/omni_version.py                    # this checkout: its e
    ↪ ngine version, or every file that differs
python3 tools/omni_version.py --commit &lt;sha&gt;    # the engine at the co
    ↪ mmit any receipt ran on
```

A receipt names the commit it ran on, and every table made by rule checks each run's commit against the fingerprint and heads itself with a warning if any run is not the engine it claims or if the three are not separate runs. An operator's own tune (section 3.6) does not change the engine's fingerprint, and must therefore be written down beside the receipt.

**The three-run rule.** A result is run as **A, B and C**, three separate runs on the same engine version. Every judged row gets one of three readings, and all three runs are shown beside it:

- **GAIN** or **LOSS in all three runs** (a resource spent that rose reads **power given**, never a loss): the same sign in all three runs, each with its 95% interval clear of zero.
- **No difference beyond the noise** (EVEN): in at least one run the interval includes zero, so native and omni could not be told apart on that measure. That is the result, stated as such, with the count of runs it holds in.
- **The runs disagree**: runs clear of the noise point different ways. The test itself is then unstable on that measure, and it is looked into before anything is claimed.

The three-run table is made by a tool, never by hand: `tools/confirm_abc.py` for the paired Kubernetes and cloud runs, with one sibling tool per kind of raw record for the other stacks (`pgbench_abc.py`, `kafka_abc.py`, `redis_abc.py`, `ycsb_abc.py`, `sysbench_abc.py`), each checking the engine of every run it reads. A deterministic simulator's three runs must reproduce each other to the digit, or to the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

declared tolerance where the simulator is not bit-reproducible across machines.

**Why three separate runs, and why the readings are what they are.** One run of ten paired repetitions gives one interval, and an interval from one run can be narrow by luck: the machine was quiet that hour, the order of arms happened to favour one side. A second run on another day and another machine is a replication; a third makes a coincidence expensive. The three runs are separate runs, on separate machines, at separate hours, with separate installations of every dependency, so what agrees across them is not an artefact of one machine. The readings are designed so that no reading can be mistaken for a weaker form of "yes": a row counts only when all three runs move the same way with every interval clear of zero; a single run whose interval crosses zero makes the row "no difference beyond the noise", and the count says how many of the three; and two clear runs that disagree in sign are reported as a disagreement, which is an instability in the test to be found and fixed, not averaged away.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

PART SIX

# License and Your Copy

*The terms the software and this manual are licensed under, and how to verify a copy before it is installed.*

CHAPTER 13

## License and Commercial Terms

The software and this manual are licensed under the Omni-Compass Evaluation License (`LICENSE`): evaluation and simulation use only. Everything else - commercial use, production use, operating any system beyond evaluation, redistribution, a hosted or managed service, incorporation into a product or service, or using the software or its results to build a competing product - requires a written Omni-Compass Enterprise License signed by The Omni-Compass LLC and paid for. All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC. No patent or trademark license is granted for any other use. Contributions are accepted only on the terms in `.github/CONTRIBUTING.md`, which assign their rights to The Omni-Compass LLC.

The copyright in Omni-Compass belongs to The Omni-Compass LLC. The source is visible under the Evaluation License, which is not an open-source license; seeing the source grants no right beyond it. The owner of a copyright may grant it under more than one license at once (the Evaluation License to everyone, a paid Enterprise License to a customer), may set the terms of each new version, and may assign the copyright; copies already received keep the terms they were received under. The license texts (`LICENSE`, `NOTICE`, `DISCLOSURES.md`) govern.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 14

# Verifying Your Copy

Before a copy is installed anywhere, and again after every upgrade, it is checked in three ways, and an installation proceeds only when all three pass:

1. **The whole check.** `python3 verify.py` runs every test and every fingerprint check and ends with `VERIFICATION: PASS` or `FAIL`. A copy that fails it is not installed.
2. **The engine version.** `python3 tools/omni_version.py` prints the engine version the copy carries, or every file that differs from it. Receipts made on one version are never read against another (chapter 12).
3. **The seal and the manifest.** `python3 tools/seal.py --check` confirms that every sealed file is the file that was sealed, and the release manifest (`RELEASE_MANIFEST.json`) names the commit, the engine and the verification receipt the copy was released with. A copy whose files do not match its manifest is not the release it claims to be.

A reader holding any one of the three can tell whether the files in front of them are the files a receipt was made on. Every copy, export or printout carries the legal notice at its head and foot, with `LICENSE`, `NOTICE` and `DISCLOSURES.md` beside it.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

# Back Matter

BACK MATTER

## Glossary

| Term | Meaning |
|---|---|
| Authority | how far the governor lets a muscle move this decision; adding is always allowed, giving back needs calm and a clean service record (chapter 5) |
| Band | a reading's or a knob's safe range |
| Card | a muscle's own settings, found for it and kept only where it was never behind native; the factory tune for that muscle (section 3.3) |
| Dial | max (the default) or save (the tune-down), the same switch on every governor (section 3.2) |
| Factory tune | the engine as The Omni-Compass LLC tuned and ships it: every muscle on its own card or the engine's setting (section 3.1) |
| Line | the limit the service must meet, from the operator's own objectives (`--slo-ms`) |
| Engine version | the engine as fingerprinted file by file; a receipt belongs to the version its commit carries (chapter 12) |
| Compass | the band seen as a position from 0 (calm) to 1 (the line), with a 5% cushion at each edge and the centre where the reading is held (section 3.4) |
| EVEN | Equalized, Verified baseline, Equilibrium, Neutral: native and Omni could not be told apart (the same, or no difference beyond the noise); the key, front matter |
| GAIN | Growth, Advantage, Improvement, Net-positive: a gauge moved the way its declared aim asks, against native; a resource cut is a GAIN; the key, front matter |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Term | Meaning |
|---|---|
| GAIN in all three of our runs | the same sign in all three runs (A, B, C), every 95% interval clear of zero, and that sign a GAIN for the gauge's aim |
| LOSS | Leakage, Off-track, Setback, Subpar: a gauge moved against its declared aim (less work, a longer wait, more misses); it should never appear, and where it does it is shown, its cause found and fixed; the key, front matter |
| Power given | a resource spent or held that rose: what Omni put in to buy the work and speed beside it; shown, never called a loss |
| Cover | a knob's hard range; every write is clipped to it |
| Cushion | the 5% at each edge of the compass, 10% in all |
| DEX-YLL-1 | the company's number, 1-888-DEX-YLL-1, read in order as the engine runs: DEX, Data Exchange (the input, the two-way plug); YLL, Yield Limit Line (the boundary: every muscle up to its own yield limit, never past it); 1, one closed loop (the output; in code 1 is true, in science 1 is unity) |
| Fail up | the return to full capacity the moment service crosses the wall or a sense goes blind |
| Governor | the process that reads, decides and writes; Omni-Compass |
| Muscle | any machine, service or controller Omni-Compass reads and sets |
| Native | the system as it runs without Omni-Compass |
| No difference beyond the noise | an interval over zero in at least one run: native and omni could not be told apart; the result, stated as such |
| Plug | the two-way connection to one muscle: read, write, read back, restore |
| Pool (Azure) | one machine family under Azure's autoscaler with its own ceiling; several pools make the fleet |
| Profile | service (the default) or batch: what the pedals do with a pile of work (section 3.5) |
| Race | full speed at once while work waits, so a burst is never served slowly |
| Yield limit line | a muscle's own limit, read live: Omni-Compass takes it right up to the line and never past it, as an engineer loads a material up to its yield limit, the most it carries and still springs back unharmed |
| Speed floor | the clock a card reaches on its own while busy, learned from its own meter; the governor never sets the ceiling under it |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Term | Meaning |
|---|---|
| Receipt | the paired record of native against Omni-Compass for one run |
| Snapshot | a knob's value read once before the first write; the restore point |
| The runs disagree | runs clear of the noise point different ways; the test is unstable on that measure and is looked into |
| Wire check | the test that proves every wire follows, reads back and returns before anything runs |
| Work per energy | work done divided by energy used |
| Work inside the line | the requests, transactions or messages a second answered within the response line; the product number on every real stack |
| One pedal | push for more, lift off for less, idle (the pedal lifted all the way), reset (everything back to native): how the governor moves capacity under autopilot |
| Cruise | every machine in service while work waits for a place, held without second-guessing until the line has been empty for two decisions |
| Idle at once | the work done and demand at zero: the pedal lifted all the way in one move, every safety check still holding |
| Kill switch | security only: one switch in a human hand that turns every governor on the machine off at once; never the reset |
| Lease | the record every governor renews each decision; a lease older than its limit, or a dead process, makes the watchdog hand back for it |
| Watchdog | the service beside the governors that hands back for any governor that died or hung without doing it itself |
| Verdict, paired trial | the trial on the muscle itself that allows a slow knob to move only where the cost is within 2%; "left native" where no step passes (section 1.2) |
| Tuning case | the one workload or case the rules were fixed on; shown in every receipt and never counted |
| Untouched case | a workload or case the rules were applied to unchanged; the judged rows |
| Amendment | a change to the written rules of a paired run after they were written, with its time and reason; one made after a result was seen says so |
| Dwell | the seconds after a knob moves during which the law will not move it back |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Term | Meaning |
|---|---|
| Notch | the smallest step a knob moves in one decision (one consumer, 8 MB of cache, one tap) |
| Evidence class P, L, S | a physical meter; live software with no meter; a model (chapter 11) |
| Handed back | the knob read back at the operator's value at the end of an arm; every receipt says whether every arm was |
| Working set | the records a workload actually touches over a window; a cache benchmark measures nothing unless the working set reaches the cache |
| Full-cache gate | the MongoDB and Redis rule: slow reads grow the cache only while the cache is full (nine tenths used), because a miss in a cache with room to spare is not the cache's to mend |
| Buffer pool | InnoDB's cache of table and index pages, the one memory setting every MySQL operator sizes; resized online by the server in whole chunks |
| Chunk | the unit in which MySQL resizes its buffer pool (128 MB as shipped); the knob's notch on that stack |
| Page cache | the operating system's own cache of file contents; where the data fits in it, a storage-engine miss is a read from memory and a decompression, not a disk read, and the gain available to the cache knob is smaller |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix A. Command Reference

| Task | Command | | | |
|---|---|---|---|---|
| Verify the copy | `python3 verify.py` (ends `VERIFICATION: PASS`) | | | |
| Which engine version a checkout or a commit carries | `python3 tools/omni_version.py [--commit <sha>]` | | | |
| Seal check | `python3 tools/seal.py --check` | | | |
| Evaluate without touching anything: the organisms, modelled | `python3 tools/run_scale.py --runs N --scale K --out <dir>` | | | |
| Evaluate without touching anything: the two-wire card, modelled | `python3 tools/run_gpu_card.py <dir>` | | | |
| Kubernetes, watch (read-only) | `kubectl apply -f deploy/install/omni-compass.yaml` (it runs `--mode observe`) | | | |
| Kubernetes, govern | `python3 -m omni_controller.controller --mode target | nodepool --latency-file <feed> --slo-ms <line> [--law compass] [--dial max | save ]` | | |
| A response-time feed against any HTTP endpoint | `python3 scripts/latency_probe.py <url> <out.csv>` | | | |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Task | Command | | | |
|---|---|---|---|---|
| GPU wire check | `sudo python3 tools/gpu_wire_check.py --gpu 0` (must end `WIRED RIGHT`) | | | |
| GPU governor, two wires | `sudo python3 -m omni_controller.gpu_compass --mode watch | cap --gpus 0 --latency-file <feed> --slo-ms <line> [--dial max | save ]` | |
| GPU, the whole paired run in one command | `sudo nohup bash scripts/gpu_rented_run.sh > run.log 2>&1 &` | | | |
| The processor's clock under its meter, the paired run | `sudo bash scripts/cpu_power_run.sh` | | | |
| Kubernetes paired runs, then the receipt | `scripts/kind_paired.sh`, then `python3 tools/live_reps.py <dir>` | | | |
| The three-run table (Kubernetes, cloud) | `python3 tools/confirm_abc.py "<title>" <run A dir> <run B dir> <run C dir> --out <table.md>` | | | |
| The three-run table (pooler, broker, cache, storage engine, buffer pool) | `tools/pgbench_abc.py`, `tools/kafka_abc.py`, `tools/redis_abc.py`, `tools/ycsb_abc.py`, `tools/sysbench_abc.py`, the same arguments | | | |
| OFF, one muscle (Kubernetes) | `touch /tmp/omni.kill` | | | |
| OFF, one card | `sudo touch /tmp/omni-gpu-kill` | | | |
| The whole harness OFF, ON, status, watchdog | `python3 tools/omni_switch.py off | on | status | watchdog` | | |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

APPENDIX

# Appendix B. The Files an Installation Touches

| Path | What it is for |
|---|---|
| `verify.py`, `tools/omni_version.py`, `tools/seal.py`, `RELEASE_MANIFEST.json` | checking the copy before it is installed (chapter 14) |
| `deploy/Dockerfile` | the container image: non-root, read-only root filesystem, no Linux capabilities |
| `deploy/install/omni-compass.yaml` | the watch-mode install with a read-only identity (level 1) |
| `deploy/rbac-target.yaml`, `deploy/kind/rbac-omni.yaml` | the permissions of levels 2 and 3, exactly the verbs each level needs |
| `scripts/latency_probe.py` | the response-time feed against any HTTP endpoint |
| `scripts/kind_paired.sh`, `tools/live_reps.py`, `tools/confirm_abc.py` | the paired runs, the receipt and the three-run table on Kubernetes |
| `tools/gpu_wire_check.py`, `scripts/gpu_rented_run.sh`, `tools/gpu_workload.py` | the card's wire check, its paired run, and a workload that writes the card's own time per request |
| `scripts/cpu_power_run.sh` | the processor's paired run on bare metal |
| `tools/omni_switch.py` | the master switch, the status and the watchdog (chapter 8) |
| `realms/catalog_v5.csv`, `realms/muscle_cards_v5.csv`, `docs/MUSCLE_CATALOG.md` | every muscle, its wiring and its own card (Appendix E) |
| `docs/AZURE_SETUP.md` | the Azure account and its secrets, click by click |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Path | What it is for |
|---|---|
| `LICENSE`, `NOTICE`, `DISCLOSURES.md`, `SECURITY.md` | the license, the notices, every declaration and disclaimer, and how to report a security issue |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix C. The Gauges

Every gauge in a receipt, where it comes from, and whether it is measured or modelled:

| Gauge | Source | Measured or modelled | Its aim |
|---|---|---|---|
| work inside the response line (requests, transactions or messages a second answered within the line) | the probe's own records, pgbench's log, the consumers' records, the application's records | measured | higher; the product number |
| response time, mean, median, 95th and 99th percentile | the same | measured | lower |
| time over the response line (share of samples) | the same, against the written line | measured | lower |
| failed requests, lost messages | the same | measured | lower; any increase is marked |
| alert minutes: whole minutes whose 95th percentile, failed requests counted over, was over the line | the probe's own records, minute by minute | measured | lower: the minutes an on-call page would be open |
| settings commands the governor wrote on its own | the governor's audit | measured | shown, not judged (no person touched either arm) |
| the governor's additions decided while the service was still inside its line (% of its additions) | the governor's audit | measured | shown, not judged |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Gauge | Source | Measured or modelled | Its aim |
|---|---|---|---|
| worker machines in service, mean; node-hours | the cluster's own node records every 15 s | measured (machine-hours on a local cluster; the provider's own count in a cloud) | lower; a rise is power given |
| compute bill at list price | the provider's own machine count × list price per family | measured (a real bill) | lower; a rise is power given |
| energy (Wh), the standby model | a declared formula over the machines in service | **modelled**, said so on every line | lower; a rise is power given |
| energy, the machine's own meter | the processor's RAPL counters, a card's own power reading, a site meter | measured (class P) | lower; a rise is power given |
| host CPU busy; host CPU-seconds; CPU-seconds per 1,000 units inside the line | `/proc/stat` of the real machine under the run | measured | lower |
| Omni's own CPU (cores), mean | the governor's audit (its process and every command it ran) | measured | shown beside the receipt |
| connections held open, consumers running, memory ceiling held, cache size held | the pooler's, the group's, the cache's and the server's own reports | measured | lower; a rise is power given |
| cache hit rate; consumer lag | Redis's INFO; the consumers' records | measured | higher; lower |
| pages read into a cache, the cache's miss share | the server's own counters | measured | shown, not judged |
| HPA replicas, pods started, pod start wait | the API server's own records | measured | shown, not judged except pod start wait (lower) |
| seconds from a kill to every setting back at the operator's | the harness polling the system once a second from the kill | measured | lower; past 60 s marked; never, INVALID |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Gauge | Source | Measured or modelled | Its aim |
|---|---|---|---|
| handed back (every arm) | the knob read back at the end of the arm | measured | yes in every arm, or the run is invalid |

**Three rules for reading any gauge in this appendix.** A measured gauge is one the system under test, or the host's own kernel, reported about itself; a modelled gauge is one computed from a declared formula over measured inputs, and every receipt says which it is on the line itself. A resource gauge (machines, connections, consumers, memory, cache) is always read against an aim of lower, even where the governor's purpose was to add the resource to buy service, so that a gain bought with a resource is shown as power given in the same receipt. And a gauge marked "shown, not judged" never enters a reading; it is there so that a reader can see what the governor did (how often it moved, how many pages it read, how many pods it started), not to be counted for or against it.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix D. Troubleshooting

| Symptom | Cause | Fix |
|---|---|---|
| Wire check `FAIL 3 up wire (lock)` | the driver or VM refuses clock locking | use bare metal or full passthrough; or run the one-wire governor |
| Wire check `FAIL 3 up wire (follows down)` | the card ignores the ceiling | update the driver; check for another management agent holding clocks |
| Wire check `FAIL 4 down wire` | the power limit is refused or capped by the board | check `nvidia-smi -q -d POWER`; run as root |
| `another copy of this test is already running` | a second copy was started | wait for the first or reboot; start once |
| `something else is using the GPU` | another process holds the card | stop it; the paired run must run alone |
| Governor exit 5 | another writer changed a knob | find the other controller; Omni-Compass left its value alone (section 5.2) |
| Governor exit 3 | a restore did not read back | restore by hand (`nvidia-smi -rgc`, `-pl <start>`); investigate before running again |
| `decision failed (n in a row)` | the cluster API is unreachable | turn it OFF; native runs on |
| Energy saved but p95 slower than native | the card served bursts below its own clock, or the lid sat under its own draw | read section 4.5; in the audit compare `telemetry.clock_mhz` while busy with `native_busy_clock_mhz`, and `want_w` with `native_busy_draw_w` |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Symptom | Cause | Fix |
|---|---|---|
| Fail-up in most decisions | `--slo-ms` set too low for the workload, or a stale feed read as blind | set `--slo-ms` from the workload's own target; check the feed's age (section 5.3) |
| No saving at all, service unchanged | the card is saturated almost all the time (it races), or the dial is at the max | expected: the max spends where it buys work; the tune-down (section 3.2) saves in the quiet stretches |
| Kubernetes: no machine ever given back | the dial is at the max (no machine below the cluster's own is given back), or the release gate refuses (its reason is in the audit) | expected at the max; at the tune-down read `node_gate.reason` in the audit and check `--slo-ms` and the latency feed |
| A pooler's clock reads calm while users wait | the pooler cannot see a client waiting behind its own schedule | wire the queue share beside the service time and read the larger (section 7.2) |
| Broker: consumers climb to the cover and stay | messages wait at every step (the producer is past the group's capacity) | check the offered rate against the group's measured capacity in the run's log |
| Cache: the ceiling grows while little memory is used | cold misses read as pressure | the full-cache gate is the rule (used ≥ 90% of the ceiling); check `used_mb` beside `maxmemory_mb` in the run's log |
| Cache: the ceiling never grows | the working set fits the operator's ceiling | expected: a cache that fits has nothing for the ceiling to mend, and the row reads inside the noise |
| Storage-engine cache: the cache never fills, whatever the load, and omni reads as a free memory saving | the requests concentrate on a few hot records, or the run asks for records the load did not write, so the reads are served from the index | a result that flatters the governor is the first thing to suspect: check the bytes in the cache against its size; draw requests over the whole key space and use the same key order in load and run; a knob with nothing to do is not a saving |
| **Symptom** | **Cause** | **Fix** |
| Storage-engine cache or buffer pool: the governor only ever reads calm | the line is set far above the stack's own latencies | set the line on the tuning case's own figures so that a hit is calm and a full-cache miss is slow, and |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Symptom | Cause | Fix |
|---|---|---|
| | | write the figures down before any counted run |
| Buffer pool: not handed back at the end of an arm | the server was still carrying out a shrink and ignored the restore | the plug waits for the resize and writes again (section 7.8); check the restore record in the arm's receipt |
| A three-run table heads itself with a warning | a run's commit is not the engine version it claims, or the three are not separate runs | run `tools/omni_version.py --commit <sha>` on each; make the table only from runs on one engine version |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix E. The Muscles: What Each Is For, and How It Is Wired

A **muscle** is one setting on one machine that already has its own control: a replica target, a node pool's size, a GPU's clock ceiling, a chiller's setpoint, a battery's reserve, a joint's effort. Omni-Compass does not replace that control. It reads the machine's meters and, at its default, the max, moves the setting the machine already accepts toward the machine's own limit for the most work, never past it and never below native, through a plug that reads the setting once before the first write, reads back every write, steps aside if another controller moves it, and walks it back to native at the end. A knob native already runs at the top of its range is at its limit on native and is left there.

This chapter lists all 1107 muscles of the catalog (`realms/catalog_v5.csv`, engine omni-v5). For each one it says what kind of machine it is, which of the four kinds of knob it is, what Omni-Compass reads and does with it, how such a muscle is reached in a real stack, and which organisms it belongs to. The plants behind the benchmark numbers are models of these machines (evidence class **S**); a muscle in this list is wired on a real system only through the levels and the checks of the manual (chapters 4 and 6). See `DISCLOSURES.md`.

## How to read an entry

**The four kinds of knob.**

- **capacity**: how much of the machine is in service (replicas, instances, units, speed). At the max Omni-Compass pushes it to the machine's own limit for the most work, never past it and never below native.
- **admission**: what is let in (queues, rate limits, admission control). Native already lets everything in: the knob is at its limit on native, and Omni-Compass reads it and leaves it native.
- **power**: the power share or limit the machine may draw. Native already runs it at full: at its limit on native, read and left native.
- **setpoint**: the target the machine's own loop holds (a temperature, a reserve, a process value). At the max a room is cooled ahead of the heat and a battery's charge is held for its line, inside the safe band, never behind native on its line.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**The five kinds of plant.** Every muscle is modelled on one of five plants, each with its own service reading and lever:

| Plant | What it is | What Omni-Compass reads | What Omni-Compass moves |
|---|---|---|---|
| `compute_pool` | a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs | how far its queue and its load sit toward the service line (queue or lateness, and load above half) | the capacity it may use: its replica or instance count (through its autoscaler's target or floor), its power share, or one unit given back through the release gate |
| `thermal_zone` | a data hall or building zone cooled by chillers, air handlers or fans | its temperature against its limit | its supply setpoint (colder is more cooling), its power share, or one cooling unit given back when the room has margin |
| `energy_storage` | a battery, UPS or microgrid store with a load and a grid connection | its draw on the grid connection and its reserve | its reserve setpoint (how much charge it holds back); its power limit is left native |
| `motion_axis` | a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor | its tracking error and its load | its speed and effort share, or its power share |
| `process_loop` | a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint | how far the process sits from its band | its setpoint within its safe band, its actuator range, or its power share |

**Every muscle its own card.** The brain is one, its settings each muscle's own. A muscle's card holds the settings the max uses for that muscle (a machine pool: how full it runs on its forecast before it calls for more, and how many decisions of history it reads; a battery: how many wobbles of margin it keeps under its line, and, wired by its power, how long it reads its line before it holds and how much of it it keeps clear). The cards were found by an automatic search over every muscle on the development seeds, today's global setting always one of the choices, and kept only where the card beat the global setting beyond the noise on two separate sets of seeds and on no seed under any load fell behind native: never over its line more often, never less work, never its line cleared later (`tools/tune_cards.py`, `realms/muscle_cards_v5.csv`, `results/cards_v5/`). Inside a body, a battery wired by its power keeps the body's own margins on its line. 364 muscles carry a card of their own; every other muscle runs the global setting (machines started at 95% full, the last 20 decisions read, 3.09 wobbles of margin on a battery's line, 5% of it kept clear). Each muscle's card is in the last column of its family's table.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**The organisms.** The 1107 muscles build six organisms: each of the four realms (every muscle whose realm list includes it), the four stacked with every duplicate kept (1,908), and the whole tower with every muscle once (1107). A muscle of the shared spine sits in all four realms.

| Organism | Muscles |
|---|---|
| Compute / AI / Cloud | 540 |
| Physics / Robotics / Autonomous | 404 |
| Energy / Facility / Industrial | 480 |
| Distribution / Specialized | 484 |
| The four stacked, every duplicate kept | 1,908 |
| The whole tower, every muscle once | 1107 |

**How a muscle is reached in a real stack.** The class of machine decides the wire:

| Class of machine | What it is | The wire in a real stack |
|---|---|---|
| `batch` | batch and queued jobs | the job queue (Kubernetes Jobs, Slurm, Spark) |
| `batch_reactor` | a GMP batch reactor, fermenter or food process | the batch control system (ISA-88, OPC UA) |
| `building` | a building zone | the building management system |
| `chamber` | a controlled chamber (clean room, kiln, reactor) | the PLC or DCS |
| `clinical` | clinical systems: imaging archives, records, interface engines, monitoring gateways | the application's own scaling and admission settings (Kubernetes, the vendor console) |
| `commerce` | a customer-facing service with bursts | the Kubernetes API |
| `cpu_host` | a CPU host whose clock and power can be set | Linux cpufreq and RAPL |
| `data_hall` | a data hall cooling plant | the building management system (BACnet, Modbus) or its command line |
| `database` | a database cluster | the database operator (replicas, connection pools) |
| `district_heat` | a district heating or cooling network | the network control system and substation controllers (Modbus, M-Bus) |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Class of machine | What it is | The wire in a real stack |
|---|---|---|
| `elevator_hoist` | an elevator or escalator drive and its group control | the lift controller and destination dispatch (modelled only) |
| `ev_traction` | an electric vehicle traction motor | the vehicle's motor controller (CAN; modelled only) |
| `fabric` | a high-speed fabric (RDMA, DPU, SmartNIC) | the fabric manager's API |
| `facility` | a facility battery behind the meter | the energy management system |
| `feeder_voltage` | a distribution feeder's voltage | the distribution management system (IEC 61850, DNP3; modelled only) |
| `flight_axis` | a flight-control axis | the flight controller (modelled only) |
| `gpu` | GPU serving with a response-time target | nvidia-smi (clock ceiling and power limit) or the serving autoscaler |
| `gpu_batch` | GPU training and batch work | nvidia-smi, the job scheduler's pause and resume |
| `hospital` | a hospital's critical rooms (operating rooms, isolation, pharmacy, imaging suites) | the hospital's building management system (BACnet) |
| `inverter` | a plant of grid-support inverters or wind turbines | the plant controller and inverter settings (SunSpec, IEC 61850, IEEE 1547) |
| `irrigation` | pumps, pressure and climate on a farm or in a greenhouse | the pump station controller, the pivot panel or the climate computer (Modbus, the vendor cloud) |
| `marine_propulsion` | a ship's propulsion shaft and power plant | the vessel automation and power management system (modelled only) |
| `microgrid` | a microgrid battery and loads | the inverter or energy management system (IEEE 2030.5, SunSpec, OpenADR) |
| `mill` | a grinding, flotation or materials-handling circuit | the plant DCS or PLC (OPC UA) |
| `network` | network routing and switching | the network controller (SDN, routing API) |
| `node` | a fleet of machines or VMs that boot in minutes | the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG) |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Class of machine | What it is | The wire in a real stack |
|---|---|---|
| `pipeline` | a pipeline segment with its compressors, pumps and valves | pipeline SCADA (DNP3, Modbus, OPC UA) |
| `port` | a container terminal's cranes and vehicles taking moves | the terminal operating system and fleet controllers |
| `process` | an industrial process loop | the PLC or DCS (OPC UA, EtherNet/IP, PROFINET) |
| `qpu` | a quantum or specialised accelerator queue | the accelerator's job queue |
| `rail_traction` | a train's traction and auxiliary systems | the train control and management system and ATO (modelled only) |
| `ran` | a radio access network cell or site | the RAN controller (O-RAN interfaces) |
| `reaction_wheel` | a spacecraft reaction wheel | the attitude controller (modelled only) |
| `robot_fleet` | a fleet of robots or vehicles taking tasks | the fleet manager's dispatch API |
| `robot_joint` | a robot joint servo | the robot controller (ROS 2, EtherCAT, the drive's fieldbus) |
| `server` | an application service on servers or pods | the Kubernetes API (HPA target and floor, pod resize) |
| `storage` | block, file or object storage | the storage system's QoS and tiering API |
| `turbine` | a generating unit's governor, boiler or excitation loop | the plant DCS and governor (OPC UA, IEC 61850; modelled only) |
| `ups` | an uninterruptible power supply | the UPS and PDU management interface (SNMP, Modbus) |
| `water` | a water or pumping process | the SCADA system (Modbus, DNP3) |
| `workflow` | a workflow or pipeline engine | the workflow engine's concurrency settings |

Wires marked *modelled only* exist as plants in the benchmark and are not built for live use.

## The shared spine: 267 muscles in all four realms

The machines every realm stands on: servers, machines, GPUs and CPUs, network, storage, observability, security, cooling and electrical distribution. Each spine muscle is part of every realm's organism, counted once in the tower and four times in the stack.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

### *Cloud VM & Capacity (21 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 97 | vm count | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 98 | vm size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 40 decisions |
| 99 | vm start stop | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 100 | vm migrate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 101 | instance family | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 102 | asg min | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 103 | asg max | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 104 | spot mix | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 105 | reservation mix | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 40 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 106 | zone selection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 107 | region selection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 108 | architecture selection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 109 | accelerator selection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 110 | boot disk class | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 111 | placement group | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 112 | interruption response | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 1038 | target tracking policy | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 1039 | scaling cooldown warmup | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 1114 | vm cpu cap by priority | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 1120 | predictive scaling | capacity | starts the machines the forecast needs before the rush lands | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 1121 | instance refresh | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |

### *Container Resources (25 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 33 | cpu request | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 34 | cpu limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | than native runs; hands back what it no longer needs at once | | |
| 35 | memory request | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 36 | memory limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 37 | ephemeral storage | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 38 | hugepages | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 39 | io weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 40 | pids limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 41 | memory high | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 42 | memory reclaim | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 43 | swap limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 44 | cgroup io max | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 45 | cpu weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 46 | cpu quota | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 47 | cpuset | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 48 | runtime class | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1017 | in place resize | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 1019 | namespace quota | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1020 | default limits | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1022 | memory hard limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1023 | memory protection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 1024 | io latency target | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 1025 | freeze | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 2176 | cruisekube request rightsizing | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 2177 | goldilocks vpa request recommendation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *Host CPU & Memory (26 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `cpu_host` (a CPU host whose clock and power can be set), reached through Linux cpufreq and RAPL. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 113 | cpufreq min | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 114 | cpufreq max | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 115 | rapl package power | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 116 | uncore frequency | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 117 | energy perf preference | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 118 | cpu idle policy | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 119 | memory bandwidth | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 120 | numa balance | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 121 | irq affinity | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 122 | llc allocation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 123 | memory pressure gate | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 124 | page reclaim rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 125 | transparent hugepages | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 126 | core online offline | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 127 | thermal throttle policy | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 128 | host power profile | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 1040 | frequency governor | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 1041 | turbo boost | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1042 | dram power limit | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 1043 | swappiness reclaim | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 1112 | core packing | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 1115 | per pod cpu frequency range and c state access | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 1116 | uncore frequency per node | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 2174 | ear cpu frequency policy | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 2175 | ear global energy budget | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 2178 | k8s power manager core profile | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *Kubernetes Placement & Scheduling (21 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 49 | node selector | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 85% full; reads the last 10 decisions |
| 50 | node affinity | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 51 | pod anti affinity | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 52 | topology spread | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 53 | numa placement | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 54 | gpu topology | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 20 decisions |
| 55 | storage locality | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 56 | network locality | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 57 | taint toleration | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 58 | priority class | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 59 | preemption policy | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 60 | device claim | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 61 | failure domain spread | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 62 | scheduler backoff | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 63 | gang admission | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 64 | deschedule | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1026 | scheduler scoring strategy | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1027 | scheduler plugin weights | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 40 decisions |
| 1028 | scheduling gates | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1029 | cpu manager | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 2179 | peaks power aware placement | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *Kubernetes Workload Scaling (32 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 17 | replicas | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 18 | hpa cpu target | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 10 decisions |
| 19 | hpa memory target | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 20 | hpa custom target | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 21 | vpa apply | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 22 | scale to zero | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 80% full; reads the last 40 decisions |
| 23 | keda threshold | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 24 | rollout rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 25 | max surge | capacity | starts the machines the forecast needs before the | all four realms; stack; | machines started at 70% full; reads the last 10 |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | tower | decisions |
| 26 | max unavailable | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 27 | deployment pause resume | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 28 | rollout abort | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 29 | pod eviction | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 30 | pdb policy | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 31 | scheduler queue priority | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 32 | api priority fairness | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1001 | hpa min replicas | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 1002 | hpa max replicas | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 1003 | hpa scale down stabilisation window | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1004 | hpa scale up stabilisation window | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1005 | hpa scaling rate policies | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1006 | hpa policy selection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1007 | hpa tolerance | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1008 | hpa sync period | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1009 | vpa minmax allowed resources | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 1010 | keda polling interval | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1011 | rollout progress deadline | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1012 | rollout readiness delay | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 1014 | unhealthy pod eviction policy | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 1015 | graceful termination | capacity | starts the machines the forecast needs before the rush lands and clears work | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 1016 | probes | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 2172 | kepler energy target autoscaling | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |

### *GPU Hardware (22 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `gpu` (GPU serving with a response-time target), reached through nvidia-smi (clock ceiling and power limit) or the serving autoscaler. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 129 | gpu allocate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | longer needs at once | | |
| 130 | gpu power limit | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 131 | gpu sm clock | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 132 | gpu memory clock | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 133 | gpu persistence mode | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 134 | gpu compute mode | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 135 | mig mode | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 136 | mig geometry | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 85% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 137 | gpu timeslice | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 40 decisions |
| 138 | gpu mps | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 139 | gpu quarantine | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 140 | gpu reset | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 141 | gpu thermal limit | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 142 | gpu ecc response | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 143 | gpu job power budget | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1044 | application clocks | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 1045 | gpu temperature target | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 1117 | sync boost | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 1118 | target clocks | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 2173 | climatik inference power cap | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 2180 | adaptive gpu power cap | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 2181 | nvidia platform power budget | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

### *Node Fleet & Karpenter-Class Control (24 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 65 | node desired | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 66 | node pool min | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 10 decisions |
| 67 | node pool max | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 40 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 68 | node provision | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | all four realms; stack; tower | machines started at 75% full; reads the last 60 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 69 | node cordon | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 70 | node drain | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 71 | node consolidate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 72 | node replace | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 73 | node shutdown | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 40 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 74 | node power on | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 75 | nodepool weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 76 | disruption budget | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 77 | consolidation policy | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 20 decisions |
| 78 | consolidate after | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 90% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 79 | expire after | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 80 | capacity class | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 1030 | scale down utilisation threshold | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1031 | scale down unneeded time | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 1032 | scale down delay after add | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1033 | expander | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 80% full; reads the last 10 decisions |
| 1034 | node pool limits | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 1035 | kubelet eviction thresholds | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1036 | kubelet reserved resources | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 1037 | max pods per node | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *Network Routing & Switching (19 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `network` (network routing and switching), reached through the network controller (SDN, routing API). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 225 | lb weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 226 | route weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 227 | rate limit | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 228 | bandwidth limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 229 | qos class | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine | all four realms; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | start, never fewer than native runs; hands back what it no longer needs at once | | |
| 230 | connection limit | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 231 | failover route | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 232 | nic rate limit | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 233 | nic queue count | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 234 | queue discipline | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 235 | congestion control | capacity | starts the machines the forecast needs before the rush lands and clears work waiting | all four realms; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 236 | egress budget | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 237 | ingress budget | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 238 | ecmp weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 239 | path selection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 240 | network isolation | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1046 | nic interrupt coalescing | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1047 | nic ring size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 80% full; reads the last 10 decisions |
| 1048 | port link power | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |

### *Observability & Telemetry (9 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 373 | collector memory limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 374 | export concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 376 | cardinality budget | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 377 | retention window | setpoint | starts the machines the forecast needs before the rush | all four realms; stack; tower | machines started at 85% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 378 | remote write queue | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 379 | telemetry shed | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1053 | metric scrape interval | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1054 | batching | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 80% full; reads the last 10 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 1119 | per pod energy attribution | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | start, never fewer than native runs; hands back what it no longer needs at once | | |

### *Reliability, Security & Recovery (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 385 | restart | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 386 | rollback | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 388 | traffic divert recovery | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 389 | degraded mode | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 390 | actuator freeze | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 391 | workload isolate | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 393 | credential rotation gate | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 394 | policy enforcement | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 395 | rate abuse gate | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 396 | fault domain isolate | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 397 | backup trigger | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | start, never fewer than native runs; hands back what it no longer needs at once | | |
| 400 | kill switch | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |

### *Storage Block/File/Object (20 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `storage` (block, file or object storage), reached through the storage system's QoS and tiering API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 257 | volume size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 258 | iops limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 259 | throughput limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never | all four realms; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | fewer than native runs; hands back what it no longer needs at once | | |
| 260 | replica count | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 261 | storage tier | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 262 | volume placement | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 263 | snapshot trigger | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 10 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 264 | rebalance | capacity | starts the machines the forecast needs before the rush lands and | all four realms; stack; tower | machines started at 75% full; reads the last 10 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 265 | recovery rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 266 | backfill rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 267 | compaction pressure | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 268 | cache allocation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 269 | object replication | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 270 | erasure code profile | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 271 | storage admission | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 272 | degraded storage gate | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1049 | scrub schedule | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 1050 | io scheduler | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1051 | disk power spin down | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 1052 | lifecycle tiering | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |

### *Cooling, Chillers & Thermodynamics (18 muscles)*

Each is a data hall or building zone cooled by chillers, air handlers or fans; its class is `data_hall` (a data hall cooling plant), reached through the building management system (BACnet, Modbus) or its command line. Omni-Compass reads its temperature against its limit. It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 545 | supply air temperature | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | all four realms; stack; tower | global |
| 546 | return air target | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | all four realms; stack; tower | global |
| 547 | coolant supply temperature | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | all four realms; stack; tower | global |
| 548 | coolant flow | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | all four realms; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 549 | pump speed | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | all four realms; stack; tower | global |
| 550 | fan speed | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | all four realms; stack; tower | global |
| 551 | chiller setpoint | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | all four realms; stack; tower | global |
| 552 | compressor authority | power | at its limit on native (the site envelope at full): read and left native | all four realms; stack; tower | global |
| 553 | cooling tower fan | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | all four realms; stack; tower | global |
| 554 | cooling capacity | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | all four realms; stack; tower | global |
| 555 | rack thermal budget | power | at its limit on native (the site envelope at full): read and left native | all four realms; stack; tower | global |
| 556 | gpu thermal envelope | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | all four realms; stack; tower | global |
| 557 | cpu thermal envelope | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | all four realms; stack; tower | global |
| 558 | thermal workload migrate | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 559 | thermal load shed | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1055 | server fan speed | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | all four realms; stack; tower | global |
| 1056 | economiser | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | all four realms; stack; tower | global |
| 1122 | chiller plant supervisory control rl with safety layer | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | all four realms; stack; tower | global |

### *PDU, UPS & Electrical Distribution (18 muscles)*

Each is a battery, UPS or microgrid store with a load and a grid connection; its class is `ups` (an uninterruptible power supply), reached through the UPS and PDU management interface (SNMP, Modbus). Omni-Compass reads its draw on the grid connection and its reserve. It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 529 | server power cap | power | at its limit on native (full battery power): read and left native | all four realms; stack; tower | global |
| 530 | rack power cap | power | at its limit on native (full battery power): read and left native | all four realms; stack; tower | global |
| 531 | pdu branch power limit | power | at its limit on native (full battery power): read and left native | all four realms; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 532 | pdu outlet control | capacity | holds the charge for the line and clips only the draw that | all four realms; stack; | 0 wobbles of margin on its |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | would cross it; a UPS lends only the charge above its outage floor | tower | line |
| 534 | ups operating mode | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | all four realms; stack; tower | 0 wobbles of margin on its line |
| 535 | ups charge rate | power | at its limit on native (full battery power): read and left native | all four realms; stack; tower | global |
| 536 | ups discharge rate | power | at its limit on native (full battery power): read and left native | all four realms; stack; tower | global |
| 537 | phase balance | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | all four realms; stack; tower | 0 wobbles of margin on its line |
| 538 | load transfer | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 539 | reactive power target | setpoint | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | all four realms; stack; tower | 0 wobbles of margin on its line |
| 540 | voltage target | setpoint | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | all four realms; stack; tower | 0.25 wobbles of margin on its line |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 541 | generator dispatch | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | all four realms; stack; tower | 0 wobbles of margin on its line |
| 542 | electrical isolation | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 543 | breaker trip gate | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1057 | ups shutdown battery test | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | all four realms; stack; tower | 0 wobbles of margin on its line |
| 1110 | hierarchical power budget row pdu switchboard controllers | power | at its limit on native (full battery power): read and left native | all four realms; stack; tower | global |
| 1111 | priority aware capping throttle low priority first | power | at its limit on native (full battery power): read and left native | all four realms; stack; tower | global |
| 1113 | power oversubscription with prediction | power | at its limit on native (full battery power): read and left native | all four realms; stack; tower | global |

# Realm: Compute / AI / Cloud (273 muscles of its own, 540 in its organism with the spine)

## *AI Inference Serving (24 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `gpu` (GPU serving with a response-time target), reached through nvidia-smi (clock ceiling and power limit) or the serving autoscaler. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 161 | model replicas | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 162 | model route weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 163 | model load | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 164 | model unload | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 165 | model instance count | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 166 | continuous batching | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 167 | max batch size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 168 | batch queue delay | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 169 | inference concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 170 | inference max tokens | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 85% full; reads the last 60 decisions |
| 171 | kv cache budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 172 | prefix cache budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 173 | speculative decode budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 174 | model precision | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |
| 175 | inference priority | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 176 | inference slo gate | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 1058 | autoscaling target concurrency qps | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 1059 | scale to zero panic window | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 1060 | request priority levels | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 1061 | kv cache swap space | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | start, never fewer than native runs; hands back what it no longer needs at once | | |
| 1062 | chunked prefill | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 80% full; reads the last 20 decisions |
| 1063 | tensor parallel degree | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 80% full; reads the last 40 decisions |
| 1064 | disaggregated prefill decode | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1065 | lora adapter load | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *AI Training (18 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `gpu_batch` (GPU training and batch work), reached through nvidia-smi, the job scheduler's pause and resume. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 177 | training workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 178 | global batch size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 85% full; reads the last 10 decisions |
| 179 | microbatch size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 90% full; reads the last 10 decisions |
| 180 | gradient accumulation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 181 | data parallelism | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 182 | tensor parallelism | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 10 decisions |
| 183 | pipeline parallelism | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 10 decisions |
| 184 | expert parallelism | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 85% full; reads the last 20 decisions |
| 185 | checkpoint interval | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 186 | checkpoint trigger | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 187 | training preempt | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 188 | training gang size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 80% full; reads the last 10 decisions |
| 189 | elastic worker count | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 90% full; reads the last 10 decisions |
| 190 | straggler mitigation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 20 decisions |
| 191 | training precision | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 192 | compute comm overlap | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 80% full; reads the last 40 decisions |
| 1066 | activation checkpointing | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 1067 | dataloader workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |

### *Cross-Cluster, Multi-Region & Edge (18 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 609 | multi cluster dispatch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at | Compute; stack; tower | machines started at 70% full; reads the last 60 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | once | | |
| 610 | region dispatch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 40 decisions |
| 611 | zone dispatch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 612 | edge dispatch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 613 | cloud capacity class | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 614 | workload migrate region | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | fewer than native runs; hands back what it no longer needs at once | | |
| 615 | data residency gate | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 616 | latency region gate | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 617 | cost region gate | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 618 | carbon region gate | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 619 | global failover | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 40 decisions |
| 620 | federation quota | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 621 | cross cluster replication | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 622 | edge offload | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 623 | global admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5128 | edge site node pool | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| V5-5129 | edge inference gpu clocks | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |
| V5-5130 | edge sync interval | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |

### *DPU SmartNIC & Programmable IO (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `fabric` (a high-speed fabric (RDMA, DPU, SmartNIC)), reached through the fabric manager's API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDIT-0021 | dpu pf tx rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| AUDIT-0022 | dpu vf tx rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| AUDIT-0023 | dpu sf tx rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| AUDIT-0024 | dpu bandwidth share | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| AUDIT-0025 | dpu qos group | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDIT-0026 | sriov vf count | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| AUDIT-0027 | smartnic flow steering | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 80% full; reads the last 10 decisions |
| AUDIT-0028 | smartnic offload enable | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| AUDIT-0029 | dpu cpu budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| AUDIT-0030 | dpu memory budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| AUDIT-0031 | dpu service placement | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDIT-0032 | dpu failover | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |

### *Distributed Cluster Managers (15 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `batch` (batch and queued jobs), reached through the job queue (Kubernetes Jobs, Slurm, Spark). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 209 | task admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 210 | task priority | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 211 | resource reservation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 80% full; reads the last 10 decisions |
| 212 | cluster quota | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 213 | task preemption | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 214 | binpack pressure | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 85% full; reads the last 10 decisions |
| 215 | spread pressure | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 80% full; reads the last 10 decisions |
| 216 | gang schedule | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 90% full; reads the last 60 decisions |
| 217 | worker allocation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 218 | maintenance evacuation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 90% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 219 | oversubscription | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 10 decisions |
| 220 | resource reclaim | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 85% full; reads the last 20 decisions |
| 221 | queue fairness | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 222 | deadline pressure | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 223 | scheduler retry | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |

### *GPU Fabric & RDMA (18 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `fabric` (a high-speed fabric (RDMA, DPU, SmartNIC)), reached through the fabric manager's API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 145 | nvlink placement | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 146 | nvswitch route | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 147 | gpu fabric quarantine | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 148 | rdma bandwidth | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 10 decisions |
| 149 | rdma route | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 150 | nic affinity | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 151 | collective concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 152 | collective algorithm | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 153 | collective chunk size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 154 | rank placement | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 155 | gpudirect policy | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 156 | congestion response | capacity | starts the machines the forecast needs before the rush lands and | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 157 | rail selection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 158 | fabric failover | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 159 | communication priority | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 160 | fabric isolation | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 1068 | collective protocol | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1069 | nic hca selection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |

### *HPC & Distributed Compute (19 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `batch` (batch and queued jobs), reached through the job queue (Kubernetes Jobs, Slurm, Spark). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 193 | job slots | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 194 | mpi ranks | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 20 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 195 | rank mapping | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | fewer than native runs; hands back what it no longer needs at once | | |
| 196 | node allocation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 197 | job walltime | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 198 | job priority | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 199 | job preemption | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 200 | checkpoint restart | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 80% full; reads the last 20 decisions |
| 201 | parallel io budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 202 | collective budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 203 | accelerator share | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 20 decisions |
| 204 | cpu gpu ratio | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 205 | memory per rank | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 206 | scratch budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 207 | scheduler fair share | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 208 | backfill policy | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 1070 | node power saving | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |
| 1071 | cpu frequency per job | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |
| 1072 | partition limits | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

### *Kubernetes Dynamic Device Allocation (8 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `gpu` (GPU serving with a response-time target), reached through nvidia-smi (clock ceiling and power limit) or the serving autoscaler. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDIT-0001 | dra device class selection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 85% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDIT-0002 | dra claim capacity | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| AUDIT-0003 | dra claim sharing | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 20 decisions |
| AUDIT-0004 | dra device taint | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| AUDIT-0005 | dra device eviction | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| AUDIT-0006 | dra binding readiness | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 85% full; reads the last 20 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| AUDIT-0 | dra binding failure | capacity | starts the machines the forecast needs before the rush lands and | Compute; stack; | machines started at 70% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 007 | response | | clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | tower | |
| AUDIT-0008 | dra device configuration | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

### *OpenShift & Machine API (9 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 85 | machine remediation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 88 | machine health gate | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 89 | mcp pause | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 90 | mcp max unavailable | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 91 | node config rollout | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 92 | operator reconcile budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 93 | cluster version pacing | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 94 | infra machine admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 95 | machine failure domain | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *Quantum Computing Control Simulation (16 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `qpu` (a quantum or specialised accelerator queue), reached through the accelerator's job queue. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 593 | qubit mapping | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 594 | circuit admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 595 | shot allocation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 596 | circuit scheduling | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 597 | gate scheduling | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 598 | pulse amplitude | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 599 | pulse duration | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 600 | pulse phase | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 601 | pulse frequency | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |
| 602 | coupling control | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 603 | reset scheduling | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 604 | measurement scheduling | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 605 | dynamical decoupling | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 606 | noise aware routing | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 607 | error mitigation budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 608 | quantum queue priority | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

### *Work Admission & Demand Shaping (19 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1 | api concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 2 | queue concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 3 | queue backpressure | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 4 | job admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 5 | batch admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 6 | inference admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 7 | load shed | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 8 | priority gate | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 9 | tenant admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 10 | burst limit | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 11 | deadline admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 12 | work budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 13 | request queue limit | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 14 | retry admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 15 | background work gate | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 16 | maintenance work gate | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 1074 | request concurrency limit | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 1123 | adaptive concurrency limit | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 1124 | priority request queue under limit | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |

### *Linux Kernel & OS Control (20 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `cpu_host` (a CPU host whose clock and power can be set), reached through Linux cpufreq and RAPL. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5001 | cgroup cpu weight | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| V5-5002 | cgroup cpu quota | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | fewer than native runs; hands back what it no longer needs at once | | |
| V5-5003 | cgroup memory high | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 10 decisions |
| V5-5004 | cfs scheduler latency | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5005 | cpufreq dvfs governor | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |
| V5-5006 | transparent hugepages | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5007 | io scheduler depth | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| V5-5008 | numa auto balancing | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5009 | dirty ratio writeback | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5010 | vm swappiness | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5011 | tcp buffer autotune | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 60 decisions |
| V5-5012 | block nr requests | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| V5-5013 | ksm memory dedup | capacity | at its limit on native (the full actuator): read and left native | Compute; stack; tower | global |
| V5-5014 | net rx coalesce | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| V5-5015 | cpu cstate governor | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |
| V5-5016 | zswap pool fraction | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5017 | page cache reclaim | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5018 | irqbalance affinity | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 60 decisions |
| V5-5019 | net qdisc codel target | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5020 | hugepage pool reserve | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 20 decisions |

### *Serverless & Functions (FaaS) (9 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5021 | fn concurrency limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5022 | fn provisioned concurrency | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |
| V5-5023 | fn scale to zero idle | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5024 | fn memory size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| V5-5025 | fn prewarm pool | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |
| V5-5026 | fn reserved concurrency | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |
| V5-5027 | fn burst admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5028 | fn snapstart restore | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| V5-5029 | fn timeout budget | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |

### *Virtualization & Hypervisor (9 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `cpu_host` (a CPU host whose clock and power can be set), reached through Linux cpufreq and RAPL. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5030 | vcpu allocation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |
| V5-5031 | memory ballooning | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5032 | vm overcommit density | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| V5-5033 | live migration pacing | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5034 | numa vcpu pinning | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5035 | sriov vf allocation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| V5-5036 | vgpu partition | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| V5-5037 | vm memory dedup ksm | capacity | at its limit on native (the full actuator): read and left native | Compute; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| V5-5038 | hypervisor cpu pinning | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | fewer than native runs; hands back what it no longer needs at once | | |

### *Gut: Data-plane Intake & Waste (8 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `process` (an industrial process loop), reached through the PLC or DCS (OPC UA, EtherNet/IP, PROFINET). Omni-Compass reads how far the process sits from its band. Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5045 | ingress intake pacing | capacity | at its limit on native (the full actuator): read and left native | Compute; stack; tower | global |
| V5-5046 | dead letter retry cap | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5047 | gc compaction pacing | capacity | at its limit on native (the full actuator): read and left native | Compute; stack; tower | global |
| V5-5048 | ttl eviction waste | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5049 | backpressure threshold | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5050 | queue drain rate | capacity | at its limit on native (the full actuator): read and left native | Compute; stack; tower | global |
| V5-5051 | retry budget window | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5052 | compaction debt drain | capacity | at its limit on native (the full actuator): read and left native | Compute; stack; tower | global |

### *Skin: Perimeter & Admission (5 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5053 | rate limit token bucket | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5054 | connection admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5055 | waf inspection sampling | capacity | at its limit on native (the full actuator): read and left native | Compute; stack; tower | global |
| V5-5056 | tls session cache | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| V5-5057 | ddos shield admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |

### *Media Streaming & CDN (14 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `process` (an industrial process loop), reached through the PLC or DCS (OPC UA, EtherNet/IP, PROFINET). Omni-Compass reads how far the process sits from its band. Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5072 | abr bitrate ladder cap | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5073 | player buffer target | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| V5-5074 | transcoder worker pool | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | fewer than native runs; hands back what it no longer needs at once | | |
| V5-5075 | live transcode gpu clocks | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |
| V5-5076 | vod transcode batch power | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |
| V5-5077 | packager workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 85% full; reads the last 60 decisions |
| V5-5078 | origin shield cache size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| V5-5079 | edge cache ttl | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5080 | segment duration target | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5081 | cdn request coalescing | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| V5-5082 | drm license server pool | capacity | starts the machines the forecast needs before the rush lands and | Compute; stack; tower | machines started at 85% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| V5-5083 | thumbnail sprite batch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 80% full; reads the last 10 decisions |
| V5-5084 | media ingest admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5085 | encoder lookahead frames | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |

### *Game Servers & Real-Time Interactive (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5086 | agones fleet buffer size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5087 | agones fleet min replicas | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 60 decisions |
| V5-5088 | agones fleet max replicas | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| V5-5089 | game server tick rate | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5090 | players per server cap | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5091 | matchmaking admission rate | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5092 | matchmaking wait window | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5093 | relay server pool | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5094 | voice chat mixer pool | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |
| V5-5095 | game node pool autoscaler | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| V5-5096 | replay render batch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| V5-5097 | cloud gaming gpu clocks | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |

### *Blockchain & Ledger Nodes (10 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `storage` (block, file or object storage), reached through the storage system's QoS and tiering API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5098 | node db cache size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| V5-5099 | mempool max size | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5100 | max peer connections | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| V5-5101 | script validation threads | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |
| V5-5102 | block relay admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5103 | rpc work queue depth | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5104 | utxo cache flush interval | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| V5-5105 | indexer batch workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| V5-5106 | archive node storage tier | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| V5-5107 | validator host power profile | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |

### *CI/CD & Build Farms (10 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `batch` (batch and queued jobs), reached through the job queue (Kubernetes Jobs, Slurm, Spark). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5108 | runner scale set min runners | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| V5-5109 | runner scale set max runners | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line | Compute; stack; tower | machines started at 70% full; reads the last 60 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| V5-5110 | build concurrency limit | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5111 | build cache size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| V5-5112 | test shard parallelism | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| V5-5113 | job queue admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5114 | artifact retention ttl | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5115 | build node cpu frequency | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| V5-5116 | remote execution workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| V5-5117 | warm runner pool | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 40 decisions |

## Realm: Physics / Robotics / Autonomous (137 muscles of its own, 404 in its organism with the spine)

### *Automotive EV & Mobile Powertrain (14 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `ev_traction` (an electric vehicle traction motor), reached through the vehicle's motor controller (CAN; modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1083 | state of charge limits | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 1084 | smart charging schedule | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| AUDI T-0045 | traction torque limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| AUDI T-0046 | regen braking level | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| AUDI T-0047 | battery charge limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDI T-0048 | battery discharge limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| AUDI T-0049 | battery thermal target | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| AUDI T-0050 | motor thermal limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| AUDI T-0051 | vehicle speed envelope | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| AUDI T-0052 | energy recovery target | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| AUDI T-0053 | auxiliary power budget | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| AUDI T-0054 | fast charge current | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| AUDI T-0055 | fast charge voltage | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| AUDI T-0056 | vehicle safe state | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |

### *Aviation & Autonomous Flight (28 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `flight_axis` (a flight-control axis), reached through the flight controller (modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 481 | throttle envelope | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 482 | attitude target | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 483 | attitude rate target | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 484 | velocity target | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 485 | altitude target | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 486 | waypoint authority | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 487 | flight hold | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 488 | return to home | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 489 | land action | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 490 | mission admission | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 491 | geofence response | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 492 | failsafe selection | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 493 | battery reserve threshold | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 494 | actuator saturation envelope | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 495 | flight mode transition | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 496 | flight termination safe state | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 1079 | attitude rate gains | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 1080 | horizontal speed limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 1081 | vertical speed limits | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 1082 | acceleration jerk limits | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5143 | px4 mpc xy vel max | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| V5-5144 | px4 mpc z vel max up | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| V5-5145 | px4 mpc acc hor | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| V5-5146 | px4 mc rollrate max | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| V5-5147 | ardupilot wpnav speed | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| V5-5148 | ardupilot atc accel r max | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| V5-5149 | drone battery failsafe reserve | setpoint | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Physics; stack; tower | 1.5 wobbles of margin on its line |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| V5-5150 | swarm formation spacing | capacity | its own peak speed while tasks wait, back to its everyday speed | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | | |

### *Robotics Fleet & Warehouse Automation (17 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `robot_fleet` (a fleet of robots or vehicles taking tasks), reached through the fleet manager's dispatch API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 449 | robot dispatch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |
| 450 | task assignment | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |
| 451 | traffic reservation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 452 | robot route | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 453 | charging dispatch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |
| 454 | battery reserve | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |
| 455 | elevator request | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |
| 456 | door request | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |
| 457 | conveyor speed | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 458 | agv speed | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |
| 459 | warehouse zone admission | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 460 | robot quarantine | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 461 | fleet failover | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |
| 462 | human safe stop | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |
| 463 | fleet concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| V5-5151 | amr fleet charge threshold | setpoint | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Physics; stack; tower | 1.5 wobbles of margin on its line |
| V5-5152 | amr fleet active robots | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *Robotics Motion Control (26 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `robot_joint` (a robot joint servo), reached through the robot controller (ROS 2, EtherCAT, the drive's fieldbus). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 433 | joint position | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 434 | joint velocity | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 435 | joint acceleration | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 436 | joint effort | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 437 | cartesian velocity | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 438 | trajectory speed | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 439 | trajectory acceleration | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 440 | jerk limit | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 441 | collision margin | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 442 | force limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 443 | gripper force | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 444 | locomotion speed | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | when its tracking strains (alone; inside a body, its everyday speed) | | |
| 445 | steering angle | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 446 | braking force | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 447 | balance correction | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 1076 | trajectory tolerances | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 1077 | velocity acceleration scaling | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 1078 | controller update rate | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | when its tracking strains (alone; inside a body, its everyday speed) | | |
| V5-5135 | ros2 executor threads | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | machines started at 70% full; reads the last 40 decisions |
| V5-5136 | ros2 qos history depth | setpoint | left native: native already holds it inside its band (nothing to win) | Physics; stack; tower | global |
| V5-5137 | ros2 control loop rate | setpoint | left native: native already holds it inside its band (nothing to win) | Physics; stack; tower | global |
| V5-5138 | humanoid balance margin | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| V5-5139 | humanoid gait speed | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| V5-5140 | humanoid joint torque limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| V5-5141 | humanoid step height | capacity | its own peak speed while tasks wait, back to its everyday | Physics; stack; | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | tower | |
| V5-5142 | manipulator payload speed scaling | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |

### *Spacecraft & Flight Software (14 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `reaction_wheel` (a spacecraft reaction wheel), reached through the attitude controller (modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 497 | space command admission | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 499 | flight task schedule | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 500 | space mode transition | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 501 | payload duty cycle | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 502 | communication allocation | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |
| 503 | space power budget | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 504 | space thermal command | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 505 | attitude command envelope | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 506 | reaction wheel allocation | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |
| 507 | rcs authority | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 508 | safe mode transition | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 509 | watchdog recovery | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 510 | instrument activation | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |
| 511 | fault isolation | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |

### *Rail Traction & Train Control (14 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `rail_traction` (a train's traction and auxiliary systems), reached through the train control and management system and ATO (modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2054 | traction effort limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2055 | regenerative braking share | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2056 | coasting speed target | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |
| 2057 | dwell time schedule | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2058 | hvac duty cycle | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2059 | train auxiliary power budget | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2060 | platform approach speed | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |
| 2061 | acceleration rate setpoint | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |
| 2062 | wheel slip protection mode | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2063 | traction motor thermal derate | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2064 | catenary voltage limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2065 | timetable recovery margin | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2066 | headway target | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |
| 2067 | door release hold | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |

### *Marine Propulsion & Vessel Automation (12 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `marine_propulsion` (a ship's propulsion shaft and power plant), reached through the vessel automation and power management system (modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2068 | shaft speed target | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.
www.omni-compass.com

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2069 | propeller pitch limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2070 | shaft power limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2071 | bow thruster duty | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2072 | ballast pump mode | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2073 | engine load sharing setpoint | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 2074 | slow steaming speed target | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 2075 | auxiliary engine staging | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 2076 | shore power mode | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2077 | hotel load budget | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2078 | trim target | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 2079 | rudder rate limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *Elevators & Vertical Transport (12 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `elevator_hoist` (an elevator or escalator drive and its group control), reached through the lift controller and destination dispatch (modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2146 | hoist speed target | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |
| 2147 | acceleration limit | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |
| 2148 | regen drive mode | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2149 | standby power mode | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2150 | destination dispatch schedule | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2151 | car parking mode | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2152 | door dwell hold | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2153 | escalator speed target | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |
| 2154 | group capacity mode | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2155 | motor thermal derate | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2156 | brake test schedule | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2157 | fire recall mode | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

# Realm: Energy / Facility / Industrial (213 muscles of its own, 480 in its organism with the spine)

## *Building & Critical Environment HVAC (15 muscles)*

Each is a data hall or building zone cooled by chillers, air handlers or fans; its class is `building` (a building zone), reached through the building management system. Omni-Compass reads its temperature against its limit. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1094 | setpoint reset trim and respond | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |
| 1095 | duct static pressure setpoint | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |
| 1096 | optimal start stop | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | Energy; stack; tower | global |
| AUDIT-0081 | zone temperature target | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |
| AUDIT-0082 | zone airflow | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | Energy; stack; tower | global |
| AUDIT-0083 | ahu fan speed | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | Energy; stack; tower | global |
| AUDIT-0084 | damper position | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | Energy; stack; tower | global |
| AUDIT-0085 | economizer position | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | Energy; stack; tower | global |
| AUDIT-0086 | boiler setpoint | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDIT-0087 | heat pump mode | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDIT-0088 | humidity target | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |
| AUDIT-0089 | occupancy ventilation | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDIT-0090 | building demand limit | power | at its limit on native (the site envelope at full): read and left native | Energy; stack; tower | global |
| AUDIT-0091 | thermal storage dispatch | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | Energy; stack; tower | global |
| AUDIT-0092 | hvac emergency mode | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

### *Energy Storage & Microgrid (19 muscles)*

Each is a battery, UPS or microgrid store with a load and a grid connection; its class is `microgrid` (a microgrid battery and loads), reached through the inverter or energy management system (IEEE 2030.5, SunSpec, OpenADR). Omni-Compass reads its draw on the grid connection and its reserve. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 513 | battery charge power | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | 0 wobbles of margin on its line; reads its line 10 decisions before it holds; keeps 0% of its line clear |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 514 | battery discharge power | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear |
| 515 | battery soc reserve | setpoint | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 0.75 wobbles of margin on its line |
| 516 | grid import limit | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | global |
| 517 | grid export limit | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 5% of its line clear |
| 518 | pv curtailment | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear |
| 519 | ev charge power | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | global |
| 520 | heat pump power | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear |
| 521 | electrolyzer power | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | 0 wobbles of margin on its line; reads its line 5 decisions before it holds; keeps 0% of its line clear |
| 522 | microgrid demand limit | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 523 | peak shaving | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 0 wobbles of margin on its line |
| 524 | time of use schedule | setpoint | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 1 wobbles of margin on its line |
| 525 | energy load shed | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 526 | flex load admission | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 527 | storage dispatch | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 0 wobbles of margin on its line |
| 528 | microgrid emergency reserve | setpoint | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 1 wobbles of margin on its line |
| 1085 | volt var | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 0 wobbles of margin on its line |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 1086 | volt watt | capacity | holds the charge for the line and clips only the draw that | Energy; stack; | 0 wobbles of margin on its line |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | would cross it; a UPS lends only the charge above its outage floor | tower | |
| 1087 | constant power factor reactive power | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | global |

### *Facility & Grid Optimization (15 muscles)*

Each is a battery, UPS or microgrid store with a load and a grid connection; its class is `facility` (a facility battery behind the meter), reached through the energy management system. Omni-Compass reads its draw on the grid connection and its reserve. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 561 | facility power budget | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear |
| 562 | utility demand limit | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear |
| 563 | demand response | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 564 | electricity price gate | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 565 | carbon intensity gate | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 566 | renewable dispatch | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 0 wobbles of margin on its line |
| 567 | generator start stop | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 0 wobbles of margin on its line |
| 568 | site battery dispatch | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 0 wobbles of margin on its line |
| 569 | pue target | setpoint | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 1 wobbles of margin on its line |
| 570 | cooling power budget | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear |
| 571 | it power budget | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear |
| 572 | rack power allocation | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | global |
| 573 | facility peak guard | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 0 wobbles of margin on its line |
| 574 | grid frequency response | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 575 | facility islanding | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

### *Grid Transmission & Distribution (12 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `feeder_voltage` (a distribution feeder's voltage), reached through the distribution management system (IEC 61850, DNP3; modelled only). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDIT-0033 | capacitor bank switch | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDIT-0034 | voltage regulator tap | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| AUDIT-0035 | transformer tap | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| AUDIT-0036 | inverter real power | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDIT-0037 | inverter reactive power | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDIT-0038 | feeder voltage target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| AUDIT-0039 | feeder load transfer | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDIT-0040 | distribution storage dispatch | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDIT-0041 | demand response dispatch | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDIT-0042 | frequency droop setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDI T-0043 | grid protection mode | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDI T-0044 | grid restoration sequence | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

### *Industrial PLC & Process Automation (18 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `process` (an industrial process loop), reached through the PLC or DCS (OPC UA, EtherNet/IP, PROFINET). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 465 | plc cycle authority | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 466 | machine cell admission | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 467 | valve position | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 468 | pump flow | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 469 | compressor speed | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 470 | heater power | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 471 | furnace setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 472 | pressure setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 473 | temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 474 | mass flow setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 475 | tank level target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 476 | conveyor rate | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 477 | feed rate | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 478 | purge vent action | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 1088 | controller mode | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 1089 | alarm limits | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 1090 | safety interlock trip | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 1091 | opc ua writes | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |

### *Semiconductor Fab & Precision Manufacturing (13 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `chamber` (a controlled chamber (clean room, kiln, reactor)), reached through the PLC or DCS. Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1092 | run to run control | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 1093 | idle sleep mode | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDIT-0057 | tool job dispatch | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDIT-0058 | wafer route | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDI T-0059 | chamber recipe selection | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDI T-0060 | chamber temperature | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| AUDI T-0061 | chamber pressure | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| AUDI T-0062 | gas flow | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDI T-0063 | rf power | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDI T-0064 | vacuum pump speed | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDI T-0065 | robot transfer rate | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDI T-0066 | lot priority | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDI T-0068 | tool quarantine | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

### *Water Wastewater & Pumping (12 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `water` (a water or pumping process), reached through the SCADA system (Modbus, DNP3). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDI T-0069 | pump speed water | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDI T-0070 | valve position water | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| AUDI T-0071 | reservoir level target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDI T-0072 | line pressure target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| AUDI T-0073 | flow target water | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| AUDI T-0074 | aeration rate | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDI T-0075 | chemical dose rate | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDI T-0076 | filtration backwash | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDI T-0077 | lift station dispatch | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDI T-0078 | leak isolation | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDI T-0079 | water demand shed | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDI T-0080 | water emergency shutdown | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

### *Healthcare Critical Environments (13 muscles)*

Each is a data hall or building zone cooled by chillers, air handlers or fans; its class is `hospital` (a hospital's critical rooms (operating rooms, isolation, pharmacy, imaging suites)), reached through the hospital's building management system (BACnet). Omni-Compass reads its temperature against its limit. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2001 | operating room air change setpoint | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |
| 2002 | isolation room pressure target | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2003 | patient room temperature setpoint | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |
| 2004 | surgical suite humidity target | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |
| 2005 | ahu supply air temperature | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |
| 2006 | pharmacy cold room setpoint | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |
| 2007 | sterile storage humidity setpoint | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |
| 2008 | imaging suite cooling capacity | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | Energy; stack; tower | global |
| 2009 | chiller plant staging | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | Energy; stack; tower | global |
| 2010 | exhaust fan capacity | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | Energy; stack; tower | global |
| 2011 | ward night setback mode | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2012 | medical gas plant demand limit | power | at its limit on native (the site envelope at full): read and left native | Energy; stack; tower | global |
| 2013 | emergency power load shed | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

### *Agriculture & Irrigation (14 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `irrigation` (pumps, pressure and climate on a farm or in a greenhouse), reached through the pump station controller, the pivot panel or the climate computer (Modbus, the vendor cloud). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2026 | irrigation pump speed | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2027 | mainline pressure setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2028 | soil moisture target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2029 | pivot speed setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2030 | fertigation dose | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2031 | greenhouse temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2032 | greenhouse co2 target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2033 | vent position setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2034 | grain dryer temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2035 | cold storage temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2036 | barn ventilation rate | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2037 | milking vacuum level setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2038 | well drawdown level target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2039 | drip zone dispatch | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *Oil & Gas Pipelines (14 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `pipeline` (a pipeline segment with its compressors, pumps and valves), reached through pipeline SCADA (DNP3, Modbus, OPC UA). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2040 | compressor discharge pressure setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2041 | pump station suction pressure target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2042 | line pack target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2043 | pipeline flow setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2044 | compressor unit staging | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2045 | vfd pump speed | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2046 | terminal tank level target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2047 | leak detection shutdown | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2048 | batch interface dispatch | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2049 | heater outlet temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2050 | drag reducing agent dose | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2051 | valve position target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2052 | cathodic protection voltage | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2053 | pressure protection priority | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

### *Mining & Mineral Processing (14 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `mill` (a grinding, flotation or materials-handling circuit), reached through the plant DCS or PLC (OPC UA). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2092 | sag mill load setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2093 | mill speed target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2094 | crusher gap setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2095 | flotation aeration rate | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2096 | cyclone feed pressure target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2097 | thickener underflow density target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2098 | reagent dose | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2099 | conveyor speed setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2100 | dewatering pump level target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2101 | ventilation on demand airflow | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2102 | stockpile level target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2103 | slurry pump speed | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2104 | tailings discharge shutdown | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2105 | ore blend dispatch | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

### *District Heating & Cooling (13 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `district_heat` (a district heating or cooling network), reached through the network control system and substation controllers (Modbus, M-Bus). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2106 | supply temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2107 | return temperature target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2108 | differential pressure setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2109 | network pump speed | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2110 | heat pump staging | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2111 | chp dispatch | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2112 | thermal storage level target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2113 | peak boiler heater output | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2114 | substation flow limit | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2115 | peak demand shed | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2116 | outdoor reset setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2117 | cooling network supply temperature | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2118 | cooling storage level target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |

### *Power Generation & Turbine Control (14 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `turbine` (a generating unit's governor, boiler or excitation loop), reached through the plant DCS and governor (OPC UA, IEC 61850; modelled only). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2119 | turbine speed droop setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2120 | unit load setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2121 | boiler steam pressure setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2122 | feedwater level target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2123 | agc participation limit | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2124 | excitation voltage setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2125 | hydro gate position target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2126 | combustion air ratio setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2127 | cooling water flow setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2128 | inlet guide vane position | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2129 | reserve dispatch priority | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2130 | turbine ramp rate limit | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2131 | emissions shutdown | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2132 | nuclear rod position target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |

### *Renewable Generation & Inverter Control (13 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `inverter` (a plant of grid-support inverters or wind turbines), reached through the plant controller and inverter settings (SunSpec, IEC 61850, IEEE 1547). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2133 | inverter volt var setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2134 | inverter volt watt setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2135 | active power curtailment | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2136 | plant reactive power target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2137 | wind turbine yaw offset | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2138 | pitch angle limit | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2139 | rotor speed setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2140 | inverter frequency droop setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2141 | ramp rate limit | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2142 | tracker stow mode | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2143 | string mppt voltage setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2144 | noise mode schedule | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2145 | ice detection shutdown | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

### *Pharmaceutical & Food Manufacturing (14 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `batch_reactor` (a GMP batch reactor, fermenter or food process), reached through the batch control system (ISA-88, OPC UA). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2158 | reactor temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2159 | agitator speed setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2160 | fermenter dissolved oxygen target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2161 | ph setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2162 | pasteurizer holding temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2163 | freezer tunnel temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2164 | cip cycle dispatch | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2165 | cleanroom pressure cascade setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2166 | lyophilizer shelf temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2167 | chromatography flow setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2168 | steam sterilizer cycle priority | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2169 | oven zone temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2170 | refrigerant compressor authority | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2171 | batch hold quarantine | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

# Realm: Distribution / Specialized (217 muscles of its own, 484 in its organism with the spine)

## *Cache & Memory Services (16 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*
**www.omni-compass.com**

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 305 | cache size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 75% full; reads the last 10 decisions |
| 306 | cache ttl | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 307 | cache eviction policy | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 308 | cache replicas | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 309 | cache sharding | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 310 | cache prefetch | capacity | starts the machines the forecast needs before the rush lands and | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 311 | cache writeback rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 312 | cache admission | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 313 | hot key isolation | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 314 | cache connection limit | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 315 | cache memory limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 316 | cache compression | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 317 | cache warmup | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 318 | cache failover | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 319 | cache flush rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1106 | memory size threads | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |

### *Commerce & Payment Systems (15 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `commerce` (a customer-facing service with bursts), reached through the Kubernetes API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 401 | payment admission | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 402 | payment concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 403 | payment retry | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 404 | payment timeout | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 405 | fraud review gate | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 406 | authorization route | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 407 | processor route weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 408 | transaction queue limit | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 409 | idempotency window | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 410 | order reservation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 411 | inventory hold | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 412 | checkout load shed | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 413 | refund queue rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 414 | settlement batch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 415 | payment failover | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 60 decisions |

### *Data Analytics & ETL (15 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `batch` (batch and queued jobs), reached through the job queue (Kubernetes Jobs, Slurm, Spark). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 338 | executor size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 339 | dynamic allocation min | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 340 | dynamic allocation max | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | fewer than native runs; hands back what it no longer needs at once | | |
| 341 | shuffle partitions | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 342 | shuffle bandwidth | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 343 | etl concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 344 | stage parallelism | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 345 | query slots | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 85% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 346 | spill threshold | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 347 | cache fraction | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 80% full; reads the last 20 decisions |
| 348 | batch interval | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 80% full; reads the last 20 decisions |
| 349 | stream back pressure | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 350 | data locality wait | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 80% full; reads the last 20 decisions |
| 351 | speculation policy | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 75% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 352 | analytics admission | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

### *Database & Transactions (19 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `database` (a database cluster), reached through the database operator (replicas, connection pools). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 273 | db replicas | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 275 | db memory | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 276 | db cache | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 90% full; reads the last 40 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 277 | query concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 278 | read route | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 279 | shard placement | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 75% full; reads the last 40 decisions |
| 280 | db failover | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 281 | replication lag gate | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 282 | db write throttle | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 283 | db pool resize | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line | Distribution; stack; tower | machines started at 75% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 284 | transaction concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 285 | lock timeout | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 286 | checkpoint rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 287 | vacuum compaction rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 1099 | parallel workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 60 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 1100 | background writer | capacity | starts the machines the forecast needs before the rush lands and | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 1101 | synchronous replication | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 1102 | buffer pool | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 75% full; reads the last 40 decisions |
| 1103 | io capacity | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |

### *Messaging & Streaming (22 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 289 | partition count | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 290 | partition placement | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 291 | producer quota | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 292 | consumer quota | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 293 | broker io quota | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 294 | message retention | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 295 | queue depth limit | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 296 | consumer concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 297 | producer batch size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 298 | fetch batch size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 299 | rebalance rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 300 | replication factor | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 75% full; reads the last 10 decisions |
| 301 | retry backoff | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 302 | dead letter divert | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 303 | stream priority | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 304 | broker failover | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 1104 | prefetch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 1105 | memory disk alarm | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| V5-5124 | mqtt broker max | capacity | starts the machines the forecast needs before the rush | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | connections | | lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| V5-5125 | mqtt max inflight messages | setpoint | left native: native already holds it inside its band (nothing to win) | Distribution; stack; tower | global |
| V5-5126 | mqtt session expiry | setpoint | left native: native already holds it inside its band (nothing to win) | Distribution; stack; tower | global |
| V5-5127 | broker cluster nodes | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |

### *Runtime & Application (21 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 353 | worker count | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 75% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 354 | thread pool | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 355 | jvm heap | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 356 | gc budget | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 357 | connection pool runtime | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 358 | application cache size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 359 | runtime memory | capacity | starts the machines the forecast needs before the rush lands | Distribution; stack; tower | machines started at 70% full; reads the last 60 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 360 | async concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 361 | event loop workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 80% full; reads the last 20 decisions |
| 362 | process count | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 363 | request timeout | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 364 | background workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 365 | runtime cpu budget | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 366 | runtime io budget | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 367 | runtime restart | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 1108 | go runtime | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 1109 | keepalive connection reuse | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 60 decisions |
| V5-5131 | dns resolver cache size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| V5-5132 | dns negative cache ttl | setpoint | left native: native already holds it inside its band (nothing to win) | Distribution; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5133 | mail queue concurrency | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| V5-5134 | mail retry interval | setpoint | left native: native already holds it inside its band (nothing to win) | Distribution; stack; tower | global |

### *Search, Indexing & Vector DB (16 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 321 | index workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 75% full; reads the last 20 decisions |
| 322 | index refresh rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 323 | segment merge rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 324 | search concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 325 | search timeout | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 326 | shard count | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 327 | shard replication | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 328 | shard rebalance | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 329 | vector search k | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 330 | vector batch size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 331 | embedding workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 332 | index memory budget | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 333 | query route | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 334 | hot shard isolation | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 335 | search admission | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 1107 | circuit breakers | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

### *Service Mesh & API Reliability (16 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 241 | circuit breaker | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 242 | retry budget | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 243 | service timeout | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 244 | service concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 245 | connection pool | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 246 | traffic divert | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 247 | traffic mirror | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 248 | canary weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 249 | outlier ejection | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 250 | health threshold | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 251 | dns traffic weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 252 | session affinity | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 253 | request hedging | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 254 | fault injection gate | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 255 | service failover | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1097 | load balancing policy | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *Telecom RAN & Edge Radio (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `ran` (a radio access network cell or site), reached through the RAN controller (O-RAN interfaces). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDIT-0009 | ran connection admission | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| AUDIT-0010 | ran ue handover | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| AUDIT-0011 | ran cell traffic steering | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| AUDIT-0012 | ran slice resource budget | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| AUDIT-0013 | ran prb allocation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| AUDIT-0014 | ran scheduler weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine | Distribution; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | start, never fewer than native runs; hands back what it no longer needs at once | | |
| AUDIT-0015 | ran tx power | power | at its limit on native (the full clock): read and left native | Distribution; stack; tower | global |
| AUDIT-0016 | ran antenna tilt | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| AUDIT-0017 | ran carrier enable | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| AUDIT-0018 | ran cell sleep | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| AUDIT-0019 | ran du cu placement | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| AUDIT-0020 | ran fronthaul budget | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *Workflow, Logistics & Fulfillment (15 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `workflow` (a workflow or pipeline engine), reached through the workflow engine's concurrency settings. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 417 | workflow admission | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 418 | workflow worker rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 419 | task queue rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 420 | workflow retry | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 421 | workflow backoff | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 422 | workflow timeout | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 423 | inventory allocation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 424 | fulfillment route | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 425 | warehouse queue | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 426 | carrier selection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 427 | dispatch priority | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 428 | shipment batch | capacity | starts the machines the forecast needs before the rush lands and | Distribution; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 429 | route replan | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 430 | sla escalation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 431 | compensation action | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |

### *Medical Imaging & Clinical Systems (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `clinical` (clinical systems: imaging archives, records, interface engines, monitoring gateways), reached through the application's own scaling and admission settings (Kubernetes, the vendor console). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2014 | pacs archive tier target | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 2015 | dicom router concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2016 | ehr application replicas | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 2017 | hl7 interface queue limit | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2018 | fhir api rate limit | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2019 | reconstruction gpu workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2020 | modality worklist timeout | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2021 | patient monitoring gateway capacity | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2022 | lab analyzer batch window | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2023 | telehealth session admission | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2024 | clinical backup window | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2025 | imaging prefetch priority | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

### *Ports & Maritime Logistics (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `port` (a container terminal's cranes and vehicles taking moves), reached through the terminal operating system and fleet controllers. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2080 | quay crane allocation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.
www.omni-compass.com

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2081 | yard crane fleet size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 2082 | berth window admission | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2083 | truck gate rate limit | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2084 | horizontal transport fleet size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 2085 | reefer plug power budget | power | at its limit on native (the full clock): read and left native | Distribution; stack; tower | global |
| 2086 | shore power connection capacity | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 2087 | rail mounted gantry dispatch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 2088 | container dwell priority | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2089 | vessel arrival pacing | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2090 | stacking height target | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 2091 | equipment charging window | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

### *Content Delivery & Edge Cache (6 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `process` (an industrial process loop), reached through the PLC or DCS (OPC UA, EtherNet/IP, PROFINET). Omni-Compass reads how far the process sits from its band. Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5039 | cdn cache ttl | setpoint | left native: native already holds it inside its band (nothing to win) | Distribution; stack; tower | global |
| V5-5040 | edge cache size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| V5-5041 | origin shield concurrency | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| V5-5042 | prefetch aggressiveness | setpoint | left native: native already holds it inside its band (nothing to win) | Distribution; stack; tower | global |
| V5-5043 | tiered cache promotion | capacity | at its limit on native (the full actuator): read and left native | Distribution; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5044 | edge pop admission | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

### *Financial Markets & Trading Systems (14 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5058 | matching engine threads | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| V5-5059 | order gateway sessions | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| V5-5060 | order admission rate limit | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| V5-5061 | pre trade risk check workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5062 | market data fanout threads | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| V5-5063 | market data conflation interval | setpoint | left native: native already holds it inside its band (nothing to win) | Distribution; stack; tower | global |
| V5-5064 | fix session heartbeat interval | setpoint | left native: native already holds it inside its band (nothing to win) | Distribution; stack; tower | global |
| V5-5065 | kernel bypass poll cores | power | at its limit on native (the full clock): read and left native | Distribution; stack; tower | global |
| V5-5066 | tick store write batch | setpoint | left native: native already holds it inside its band (nothing to win) | Distribution; stack; tower | global |
| V5-5067 | post trade settlement workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 80% full; reads the last 20 decisions |
| V5-5068 | end of day risk batch power | power | at its limit on native (the full clock): read and left native | Distribution; stack; tower | global |
| V5-5069 | colocation rack cooling setpoint | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Distribution; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*
**www.omni-compass.com**

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5070 | backtest grid nodes | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| V5-5071 | surveillance alert queue workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |

### *Reliability, Security & Recovery (6 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `network` (network routing and switching), reached through the network controller (SDN, routing API). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5118 | ids capture threads | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| V5-5119 | ids ring buffer size | setpoint | left native: native already holds it inside its band (nothing to win) | Distribution; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5120 | ids rule group admission | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| V5-5121 | waf inspection workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| V5-5122 | siem ingest pipeline workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 80% full; reads the last 10 decisions |
| V5-5123 | endpoint scan scheduling | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix F. The License

The license under which this book and the software are made available, as it stands in the file `LICENSE` at the root of the repository.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
Omni-Compass Evaluation License, version 1.0 (revised 11 October 2026)
SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0

Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
All patents, copyrights and trademarks filed in the USA. www.omni-comp
    ↪ ass.com

THIS IS NOT AN OPEN-SOURCE LICENSE. The Software is proprietary. It is
    ↪  published so that it can be evaluated, reproduced
and audited, and for no other purpose. It is licensed, not sold. Evalu
    ↪ ation and simulation are free of charge; every
other use, and in particular every commercialization or monetization o
    ↪ f any kind, requires a signed, paid Omni-Compass
Enterprise License.

ACCEPTANCE. By downloading, copying, forking, installing, running, sca
    ↪ nning or otherwise using any part of the Software,
You accept this license. If You do not accept it, You may not use the
    ↪ Software. If You use the Software on behalf of an
organization, You represent that You have authority to bind it, and "Y
    ↪ ou" includes that organization and its
Affiliates.

1. DEFINITIONS

   1.1 "Licensor" means The Omni-Compass LLC.
   1.2 "Software" means everything Licensor publishes or makes availab
    ↪ le under this license, in any form: the source
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
    code and any object code; the Omni-Compass engine, the compass
 ↪ law, the governors, controllers, brains, shields,
    plugs and wiring; the harnesses, tools, scripts, workflows, con
 ↪ figurations, presets, catalogs, organisms and realms;
    the documentation, the manual, the book, the specifications, th
 ↪ e preregistrations and the receipts; the benchmarks,
    data, tables, reports and Results published by Licensor; and ev
 ↪ ery update, version and copy of any of them.
1.3 "Mechanism" means the methods, processes, control laws, algorit
 ↪ hms, architectures and systems embodied in or
    described by the Software, including everything described in th
 ↪ e patent applications Licensor has filed.
1.4 "Evaluation" means internal use by You, free of charge, solely
 ↪ to test, study and assess the Software in
    contemplation of an Enterprise License: reading it; running its
 ↪  tests, verifiers, benchmarks and simulations; and
    running it in observe, watch, dry-run or shadow mode, or in a t
 ↪ est environment, on systems You own or control,
    where it writes to no system that serves production traffic, re
 ↪ al users, customers, patients or passengers, and to
    no power system, vehicle, machine or physical process outside a
 ↪  test bench.
1.5 "Simulation" means running the Software against modelled, recor
 ↪ ded or synthetic systems, including the organisms,
    realms, traces and simulators published with the Software.
1.6 "Production Use" means any use that is not Evaluation or Simula
 ↪ tion, including any use in which the Software, the
    Mechanism or any output of either controls, governs, tunes, siz
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
 ↪ es, schedules or otherwise influences a system that
    serves production workloads or real users, or any physical proc
 ↪ ess outside a test bench.
1.7 "Commercial Use" means any use for, or in connection with, comm
 ↪ ercial advantage, revenue, cost reduction in
    operations, or monetary or other compensation, including consul
 ↪ ting, integration, managed, outsourcing, training,
    benchmarking-for-hire and advisory services, and any use by or
 ↪ for a for-profit operation beyond Evaluation.
1.8 "Hosted Service" means making the Software, the Mechanism or an
 ↪ y of their functionality available to any third
    party over a network or as a service, including software-as-a-s
 ↪ ervice, platform-as-a-service, a managed or
    outsourced service, a cloud marketplace offering, or a feature
 ↪ of another product or service.
1.9 "Derivative" means any modification, adaptation, translation, p
 ↪ ort, extension or re-implementation of the Software
    or of the Mechanism, and any work that incorporates, is based o
 ↪ n, or is built using the Software as a reference.
1.10 "Results" means the measurements, tables, receipts, logs, repo
 ↪ rts and other outputs produced by running the
    Software.
1.11 "Enterprise License" means a written agreement for the Softwar
 ↪ e signed by Licensor and a licensee, under which
    the licensee pays the fees it sets out; it may include order fo
 ↪ rms, riders and statements of work.
1.12 "Marks" means the names "Omni-Compass" and "OmniCompass", the
 ↪ Omni-Compass compass rose with its ring of Greek
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
       letters, and every related name, logo and trade dress of Licens
    ↪ or.
   1.13 "Affiliate" means any entity that controls, is controlled by,
    ↪ or is under common control with You, where control
       means ownership of more than half of the voting interests or th
    ↪ e power to direct management.
   1.14 "You" means the individual or entity exercising rights under t
    ↪ his license, with its Affiliates where the
       Acceptance paragraph above includes them.

2. WHAT THIS LICENSE ALLOWS

   2.1 Copyright license. Subject to Your compliance with this license
    ↪ , Licensor grants You a personal, worldwide,
       royalty-free, non-exclusive, non-transferable, non-sublicensabl
    ↪ e and revocable license to download, copy, read,
       run and modify the Software solely for Evaluation and Simulatio
    ↪ n, and to reproduce and publish Results as section
       4 allows.
   2.2 Patent license. Subject to the same conditions, Licensor grants
    ↪  You a license of the same scope, under the patent
       rights Licensor holds or will hold in the filed applications an
    ↪ d any patent issuing from them, solely to the
       extent needed to carry out Evaluation and Simulation with the S
    ↪ oftware. No patent license is granted for any
       Production Use, Commercial Use or Hosted Service, for any Deriv
    ↪ ative used outside Evaluation or Simulation, or for
       any implementation of the Mechanism other than the Software its
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
    ↪ elf.
   2.3 Forks. You may keep a copy or a fork of the Software on a code-
    ↪ hosting service, publicly or privately, only for
       Evaluation and Simulation, with this license and every notice i
    ↪ ntact. A fork confers no right beyond this license.
   2.4 Reservation. Every right not expressly granted is reserved by L
    ↪ icensor. No right, title, license or interest is
       granted by implication, estoppel, exhaustion or otherwise.

3. WHAT THIS LICENSE DOES NOT ALLOW

   Unless an Enterprise License expressly permits it, You may not, and
    ↪  may not let or help anyone else:
   (a) make any Production Use or Commercial Use of the Software, the
    ↪ Mechanism or any Derivative;
   (b) commercialize or monetize the Software, the Mechanism, any Deri
    ↪ vative or any Result in any way, directly or
       indirectly, including by charging for access, services, support
    ↪ , results, savings or outcomes they produce;
   (c) offer the Software, the Mechanism or any Derivative as, or as p
    ↪ art of, a Hosted Service;
   (d) sell, resell, license, sublicense, lease, rent, lend, distribut
    ↪ e, publish as a package or binary, or otherwise
       transfer the Software or any Derivative, except forks under sec
    ↪ tion 2.3;
   (e) incorporate the Software, any part of it or any Derivative into
    ↪  any product, service, platform, appliance, image,
       firmware or offering;
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
   (f) use the Software, the Mechanism, the documentation or the Resul
    ↪ ts to design, build, train, tune, benchmark for
       sale, or improve any product or service that competes with Omni
    ↪ -Compass, or to re-implement the Mechanism;
   (g) use the Software or the Results to train, fine-tune or evaluate
    ↪  any machine-learning model intended to reproduce,
       imitate or substitute for the Mechanism;
   (h) remove, alter or obscure any notice, the SPDX identifiers, the
    ↪ patent and trademark notices, this license, the
       NOTICE or DISCLOSURES.md, or circumvent, disable or work around
    ↪  any limit, switch or safeguard in the Software,
       including the kill switch and the hand-back to native settings;
   (i) connect the Software so that it writes to a safety-critical sys
    ↪ tem, as section 10 describes;
   (j) use the Marks, except to refer to Omni-Compass accurately as TR
    ↪ ADEMARKS.md allows; or
   (k) use the Software in violation of any law, or in breach of secti
    ↪ on 11.

4. REPRODUCING AND PUBLISHING RESULTS

   You may reproduce the benchmarks published with the Software and pu
    ↪ blish Your own Results, provided that You: (a)
   name the Software's engine version and the exact commit You ran; (b
    ↪ ) report every measured row, the unfavourable ones
   included, with the evidence class the Software assigns to it; (c) m
    ↪ ake Your complete raw files and configuration
   available to Licensor on request; (d) do not state or imply any end
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ orsement, certification or affiliation by
   Licensor; and (e) carry the notice at the top of README.md. Publish
    ↪ ing Results is not Commercial Use so long as no
   compensation is received for it, and it grants no right beyond this
    ↪  license.

5. THE OMNI-COMPASS ENTERPRISE LICENSE

   5.1 Every Production Use, Commercial Use, Hosted Service, distribut
    ↪ ion, incorporation, OEM or embedded use, and every
       other commercialization or monetization of the Software, the Me
    ↪ chanism or any Derivative requires an Enterprise
       License, signed before the use begins and paid as it provides.
    ↪ Moving from Evaluation to Production Use requires
       an Enterprise License before the first write to any production
    ↪ system.
   5.2 An Enterprise License sets its own scope, fees, term, support,
    ↪ warranties, indemnities and limits
       (ENTERPRISE_LICENSE.md summarizes the standard terms; the signe
    ↪ d agreement alone is binding). For its licensee and
       its scope, a signed Enterprise License governs where it differs
    ↪  from this license.
   5.3 Enterprise licensing: The Omni-Compass LLC, main office infrast
    ↪ ructure, toll-free direct 1-888-339-9551, system route asset 1-8
    ↪ 88-DEX-YLL-1, www.omni-compass.com.

6. OWNERSHIP AND INTELLECTUAL PROPERTY
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
   6.1 Licensor and its licensors own the Software, the Mechanism, the
    ↪  Marks and all intellectual property rights in
       them, including in every Derivative to the extent it embodies t
    ↪ hem. Nothing in this license transfers ownership.
   6.2 All patents, copyrights and trademarks covering the Omni-Compas
    ↪ s engine, its mathematics and its software have
       been filed in the United States by The Omni-Compass LLC. No fil
    ↪ ing number is stated here, and no grant,
       registration or approval is claimed; the status of every filing
    ↪  is held by the office where it was filed. Every
       person who downloads, copies, forks, scans or runs the Software
    ↪  is on notice of these filings and of this
       license (PATENTS.md, TRADEMARKS.md).
   6.3 Any use outside this license, without an Enterprise License, is
    ↪  unlicensed and infringes Licensor's copyright and
       other rights, in addition to breaching this license.

7. PATENT AND RIGHTS DEFENSE

   If You or any of Your Affiliates bring or join any claim, countercl
    ↪ aim or proceeding alleging that the Software or
   the Mechanism infringes, misappropriates or violates any patent, co
    ↪ pyright or other intellectual property right, or
   challenge the validity or enforceability of any of Licensor's right
    ↪ s in them, every license granted to You under this
   license ends automatically on the date the claim or challenge is fi
    ↪ led.
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
8. FEEDBACK AND CONTRIBUTIONS

   8.1 If You give Licensor comments, suggestions, ideas, reports or o
    ↪ ther feedback about the Software, You grant
       Licensor a perpetual, irrevocable, worldwide, royalty-free righ
    ↪ t to use, disclose and exploit it for any purpose,
       without obligation of any kind.
   8.2 Contributions are accepted only under the contributor license a
    ↪ greement (.github/CLA.md).

9. CHANGES

   Licensor reserves the right to change the Software, its documentati
    ↪ on, its results and these terms at any time,
   without notice. A changed version of these terms applies to every c
    ↪ opy of the Software obtained, and to every use
   made, after it is published in the Software's repository; continued
    ↪  use after publication is acceptance of the
   changed terms. Licensor may discontinue the Software, or this licen
    ↪ se for future copies, at any time.

10. SAFETY

   The Software is not designed, tested or certified for safety-critic
    ↪ al use. Under this license it may not write to
   any system whose failure could cause death, injury, environmental d
    ↪ amage or significant property damage, including
   vehicles and aircraft, medical devices, power-grid protection and d
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
    ↪ ispatch, industrial safety systems and life-safety
   systems; such systems may be studied only in Simulation. Any live c
    ↪ onnection to a physical system requires an
   Enterprise License, a written safety case accepted by the system's
    ↪ owner, and every review that system's regulator or
   owner requires. The machines' own controls and protections stay in
    ↪ place; the kill switch must remain operable at all
   times.

11. EXPORT CONTROL, SANCTIONS AND LAWFUL USE

   You will comply with every export-control, sanctions, anti-corrupti
    ↪ on and other law that applies to the Software and
   to Your use of it, including the U.S. Export Administration Regulat
    ↪ ions and the sanctions administered by the U.S.
   Treasury's Office of Foreign Assets Control. You represent that You
    ↪  are not located in, organized under the laws of,
   or ordinarily resident in a country or region subject to comprehens
    ↪ ive U.S. sanctions, and that You are not on, or
   owned or controlled by a party on, any U.S. restricted-party list.
    ↪ You will not use the Software for any end use
   prohibited by those laws, including the design, development or prod
    ↪ uction of weapons of mass destruction.

12. DATA AND PRIVACY

   The Software contains no telemetry. It sends nothing to Licensor an
    ↪ d makes no network connection of its own except to
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
   the systems You point it at. Your data and Your Results stay on You
    ↪ r systems and are Yours, subject to section 4. If
   You send Licensor any material (for example in an issue or an e-mai
    ↪ l), You represent that You may lawfully do so and
   that it contains no personal data or confidential information of an
    ↪ y third party.

13. THIRD-PARTY COMPONENTS

   Files that carry a different license notice (for example tests/thir
    ↪ d_party/) and third-party data sets (for example
   the PlanetLab traces under fleet/traces/ and k8s_controlplane/trace
    ↪ s/) remain under their own terms
   (THIRD_PARTY_NOTICES.md). Third-party software the Software runs on
    ↪  or connects to (Kubernetes, KEDA, Karpenter,
   NVIDIA drivers, databases and others) is licensed by its owners und
    ↪ er their own terms.

14. TERM AND TERMINATION

   14.1 This license lasts until it ends under this section or section
    ↪  7.
   14.2 It ends automatically, without notice, if You breach it. If Yo
    ↪ ur breach is Your first, was inadvertent, and is
        not a breach of sections 3(a) to 3(g) or section 10, Your lice
    ↪ nse is reinstated if You cure the breach within
        thirty days after You learn of it or after Licensor notifies Y
    ↪ ou, whichever comes first.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
   14.3 Licensor may end this license for any person at any time by wr
    ↪ itten notice, and for all persons for future
        copies under section 9.
   14.4 When this license ends, You must stop all use of the Software,
    ↪  delete or destroy every copy and Derivative in Your
        possession or control, and, on request, certify in writing tha
    ↪ t You have done so.
   14.5 Sections 1, 3, 4(b) to 4(e), 6, 7, 8, 11, 12, 15 to 18 and 19
    ↪ survive the end of this license.

15. NO WARRANTY

   TO THE FULLEST EXTENT PERMITTED BY LAW, THE SOFTWARE, THE MECHANISM
    ↪  AND THE RESULTS ARE PROVIDED "AS IS" AND "AS
   AVAILABLE", WITH ALL FAULTS AND WITHOUT WARRANTY OR CONDITION OF AN
    ↪ Y KIND, EXPRESS, IMPLIED OR STATUTORY, INCLUDING ANY
   WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITL
    ↪ E, NON-INFRINGEMENT, ACCURACY, QUIET ENJOYMENT, OR
   THAT THE SOFTWARE WILL BE ERROR-FREE OR UNINTERRUPTED. NO RESULT IS
    ↪  A PROMISE OF ANY OUTCOME ON ANY OTHER SYSTEM;
   MODELLED RESULTS (EVIDENCE CLASS S) ARE STATEMENTS ABOUT THE MODEL,
    ↪  NOT MEASUREMENTS (DISCLOSURES.md). YOU USE THE
   SOFTWARE AT YOUR OWN RISK AND ARE RESPONSIBLE FOR ITS USE ON YOUR S
    ↪ YSTEMS.

16. LIMITATION OF LIABILITY

   TO THE FULLEST EXTENT PERMITTED BY LAW, LICENSOR, ITS MEMBERS, MANA
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
    ↪ GERS, EMPLOYEES, CONTRACTORS AND LICENSORS SHALL NOT
   BE LIABLE FOR ANY INDIRECT, INCIDENTAL, SPECIAL, CONSEQUENTIAL, EXE
    ↪ MPLARY OR PUNITIVE DAMAGES, OR FOR ANY LOSS OF
   PROFITS, REVENUE, SAVINGS, DATA, USE OR GOODWILL, OR ANY COST OF SU
    ↪ BSTITUTE GOODS OR SERVICES, ARISING OUT OF OR
   RELATING TO THE SOFTWARE, THE MECHANISM, THE RESULTS OR THIS LICENS
    ↪ E, UNDER ANY THEORY OF LIABILITY, EVEN IF ADVISED OF
   THEIR POSSIBILITY. LICENSOR'S TOTAL LIABILITY FOR ALL CLAIMS SHALL
    ↪ NOT EXCEED ONE HUNDRED U.S. DOLLARS (USD 100). THESE
   LIMITS APPLY EVEN IF A REMEDY FAILS OF ITS ESSENTIAL PURPOSE.

17. INDEMNITY

   You will defend, indemnify and hold harmless Licensor and its membe
    ↪ rs, managers, employees and contractors from and
   against every claim, loss, liability, damage, cost and expense (inc
    ↪ luding reasonable attorneys' fees) arising out of
   or relating to (a) Your use of the Software outside this license, (
    ↪ b) any Production Use, Commercial Use or Hosted
   Service without an Enterprise License, (c) Your Derivatives, (d) Yo
    ↪ ur Results as You publish them, or (e) Your
   violation of any law or third-party right.

18. U.S. GOVERNMENT END USERS

   The Software is "commercial computer software" and "commercial comp
    ↪ uter software documentation" as those terms are
   used in 48 C.F.R. 2.101, 12.212 and 227.7202. A U.S. Government end
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  user acquires only the rights set out in this
   license, consistent with 48 C.F.R. 12.212 and 227.7202-1 through 22
    ↪ 7.7202-4.

19. GENERAL

   19.1 Governing law. This license is governed by the laws of the Sta
    ↪ te of Delaware and the federal laws of the United
        States, without regard to conflict-of-laws rules. The United N
    ↪ ations Convention on Contracts for the
        International Sale of Goods does not apply.
   19.2 Venue. The state courts of the State of Delaware and the Unite
    ↪ d States District Court for the District of
        Delaware have exclusive jurisdiction over any dispute arising
    ↪ out of or relating to this license, and You consent
        to their personal jurisdiction. Licensor may seek
        injunctive or other equitable relief in any court of competent
    ↪  jurisdiction, without posting a bond, to protect
        its intellectual property or confidential information; a breac
    ↪ h of sections 3 or 10 would cause Licensor
        irreparable harm.
   19.3 Enforcement costs. In any action to enforce this license, the
    ↪ prevailing party may recover its reasonable
        attorneys' fees and costs.
   19.4 Assignment. You may not assign or transfer this license or any
    ↪  right under it, by operation of law or otherwise;
        any attempt is void. Licensor may assign this license freely.
   19.5 Notices. Notices to Licensor go to The Omni-Compass LLC by tel
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
    ↪ ephone at its main office infrastructure (toll-free direct 1-888
    ↪ -339-9551; system route asset 1-888-DEX-YLL-1) or through www.om
    ↪ ni-compass.com. Licensor may give notice to
        You through the Software's repository or at any address You ha
    ↪ ve given it.
   19.6 Severability and waiver. If any provision is held unenforceabl
    ↪ e, it is enforced to the maximum extent permitted and
        the rest of this license remains in effect. No failure or dela
    ↪ y in enforcing any provision is a waiver of it.
   19.7 Entire license. This license, with the NOTICE and DISCLOSURES.
    ↪ md, is the entire agreement between You and
        Licensor about its subject; for a licensee and its scope, a si
    ↪ gned Enterprise License governs where it differs.
   19.8 Interpretation. "Including" means "including without limitatio
    ↪ n". Headings are for convenience only. The
        English text of this license controls over any translation.
   19.9 Copies. Every copy, fork, export, report, archive or printout
    ↪ of any part of the Software must carry this
        LICENSE, the NOTICE and DISCLOSURES.md unchanged.

All rights reserved. Revision history: version 1.0 (evaluation and sim
    ↪ ulation only; first published 2026); revised
10 October 2026 (the notice in one wording everywhere); revised 11 Oct
    ↪ ober 2026 (definitions, patent license for
evaluation, hosted-service and machine-learning restrictions, results
    ↪ publication, patent and rights defense, safety,
export control, data, U.S. Government end users, term, cure, indemnity
    ↪ , Delaware governing law and venue).

Enterprise licensing: The Omni-Compass LLC, main office infrastructure
    ↪ , toll-free direct 1-888-339-9551, system route asset 1-888-DEX-
    ↪ YLL-1, www.omni-compass.com
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix G. Licensing: Questions and Answers

Plain answers to the questions buyers, engineers and their lawyers ask. The binding terms are in `LICENSE`; where this page and `LICENSE` differ, `LICENSE` governs. Every declaration is in `DISCLOSURES.md`.

**Is Omni-Compass open source?** No. The source is published so that it can be evaluated, reproduced and audited. It is licensed under the Omni-Compass Evaluation License 1.0 (`LicenseRef-OmniCompass-Evaluation-1.0`), which permits evaluation and simulation only.

**What may I do without a commercial license?** Evaluation and simulation, free of charge (`LICENSE`, sections 1.4, 1.5 and 2): read the code; run `verify.py`, the simulations and the benchmarks; reproduce the published results and publish your own, every row included (section 4); run Omni-Compass in watch, dry-run or shadow mode, or in a test environment, on systems you own or control, where it writes to nothing that serves production traffic or real users and to no physical process outside a test bench. A limited patent license comes with it, for evaluation only (section 2.2).

**What needs the Omni-Compass Enterprise License?** Every commercialization or monetization of any kind; production use; operating any system beyond evaluation; offering it, or its function, as a hosted, managed or software-as-a-service offering; redistributing it; incorporating it, or any part or derivative of it, into a product, service, appliance or firmware; using it or its results to build or train a competing product (`LICENSE`, sections 3 and 5). The Enterprise License is a written agreement signed by The Omni-Compass LLC and paid for; its standard terms are summarized in `ENTERPRISE_LICENSE.md` (production, hosted service, OEM and embedded, managed service provider, research).

**May I run it as a service for others?** Not without an Enterprise License with a hosted-service rider (`LICENSE`, section 3(c); `ENTERPRISE_LICENSE.md`, section 1).

**May I connect it to a grid, a plant, a vehicle or a medical device?** Not under the evaluation license: such systems may be studied only in simulation. A live connection to any physical system needs an Enterprise License and a written safety case accepted by the system's owner (`LICENSE`, section 10).

**How is the Enterprise License priced?** Against the measured gain on the customer's own paired receipts (native against Omni-Compass on the same system, the same load and the same clock). Terms are set in each signed agreement; no price stated anywhere in this repository is an offer.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**Are patents involved?** All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC (`PATENTS.md`). The evaluation license grants no patent license beyond evaluation.

**May I use the name or the compass rose?** Only to refer to Omni-Compass accurately (`TRADEMARKS.md`).

**May I contribute?** Contributions require the contributor license agreement (`.github/CLA.md`): every contribution is assigned to The Omni-Compass LLC, with a patent license from the contributor.

**Does Omni-Compass send data anywhere?** No. The software contains no telemetry and makes no network connection of its own except to the systems an operator points it at (a Kubernetes API, `nvidia-smi`, a meter command).

**What do scanners report?** The license is declared in machine-readable form for Black Duck, FOSSA, Snyk and REUSE (`pyproject.toml`, `REUSE.toml`, `.fossa.yml`, `.snyk`, `sbom/`); third-party components are listed in `THIRD_PARTY_NOTICES.md`.

**Can the terms change?** Yes. The Omni-Compass LLC reserves the right to change the software, its documentation, its results and these terms at any time, without notice; a changed version applies to copies obtained and uses made after it is published (`LICENSE`, section 9). A signed Enterprise License governs its own term.

**Which law governs?** The laws of the State of Delaware and the federal laws of the United States, with exclusive venue in the Delaware state courts and the United States District Court for the District of Delaware (`LICENSE`, section 19). The Omni-Compass LLC is based in Florida.

**Who do I contact?** The Omni-Compass LLC, main office infrastructure, toll-free direct 1-888-339-9551, system route asset 1-888-DEX-YLL-1, www.omni-compass.com.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*
**www.omni-compass.com**

APPENDIX

# Appendix H. Third-Party Notices

Omni-Compass uses the following third-party components. Each remains under its own license; nothing in the Omni-Compass Evaluation License changes those terms. Installed versions are those resolved from `requirements.txt`.

| Component | Use | License |
|---|---|---|
| NumPy | numerical arrays | BSD-3-Clause |
| Matplotlib | charts in the reports | Matplotlib License (PSF-based, BSD-compatible) |
| ReportLab (open-source edition) | the printable book and reports | BSD-3-Clause |
| Numba | compiled inner loops of the motion-axis plant | BSD-2-Clause |
| PyTorch (on GPU machines only, not installed by `requirements.txt`) | the GPU workload of the benchmark | BSD-3-Clause |
| `tests/third_party/hpa_independent.py` | an independently written HPA reference used unmodified in tests | its own terms, stated in the file |
| Liberation Serif and Liberation Sans fonts (embedded in the PDF book) | typesetting | SIL Open Font License 1.1 |
| DejaVu Sans and DejaVu Sans Mono fonts (embedded in the PDF book) | typesetting | Bitstream Vera / DejaVu license (free) |
| PlanetLab / CoMon VM CPU traces, March 2011 (`fleet/traces/`, `k8s_controlplane/traces/`; source in their `PROVENANCE.json`) | held-out demand for the modelled fleet | their own terms (public research data); no Omni-Compass copyright is claimed in them |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Component | Use | License |
|---|---|---|
| Google cluster-usage traces 2011 (`clusterdata-2011-2`), from which `results/traces/google2011/` is derived (`docs/TRACES_PREREGISTRATION.md`) | the public demand trace on the real cluster | Google's published terms for the traces; no Omni-Compass copyright is claimed in the trace itself |

Kubernetes, kind, `kubectl`, the NVIDIA driver and `nvidia-smi` are not distributed with Omni-Compass; it calls them where an operator has installed them. Their names are the property of their owners (`DISCLOSURES.md`, section 1).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

BACK MATTER

# Contact

Licensing, pilots, evaluations and the Omni-Compass Enterprise License:

**The Omni-Compass LLC**

Owner and developer: AJ Dubra

Main office infrastructure: toll-free direct 1-888-339-9551, system route asset 1-888-DEX-YLL-1

www.omni-compass.com

Repository: github.com/The-Omni-Compass-LLC/The-Omni-Compass

Any use beyond evaluation and simulation requires a signed, paid Omni-Compass Enterprise License. Write to The Omni-Compass LLC before any production use.

© 2026 The Omni-Compass LLC. All rights reserved. **Evaluation and simulation use only.** Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with `LICENSE`, `NOTICE` and `DISCLOSURES.md`. `SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0` All patents, copyrights and trademarks filed in the USA. www.omni-compass.com